

Post-Quantum Multilinear Polynomial Commitments from FRI Folding in the Boolean-Kernel Basis

Abdelali Mkhida
Algorizk Labs
`ali.mkhida@algorizk.xyz`
ORCID 0009-0009-2101-9070

September 2026

Abstract

Hash-based multilinear polynomial commitment schemes in the BaseFold paradigm are candidates for post-quantum security; they commit to a Reed–Solomon encoding and prove evaluations by interleaving a sumcheck with FRI-style folding. Over smooth multiplicative domains the classical fold acts on monomial coefficients, so a prover holding a table of values on the Boolean hypercube must first convert it to coefficient form. We introduce the *Boolean-kernel basis* $K_b(X) = \prod_{k=1}^n (X^{2^{k-1}} + b_k)$, $b \in \{0, 1\}^n$, of the univariate polynomials of degree $< 2^n$. Its change of basis to monomials is the Kronecker power of a 2×2 matrix and costs $\frac{n}{2}2^n$ additions or subtractions. In this basis a normalised FRI fold is *exactly* the restriction of one variable of the multilinear extension of the coordinate table, so folding the encoding of a table evaluates its multilinear extension directly. On this fold dictionary we build a transparent, hash-based multilinear polynomial commitment scheme and prove it sound, evaluation binding and round-by-round knowledge sound in the unique-decoding regime; the only external result about codes is the correlated-agreement theorem for Reed–Solomon codes. With the BCS theorem for interactive oracle reductions of Chiesa, Di, Hu and Zheng, the non-interactive scheme is knowledge sound, for a single opening, against quantum adversaries in the quantum random oracle model, with explicit bounds. A Rust implementation over the Goldilocks field matches the running time of a coefficient-form baseline sharing all other code; for 20 variables at 100 bits of security it proves an evaluation in 0.84s with a 237 KiB proof, about three times faster than the WHIR implementation on the same machine, whose proofs are about half as large. Companion Lean 4 files formalise the algebraic, probabilistic and soundness results, with correlated agreement as the only axiom.

Contents

1	Introduction	3
1.1	Contributions	3
1.2	Related work	5
1.3	Organisation	6
2	Preliminaries	6
2.1	Integers, bits and points	7
2.2	Fields and univariate polynomials	7
2.3	Multilinear polynomials	8
2.4	Smooth domains and Reed–Solomon codes	11

2.5	Proximity gaps	13
3	Interactive Oracle Proofs and Polynomial Commitments	13
3.1	Interactive oracle proofs	13
3.2	Round-by-round soundness	15
3.3	The sumcheck protocol	16
3.4	Polynomial commitment schemes	17
3.5	Non-interactive arguments	17
4	The Boolean-Kernel Basis	21
4.1	Kernel polynomials	22
4.2	Kronecker products	23
4.3	The basis theorem	23
4.4	Low degree in kernel coordinates	24
4.5	Evaluation as a scaled multilinear evaluation	25
5	The Fold Dictionary	26
5.1	Even-odd split in kernel coordinates	26
5.2	The kernel fold of a polynomial	26
5.3	The kernel fold of a word	27
5.4	Relation to classical FRI folding	28
6	The Polynomial Commitment Scheme	29
6.1	Parameters and encoding	29
6.2	The evaluation protocol	30
6.3	Committing to fewer words	31
6.4	Completeness	32
6.5	Costs	33
7	Soundness	34
7.1	Fibre distance and decoding	34
7.2	Two folding lemmas	35
7.3	Good challenges	36
7.4	The soundness theorem	36
7.5	The case $\ell = 0$	38
7.6	Round-by-round knowledge soundness	38
7.7	The non-interactive scheme	41
7.8	Committed levels	45
8	Implementation	46
8.1	Fields	47
8.2	Encoding and folding	47
8.3	Commitments and transcript	48
8.4	Testing	49

9 Experiments	50
9.1 Setup	50
9.2 Kernel versus coefficient-form encoding	51
9.3 Post-quantum parameters, salts and parallelism	51
9.4 Where the time goes	52
9.5 The number of folding rounds	52
9.6 The rate	53
9.7 Committed levels	53
9.8 An independent reference point	53
10 Conclusion and Future Work	54
10.1 Future work	54
10.2 Artefacts	55
A Notation	58
B Lean formalisation	61

1 Introduction

Many succinct arguments reduce the correctness of a computation to a claim about a multilinear polynomial evaluated at a random point. This is the case for sumcheck-based systems such as Spartan [Set20] and HyperPlonk [CBBZ23]. Such systems need a *multilinear polynomial commitment scheme*: the prover commits to a multilinear polynomial $\tilde{f}$ in n variables and later proves that $\tilde{f}(z) = v$ for a point z chosen by the verifier. When the scheme is built from hash functions and error-correcting codes, the resulting arguments are transparent and rely on no number-theoretic assumption, which makes them candidates for post-quantum security.

A multilinear polynomial can be given in two forms. In *table form* (also called evaluation or Lagrange form) it is given by its values $f(b)$ on the Boolean hypercube $B_n = \{0, 1\}^n$; this is the form in which provers naturally hold their witnesses. In *coefficient form* it is given by its monomial coefficients. The two forms are related by Möbius inversion on the Boolean lattice.

Code-based schemes in the BaseFold paradigm [ZCF24] commit to an encoding of f and prove an evaluation by running a sumcheck whose round challenges also drive an FRI-like folding of the codeword [BSBHR18]. With Reed–Solomon codes on smooth multiplicative subgroups, as in DeepFold [GLH⁺25], the classical FRI fold $U \mapsto U_e + \theta U_o$ acts on coefficient vectors. It computes the multilinear evaluation of the *coefficient* vector: the monomial coefficients of the committed univariate polynomial must be the monomial coefficients of $\tilde{f}$. A prover that holds a table must therefore first move to coefficient form, and must keep the two views consistent across the committed word, the sumcheck state and the folds.

This paper asks whether there is a basis of univariate polynomials in which FRI-style folding over a smooth multiplicative domain acts *directly* on the table form. There is, and it is explicit.

1.1 Contributions

The Boolean-kernel basis (Section 4). For $y \in \mathbb{F}_q^n$ let $K_y(X) = \prod_{k=1}^n (X^{2^{k-1}} + y_k)$. We show that the $N = 2^n$ polynomials K_b , $b \in B_n$, which have coefficients in $\{0, 1\}$, form a basis of $\mathbb{F}[X]_{<N}$ over every field $\mathbb{F}$. The change of basis to monomials is the Kronecker power $C^{\otimes n}$ of $C = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$. It costs exactly $\frac{n}{2}N$ additions or subtractions in either direction, and no multiplications. In this

$$\begin{array}{ccc}
\lambda \in \mathbb{F}[[0, N-1]] & \xrightarrow{\text{res}_r} & \text{res}_r(\lambda) \in \mathbb{F}[[0, N/2-1]] \\
\downarrow \sum_b \lambda(b) K_b^{(n)} & & \downarrow \sum_{b'} \text{res}_r(\lambda)(b') K_{b'}^{(n-1)} \\
U \in \mathbb{F}[X]_{<N} & \xrightarrow{\text{pfold}_r} & \text{pfold}_r(U) \in \mathbb{F}[X]_{<N/2} \\
\downarrow \text{ev}_L & & \downarrow \text{ev}_{L^2} \\
w \in \mathbb{F}^L & \xrightarrow{\text{fold}_r} & \text{fold}_r(w) \in \mathbb{F}^{L^2}
\end{array}$$

Figure 1: The fold dictionary. Top square: in kernel coordinates, the polynomial fold is the restriction of the first variable of the table (Theorem 5.3). Bottom square: on a smooth domain L of order $M \geq N$, the word fold is the polynomial fold evaluated on L^2 (Proposition 5.8). Iterating the whole diagram n times sends the codeword of a table f to the constant $\tilde{f}(r_1, \dots, r_n)$ (Corollary 5.9).

basis, having degree $< 2^t$ is a sign condition on the coordinates (Theorem 4.13). For all but at most $N - 1$ points ζ , evaluating U at ζ equals, up to an explicit nonzero scalar, evaluating the multilinear extension of its coordinates at a point on an explicit curve (Proposition 4.15).

The fold dictionary (Section 5). Our main structural result is Theorem 5.3. Let $U = \sum_b \lambda(b) K_b$ and $U = U_e(X^2) + XU_o(X^2)$. Then the kernel coordinates of the fold

$$\text{pfold}_r(U) = (2r - 1)U_e + (1 - r)U_o$$

are exactly $(1 - r)\lambda(2b') + r\lambda(2b' + 1)$, the restriction of the first variable of the table λ . Consequently, encoding a table f as the Reed–Solomon codeword of $U_f = \sum_b f(b) K_b$ and folding it n times with challenges $r_1, \dots, r_n$ yields the constant $\tilde{f}(r_1, \dots, r_n)$ (Corollary 5.9), with no change of representation at any point. Figure 1 summarises the dictionary.

The kernel fold agrees with the classical FRI fold up to a nonzero scalar and a reparametrisation of the challenge (Section 5.4); mixing the two parametrisations breaks correctness (Example 5.13), a pitfall that implementations must avoid.

A multilinear PCS with a self-contained soundness proof (Sections 6 and 7). We instantiate the BaseFold paradigm with the kernel encoding and prove it sound in the unique-decoding regime (Theorem 7.13). The only external ingredient about codes in the soundness proof is the correlated-agreement theorem for Reed–Solomon codes of Ben-Sasson, Carmon, Ishai, Kopparty and Saraf [BSCI⁺23]; the efficiency of the extractor also uses polynomial-time unique decoding of Reed–Solomon codes. The theorem enters through a one-line observation: the kernel fold of any word is an affine line $u^0 + r u^1$ whose generators determine the word (Lemma 5.7). The proof tracks the set of passing query chains level by level, and it yields evaluation binding (Corollary 7.14). The same argument gives round-by-round knowledge soundness with an explicit decoding extractor (Theorem 7.19). The BCS theorem in the quantum random oracle model of Chiesa, Manohar and Spooner [CMS19] does not cover instances containing a Merkle-committed oracle, such as the commitment. Chiesa, Di, Hu and Zheng [CDHZ25] extend the BCS transformation to interactive oracle reductions, whose instances may contain such oracles, under a relaxed round-by-round knowledge soundness hypothesis. We derive this hypothesis from Theorem 7.19 (Theorem 7.25); their theorem then makes the non-interactive scheme knowledge sound, for a single opening, against quantum adversaries that choose the commitment, with explicit concrete bounds (Theorem 3.16 and Corollary 7.26); Remark 7.28 gives parameters for this setting.

Implementation and experiments (Sections 8 and 9). We give a Rust implementation over the Goldilocks field, with the quadratic extension for the classical parameters and the quartic extension for the post-quantum ones, compiled with salted Merkle trees as in [CDHZ25]. It commits to one folded word in four and merges authentication paths; a reduction lemma (Lemma 6.5 and Theorem 7.31) transfers every soundness result to this variant with the same errors. We compare it with a coefficient-form baseline that shares all other code: the two encodings are indistinguishable in running time at the resolution of our machine, as the operation counts predict, so the kernel encoding provides its structural property at no measurable cost. We also report the cost of the post-quantum parameters, and place the implementation next to the WHIR implementation on the same machine: at $n = 20$, its prover is about three times faster, with proofs about twice as large.

Formal verification (Appendix B). Companion Lean 4 files, of about 7200 lines with no `sorry`, formalise the basis theorem, the fold dictionary, completeness, soundness, evaluation binding and round-by-round knowledge soundness. Correlated agreement (Theorem 2.23) is the only axiom; the algebraic results and completeness do not use it.

Scope. The evaluation protocol is BaseFold’s sumcheck-plus-folding applied to the kernel encoding, so its asymptotic and concrete costs are those of Reed–Solomon instantiations of BaseFold and DeepFold (Remark 6.8); what the kernel encoding adds is that the prover works on the table form throughout. The soundness analysis is in the unique-decoding regime and concerns one commitment opened at one point. The post-quantum statement (Corollary 7.26) applies to the compiler of [CDHZ25] (Remark 7.30), whose structure the implementation follows, with the quartic extension of the Goldilocks field as challenge field. List decoding, batched and repeated openings, and zero knowledge are the subject of Section 10.1.

1.2 Related work

Proximity tests for Reed–Solomon codes. FRI [BSBHR18] tests proximity to Reed–Solomon codes by repeated even–odd folding. Its soundness in the unique-decoding and list-decoding regimes rests on proximity gaps for Reed–Solomon codes [BSCI⁺23]. WHIR [ACFY24] tests proximity to constrained Reed–Solomon codes with very fast verification and supports multilinear evaluation claims. Our soundness proof uses only the unique-decoding correlated-agreement theorem of [BSCI⁺23]; extending it to the list-decoding regime is the most natural next step (Section 10.1).

Multilinear commitments from folding. BaseFold [ZCF24] introduced foldable codes and the interleaving of sumcheck rounds with folding rounds under shared challenges. Our evaluation protocol follows it directly. DeepFold [GLH⁺25] instantiates the paradigm with Reed–Solomon codes and improves proof size; its folding computes coefficient-form evaluations, and our coefficient-form baseline of Section 9 follows the same principle. Haböck [Hab24] analyses BaseFold over Reed–Solomon codes up to the Johnson bound. Zeilberger [Zei26] proves proximity gaps for multilinear evaluation for all linear codes. Both results concern the soundness of the folding paradigm; our contribution is orthogonal and concerns the encoding.

Binary fields and the novel polynomial basis. FRI-Binius [DP24] adapts BaseFold to binary towers using the novel polynomial basis of Lin, Chung and Han [LCH14]. In that basis the additive FFT, and with it FRI-style folding over additive subspaces, interacts cleanly with multilinear structure. The Boolean-kernel basis plays an analogous role for smooth *multiplicative* subgroups in

odd characteristic. The two constructions are genuinely different. In characteristic 2 every factor $X^{2^{k-1}} + 1$ equals $(X + 1)^{2^{k-1}}$, so $K_b = X^{N-1-b}(X + 1)^b$ is a Bernstein-type basis; moreover the multiplicative group of a field of characteristic 2 has odd order, so smooth multiplicative domains do not exist, and the even–odd formulas (1) divide by 2.

Multilinear-to-univariate reductions. A different line of work proves multilinear evaluations with a *univariate* polynomial commitment, typically KZG. Gemini [BCHO22] reduces a coefficient-form multilinear claim to univariate claims through recursive even–odd splitting. Zeromorph [KT24] works in table form and uses quotients and degree checks. Mercury [EG25] reaches constant proof size and expresses the evaluation as a coefficient of a product with a tensor-structured polynomial. Samaritan [GPS25] improves on Gemini in proof size, prover cost and verifier cost. Mohamed [Moh25] relates univariate and multilinear sumchecks through the inverse Kronecker substitution. These schemes treat the univariate commitment as a black box. In ours, the univariate polynomial is committed through a Reed–Solomon code, and the basis is chosen so that the code’s own folding carries the multilinear structure.

Other domains. Brakedown [GLS⁺23] and related schemes avoid FFTs by using linear-time encodable codes. Circle STARKs [HLP24] obtain smooth folding domains over the Mersenne prime $2^{31} - 1$ from the circle curve. The fold dictionary as stated needs a smooth multiplicative subgroup in odd characteristic. Whether a Boolean-kernel-type basis exists for circle domains and for additive domains is an open question (Section 10.1).

1.3 Organisation

Section 2 fixes notation and collects the facts on multilinear polynomials and Reed–Solomon codes that we use, including the correlated-agreement theorem. Section 3 recalls interactive oracle proofs and their probability space, round-by-round (knowledge) soundness, the sumcheck protocol, polynomial commitments, and the definitions and the theorem of [CDHZ25] on the BCS transformation for interactive oracle reductions, which we use for the non-interactive scheme. Section 4 develops the Boolean-kernel basis, and Section 5 proves the fold dictionary. Section 6 defines the commitment scheme and Section 7 proves its soundness. Sections 8 and 9 describe the implementation and the measurements, and Section 10 describes future work. Appendix A collects the notation, and Appendix B describes the Lean formalisation.

The paper is self-contained: every notion used in the proofs is defined, every result specific to the paper is proved, and the external results are stated in Sections 2 and 3. The post-quantum corollary uses the quantum extraction model of [CDHZ25], whose definitions we restate as far as the statement requires.

2 Preliminaries

This section fixes notation and collects the algebraic facts used throughout. Every result specific to this paper is proved. Standard facts from algebra and coding theory are quoted with a reference and labelled *Fact*; the correlated-agreement theorem of Section 2.5 is the only external result about codes used in the soundness analysis, and Fact 2.20 is used only for the efficiency of the extractor. Appendix A lists the notation in one table.

2.1 Integers, bits and points

Integers. $\mathbb{N} = \{0, 1, 2, \dots\}$. For integers a and c we write $\llbracket a, c \rrbracket = \{i \in \mathbb{Z} : a \leq i \leq c\}$, which is empty if $a > c$.

Number of variables. The letter $n \in \mathbb{N}$ denotes a *number of variables*, and $N = 2^n$. Unless a hypothesis states otherwise, every statement of Sections 2, 4 and 5 holds for every $n \geq 0$, and is applied with various numbers of variables (for instance $n - 1$ or $n - j$); when the number of variables must be explicit, we write it as a superscript, as in $K_b^{(n)}$ (Definition 4.1).

Binary decomposition. Every $b \in \llbracket 0, N - 1 \rrbracket$ has a unique *binary decomposition*

$$b = \sum_{k=1}^n b_k 2^{k-1}, \quad b_k \in \{0, 1\},$$

and $(b_1, \dots, b_n)$ is its *bit vector*; bit b_1 is the least significant one. Writing $B_n = \{0, 1\}^n$ for the *Boolean hypercube*, the map $b \mapsto (b_1, \dots, b_n)$ is a bijection $\llbracket 0, N - 1 \rrbracket \rightarrow B_n$, and we identify b with its bit vector. For a commutative ring A with $1 \neq 0$, we also regard B_n as a subset of A^n , via $\{0, 1\} \subseteq A$. For $b, c \in \llbracket 0, N - 1 \rrbracket$:

- $\text{wt}(b) = \sum_{k=1}^n b_k$ is the *Hamming weight* of b ;
- $\bar{b} = (N - 1) - b$ is the *complement* of b , with bits $\bar{b}_k = 1 - b_k$;
- $c \preceq b$ means $c_k \leq b_k$ for all $k \in \llbracket 1, n \rrbracket$; we then say that c is a *submask* of b .

Splitting indices. Let $t \in \llbracket 0, n \rrbracket$. Every $b \in \llbracket 0, N - 1 \rrbracket$ can be written uniquely as $b = c + 2^t h$ with $c \in \llbracket 0, 2^t - 1 \rrbracket$ and $h \in \llbracket 0, 2^{n-t} - 1 \rrbracket$, namely $c = b \bmod 2^t$ and $h = \lfloor b/2^t \rfloor$. The bits are $c_k = b_k$ for $k \in \llbracket 1, t \rrbracket$ and $h_k = b_{t+k}$ for $k \in \llbracket 1, n - t \rrbracket$. For $t = 1$ we write $b = b_1 + 2b'$, with $b' = \lfloor b/2 \rfloor$ of bit vector $(b_2, \dots, b_n)$. When a letter is declared to denote an index, its subscripts denote its bits.

Points. Let A be a commutative ring and $j \in \llbracket 0, n \rrbracket$. For $x = (x_1, \dots, x_j) \in A^j$ and $y = (y_1, \dots, y_{n-j}) \in A^{n-j}$, we write $(x, y) = (x_1, \dots, x_j, y_1, \dots, y_{n-j}) \in A^n$. For $x \in A^j$ and an index $b \in \llbracket 0, 2^{n-j} - 1 \rrbracket$, (x, b) is the point $(x_1, \dots, x_j, b_1, \dots, b_{n-j})$, using the bit vector of b with $n - j$ bits. For $r = (r_1, \dots, r_m) \in A^m$ and $j \in \llbracket 0, m \rrbracket$, we write $r_{\leq j} = (r_1, \dots, r_j)$.

2.2 Fields and univariate polynomials

Throughout the paper, $\mathbb{F}_q$ is a finite field with q elements and *odd characteristic*, so that 2 is invertible in $\mathbb{F}_q$. All results of Sections 2 and 4 to 7 hold for every such field; in Section 7 they are also applied to an extension field $\mathbb{K} \supseteq \mathbb{F}_q$.

For a field $\mathbb{F}$, $\mathbb{F}[X]$ is the ring of univariate polynomials over $\mathbb{F}$. For $d \in \mathbb{N}$, $\mathbb{F}[X]_{<d}$ is the $\mathbb{F}$ -vector space of polynomials of degree less than d (with $\deg 0 = -\infty$); it has dimension d . For $P \in \mathbb{F}[X]$ and $a \in \mathbb{N}$, $[X^a]P$ denotes the coefficient of X^a in P .

Lemma 2.1 (Root counting). *Let $\mathbb{F}$ be a field. A nonzero polynomial $P \in \mathbb{F}[X]$ of degree d has at most d roots in $\mathbb{F}$. Consequently, two polynomials of $\mathbb{F}[X]_{<d}$ that agree on d distinct points of $\mathbb{F}$ are equal.*

Proof. If $P(a) = 0$, Euclidean division by $X - a$ gives $P = (X - a)Q$ with $\deg Q = d - 1$, and every root of P other than a is a root of Q , because $\mathbb{F}$ has no zero divisors. Induction on d gives the first statement. For the second, apply it to the difference, which has degree $< d$. $\square$

Fact 2.2 ([LN97, Theorem 2.8]). *The multiplicative group $\mathbb{F}^\times$ of a finite field $\mathbb{F}$ is cyclic.*

Lemma 2.3 (Splitting polynomials). *Let $\mathbb{F}$ be a field.*

1. (Even-odd split.) *Let $d \geq 1$. Every $P \in \mathbb{F}[X]_{<2d}$ can be written uniquely as*

$$P(X) = P_e(X^2) + X P_o(X^2), \quad P_e, P_o \in \mathbb{F}[X]_{<d},$$

namely with $[X^i] P_e = [X^{2i}] P$ and $[X^i] P_o = [X^{2i+1}] P$ for $i \in \llbracket 0, d-1 \rrbracket$.

2. (Block split.) *Let $t \in \mathbb{N}$ and $m \geq 1$. Every $P \in \mathbb{F}[X]_{<2^t m}$ can be written uniquely as*

$$P(X) = \sum_{i=0}^{m-1} X^{2^t i} R_i(X), \quad R_i \in \mathbb{F}[X]_{<2^t},$$

namely with $[X^a] R_i = [X^{2^t i + a}] P$ for $a \in \llbracket 0, 2^t - 1 \rrbracket$.

Both maps $P \mapsto (P_e, P_o)$ and $P \mapsto (R_0, \dots, R_{m-1})$ are $\mathbb{F}$ -linear.

Proof. (1) Each monomial X^a with $a < 2d$ appears on the right-hand side exactly once: as $(X^2)^i$ in $P_e(X^2)$ if $a = 2i$, and as $X(X^2)^i$ in $X P_o(X^2)$ if $a = 2i + 1$. Comparing coefficients gives existence and uniqueness. (2) Likewise, every $a < 2^t m$ is written uniquely as $a = 2^t i + a'$ with $i \in \llbracket 0, m-1 \rrbracket$ and $a' \in \llbracket 0, 2^t - 1 \rrbracket$ (Euclidean division), and $X^{a'}$ appears exactly once, in $X^{2^t i} R_i(X)$. Linearity is clear from the coefficient formulas. $\square$

2.3 Multilinear polynomials

In this subsection, A is a commutative ring. A polynomial in $A[x_1, \dots, x_n]$ is *multilinear* if its degree in each variable x_k is at most 1. The multilinear polynomials form an A -module, which we denote by $\mathcal{L}_n(A)$, and we write $\mathcal{L}_n = \mathcal{L}_n(\mathbb{F}_q)$. Since the monomials form a basis of $A[x_1, \dots, x_n]$, the multilinear monomials $\prod_{k=1}^n x_k^{a_k}$, $a \in \llbracket 0, N-1 \rrbracket$, form a basis of $\mathcal{L}_n(A)$; in particular, for $n \geq 1$ every $P \in \mathcal{L}_n(A)$ can be written uniquely as $P = P_0 + x_n P_1$ with $P_0, P_1 \in \mathcal{L}_{n-1}(A)$ in the variables $x_1, \dots, x_{n-1}$. A polynomial with coefficients in A can be evaluated at the points of A^n for every commutative ring $A' \supseteq A$.

Definition 2.4 (Monomials, equality polynomial, Lagrange polynomials). For $a \in \llbracket 0, N-1 \rrbracket$, the *canonical monomial* is $m_a(x) = \prod_{k=1}^n x_k^{a_k}$. The *equality polynomials* are

$$\text{eq}_1(y, z) = yz + (1-y)(1-z), \quad \text{eq}(y, z) = \prod_{k=1}^n \text{eq}_1(y_k, z_k),$$

in the variables $y = (y_1, \dots, y_n)$ and $z = (z_1, \dots, z_n)$. For $b \in \llbracket 0, N-1 \rrbracket$, the *Lagrange polynomial* is $e_b(x) = \text{eq}(b, x) = \prod_{k=1}^n (b_k x_k + (1-b_k)(1-x_k))$.

Lemma 2.5 (Equality polynomial). 1. *For $b, c \in B_n$, $\text{eq}(b, c) = 1$ if $b = c$, and $\text{eq}(b, c) = 0$ otherwise.*

2. *For every commutative ring $A' \supseteq A$ and $z \in A'^n$, the polynomial $x \mapsto \text{eq}(x, z)$ is a multilinear polynomial with coefficients in A' , and $\text{eq}(x, z) = \text{eq}(z, x)$.*

3. Let $j \in \llbracket 0, n \rrbracket$, let x and z be points with j coordinates, and let x' and z' be points with $n - j$ coordinates. Then

$$\text{eq}((x, x'), (z, z')) = \text{eq}(x, z) \text{eq}(x', z'),$$

where the right-hand side uses j and $n - j$ variables.

4. $\text{eq}_1(0, r) = 1 - r$ and $\text{eq}_1(1, r) = r$ for every $r \in A$.

Proof. (1) Each factor $\text{eq}_1(b_k, c_k)$ equals 1 if $b_k = c_k$ and 0 otherwise. (2) Each factor $\text{eq}_1(x_k, z_k)$ has degree ≤ 1 in x_k and does not involve the other variables; and eq_1 is symmetric. (3) and (4) are immediate from the definition. $\square$

Proposition 2.6 (Multilinear extension). *A table over A is a function $f : \llbracket 0, N - 1 \rrbracket \rightarrow A$. For every table f there is a unique $\tilde{f} \in \mathcal{L}_n(A)$ with $\tilde{f}(b) = f(b)$ for all $b \in B_n$, namely*

$$\tilde{f}(x) = \sum_{b=0}^{N-1} f(b) e_b(x) = \sum_{b=0}^{N-1} f(b) \text{eq}(b, x).$$

It is the multilinear extension of f . The family $(e_b)_{b \in \llbracket 0, N-1 \rrbracket}$ is a basis of the A -module $\mathcal{L}_n(A)$, and $f \mapsto \tilde{f}$ is an A -linear bijection from the tables over A onto $\mathcal{L}_n(A)$.

Proof. By Lemma 2.5, the displayed polynomial is multilinear (item 2) and interpolates f (item 1). For uniqueness, it suffices to show that a multilinear P vanishing on B_n is zero, which we prove by induction on n . For $n = 0$, P is a constant and B_0 has one point. For $n \geq 1$, write $P = P_0 + x_n P_1$ as above. Setting $x_n = 0$ shows that P_0 vanishes on B_{n-1} , so $P_0 = 0$; setting $x_n = 1$ then shows that P_1 vanishes on B_{n-1} , so $P_1 = 0$. For the basis statement: if $P \in \mathcal{L}_n(A)$, then $P = \sum_b P(b) e_b$ is multilinear and vanishes on B_n , hence is zero, so the e_b generate $\mathcal{L}_n(A)$; and if $\sum_b a_b e_b = 0$, evaluating at $c \in B_n$ gives $a_c = 0$. $\square$

If f takes values in a subring $A_0 \subseteq A$, the formula of Proposition 2.6 shows that the multilinear extension of f over A is its multilinear extension over A_0 , with coefficients viewed in A ; we write $\tilde{f}$ in both cases.

Corollary 2.7 (Multilinear extension of the equality table). *For every commutative ring $A' \supseteq A$ and $z \in A'^n$, the multilinear extension of the table $b \mapsto \text{eq}(b, z)$ over A' is the polynomial $x \mapsto \text{eq}(x, z)$.*

Proof. By Lemma 2.5(2) the polynomial $x \mapsto \text{eq}(x, z)$ is multilinear, and it takes the value $\text{eq}(b, z)$ at $b \in B_n$; conclude by the uniqueness in Proposition 2.6. $\square$

Definition 2.8 (Coefficient form). The *coefficient form* of $\tilde{f}$ is the family $(\alpha_a)_{a \in \llbracket 0, N-1 \rrbracket}$ of its coefficients: $\tilde{f} = \sum_a \alpha_a m_a$. The table f itself is the family of coordinates of $\tilde{f}$ in the Lagrange basis; we call it the *table form*.

Lemma 2.9 (Table form $\leftrightarrow$ coefficient form). *With the notation of Definition 2.8,*

$$\begin{aligned} f(b) &= \sum_{\substack{a \in \llbracket 0, N-1 \rrbracket \\ a \preceq b}} \alpha_a && \text{for all } b \in \llbracket 0, N-1 \rrbracket, \\ \alpha_a &= \sum_{\substack{b \in \llbracket 0, N-1 \rrbracket \\ b \preceq a}} (-1)^{\text{wt}(a) - \text{wt}(b)} f(b) && \text{for all } a \in \llbracket 0, N-1 \rrbracket. \end{aligned}$$

Proof. For $b \in B_n$ we have $m_a(b) = \prod_{k:a_k=1} b_k$, which equals 1 if $a \preceq b$ and 0 otherwise; evaluating $\tilde{f} = \sum_a \alpha_a m_a$ at b gives the first identity. For the second, substitute the first into its right-hand side:

$$\sum_{b \preceq a} (-1)^{\text{wt}(a) - \text{wt}(b)} \sum_{c \preceq b} \alpha_c = \sum_{c \preceq a} \alpha_c \sum_{c \preceq b \preceq a} (-1)^{\text{wt}(a) - \text{wt}(b)}.$$

The inner sum runs over the subsets of the $\text{wt}(a) - \text{wt}(c)$ positions where a has a 1 and c has a 0; it equals $(1 - 1)^{\text{wt}(a) - \text{wt}(c)}$, which is 1 if $c = a$ and 0 otherwise. $\square$

Definition 2.10 (Restriction). Let $n \geq 1$, let f be a table over A with n variables, and let $r \in A$. The *restriction* of f at r is the table $\text{res}_r(f)$ over A with $n - 1$ variables defined by

$$\text{res}_r(f)(b') = (1 - r)f(2b') + rf(2b' + 1), \quad b' \in \llbracket 0, N/2 - 1 \rrbracket.$$

For $j \in \llbracket 0, n \rrbracket$ and $r = (r_1, \dots, r_j) \in A^j$, the *iterated restriction* $\text{res}_r(f)$ is defined by $\text{res}_\emptyset(f) = f$ and $\text{res}_{(r_1, \dots, r_j)}(f) = \text{res}_{r_j}(\text{res}_{(r_1, \dots, r_{j-1})}(f))$; it is a table with $n - j$ variables. For $j = 1$ the two definitions agree, identifying A with A^1 .

The restriction is A -linear in f .

Lemma 2.11 (Restriction evaluates the first variables). *Let f be a table over A with n variables.*

1. *If $n \geq 1$ and $r \in A$, then, as polynomials in $y = (y_1, \dots, y_{n-1})$,*

$$\widetilde{\text{res}_r(f)}(y) = \tilde{f}(r, y).$$

2. *For $j \in \llbracket 0, n \rrbracket$ and $r \in A^j$, $\widetilde{\text{res}_r(f)}(y) = \tilde{f}(r, y)$ as polynomials in $y = (y_1, \dots, y_{n-j})$. In particular $\text{res}_r(f)(b) = \tilde{f}(r, b)$ for all $b \in \llbracket 0, 2^{n-j} - 1 \rrbracket$, and for $j = n$ the table $\text{res}_r(f)$ has the single entry $\tilde{f}(r)$.*

Proof. (1) Write $b = b_1 + 2b'$. By Lemma 2.5(3), $e_b(x_1, y) = \text{eq}_1(b_1, x_1) e_{b'}(y)$, where $e_{b'}$ has $n - 1$ variables. Substituting $x_1 = r$, using Lemma 2.5(4) and grouping the terms $b_1 = 0$ and $b_1 = 1$,

$$\tilde{f}(r, y) = \sum_{b'=0}^{N/2-1} ((1 - r)f(2b') + rf(2b' + 1)) e_{b'}(y) = \sum_{b'} \text{res}_r(f)(b') e_{b'}(y),$$

which is $\widetilde{\text{res}_r(f)}(y)$ by Proposition 2.6. (2) Induction on j ; the case $j = 0$ is trivial. For $j \geq 1$, by (1) applied to $\text{res}_{r_{\leq j-1}}(f)$ and the induction hypothesis, $\widetilde{\text{res}_r(f)}(y) = \widetilde{\text{res}_{r_{\leq j-1}}(f)}(r_j, y) = \tilde{f}(r_{\leq j-1}, r_j, y)$. Evaluating at $y = b \in B_{n-j}$ gives the second statement, since the multilinear extension interpolates its table. $\square$

Lemma 2.12 (Naturality). *Let $\vartheta : A \rightarrow A'$ be a homomorphism of commutative rings, applied entrywise to tables and points and coefficientwise to polynomials. For every table f over A , $\vartheta(\tilde{f}) = \vartheta \circ \tilde{f}$; and for $j \in \llbracket 0, n \rrbracket$ and $r \in A^j$, $\vartheta \circ \text{res}_r(f) = \text{res}_{\vartheta(r)}(\vartheta \circ f)$.*

Proof. Both $\tilde{f} = \sum_b f(b) e_b$ and $\text{res}_r(f)$ are given by polynomial formulas with integer coefficients in the entries of f , of r and of the variables, and ϑ commutes with such formulas. $\square$

Remark 2.13 (Formal variables and extension fields). Lemma 2.11 holds over an arbitrary commutative ring A . Taking $A = \mathbb{F}_q[X]$ and $r = X$ shows that, for a table f over $\mathbb{F}_q$, the polynomial $\tilde{f}(X, y)$ is the multilinear extension, in y , of the table $\text{res}_X(f)$ over $\mathbb{F}_q[X]$; applying Lemma 2.12 to the evaluation homomorphism $\mathbb{F}_q[X] \rightarrow \mathbb{F}_q$, $X \mapsto r$, recovers $\text{res}_r(f)$ and $\tilde{f}(r, y)$. Taking $A = \mathbb{K}$ covers challenges from an extension field.

2.4 Smooth domains and Reed–Solomon codes

Definition 2.14 (Smooth domain). A *smooth domain* is a subgroup $L \leq \mathbb{F}_q^\times$ of order $M = 2^\mu$ with $\mu \in \mathbb{N}$. For $j \in \llbracket 0, \mu \rrbracket$, we write $L^{2^j} = \{\xi^{2^j} : \xi \in L\}$ for the image of the 2^j -th power map. In particular L^2 is the image of the squaring map; it is not a Cartesian power.

Lemma 2.15 (Power maps on smooth domains). *Let $\mu \in \mathbb{N}$.*

1. $\mathbb{F}_q^\times$ has a subgroup of order 2^μ if and only if $2^\mu \mid q-1$; it is then unique and cyclic.
2. Let L be a smooth domain of order $M = 2^\mu$ and $j \in \llbracket 0, \mu \rrbracket$. The map $\xi \mapsto \xi^{2^j}$ is a group homomorphism from L onto L^{2^j} , which is the smooth domain of order $M/2^j$, and every element of L^{2^j} has exactly 2^j preimages in L .
3. $(L^{2^j})^2 = L^{2^{j+1}}$ for $j \in \llbracket 0, \mu-1 \rrbracket$.
4. If $\mu \geq 1$, then $-1 \in L$, $-1 \neq 1$, and the preimages of $\eta = \xi^2 \in L^2$ under squaring are exactly ξ and $-\xi$, which are distinct.

Proof. (1) A subgroup of order e of a group of order $q-1$ exists only if $e \mid q-1$ (Lagrange's theorem). Conversely, let $e \mid q-1$. By Fact 2.2, $\mathbb{F}_q^\times$ is generated by some ω of order $q-1$, and $\omega^{(q-1)/e}$ generates a cyclic subgroup of order e . Every subgroup of order e is contained in the set of roots of $X^e - 1$, which has at most e elements by Lemma 2.1; so there is exactly one such subgroup. (2) The power map is a homomorphism because L is commutative. If ω generates L , then ω^{2^j} generates the image and has order $2^{\mu-j}$; by (1) the image is the smooth domain of that order. The preimages of an element form a coset of the kernel, which has order $|L|/|L^{2^j}| = 2^j$. (3) $(\xi^{2^j})^2 = \xi^{2^{j+1}}$. (4) The kernel of squaring has order 2 by (2) and consists of roots of $X^2 - 1 = (X-1)(X+1)$; since the characteristic is odd, $-1 \neq 1$, so the kernel is $\{1, -1\}$. The preimages of ξ^2 form the coset $\{\xi, -\xi\}$, and $\xi \neq -\xi$ because $2\xi \neq 0$. $\square$

Definition 2.16 (Fibres). Let L be a smooth domain of order $M \geq 2$. The *fibre* over $\eta \in L^2$ is the set $\{\xi, -\xi\}$ of its two square roots in L (Lemma 2.15(4)). The $M/2$ fibres partition L .

Definition 2.17 (Reed–Solomon code). Let $\mathbb{F} \supseteq \mathbb{F}_q$ be a finite field, let L be a smooth domain of order M , and let $1 \leq d \leq M$. The *evaluation map* is $\text{ev}_L : \mathbb{F}[X] \rightarrow \mathbb{F}^L$, $P \mapsto (P(\xi))_{\xi \in L}$, and the *Reed–Solomon code* of degree bound d on L over $\mathbb{F}$ is

$$\text{RS}_{\mathbb{F}}[L, d] = \text{ev}_L(\mathbb{F}[X]_{<d}) \subseteq \mathbb{F}^L.$$

We write $\text{RS}[L, d]$ when $\mathbb{F} = \mathbb{F}_q$. The *rate* is $\rho = d/M$. Elements of $\mathbb{F}^L$ are *words*, and elements of $\text{RS}_{\mathbb{F}}[L, d]$ are *codewords*. For $w, w' \in \mathbb{F}^L$, the *relative Hamming distance* is $\Delta(w, w') = |\{\xi \in L : w(\xi) \neq w'(\xi)\}|/M$; for a nonempty set $\mathcal{C} \subseteq \mathbb{F}^L$ we put $\Delta(w, \mathcal{C}) = \min_{c \in \mathcal{C}} \Delta(w, c)$. We say that w and w' *agree* on a set $S \subseteq L$ if $w(\xi) = w'(\xi)$ for all $\xi \in S$. For finite fields $\mathbb{F} \subseteq \mathbb{F}'$ we regard $\mathbb{F}^L \subseteq \mathbb{F}'^L$ entrywise; then ev_L commutes with this inclusion and $\text{RS}_{\mathbb{F}}[L, d] \subseteq \text{RS}_{\mathbb{F}'}[L, d]$.

Lemma 2.18 (Hamming distance). Δ is a metric on $\mathbb{F}^L$. In particular, if w agrees with w' on at least αM points and w' agrees with w'' on at least βM points, then w agrees with w'' on at least $(\alpha + \beta - 1)M$ points.

Proof. Symmetry, and $\Delta(w, w') = 0$ if and only if $w = w'$, are clear. If $w(\xi) \neq w''(\xi)$, then $w(\xi) \neq w'(\xi)$ or $w'(\xi) \neq w''(\xi)$; counting gives the triangle inequality. The second statement is the triangle inequality rewritten in terms of agreements, $1 - \Delta$. $\square$

Lemma 2.19 (Parameters of Reed–Solomon codes). *Let $\mathcal{C} = \text{RS}_{\mathbb{F}}[L, d]$ with $|L| = M$.*

1. *The map $\text{ev}_L : \mathbb{F}[X]_{<d} \rightarrow \mathcal{C}$ is an $\mathbb{F}$ -linear bijection. For $c \in \mathcal{C}$ we write $P_c \in \mathbb{F}[X]_{<d}$ for the unique polynomial with $\text{ev}_L(P_c) = c$. Since ev_L is injective on $\mathbb{F}[X]_{<M}$ for every finite field $\mathbb{F}' \supseteq \mathbb{F}$, P_c depends only on the word c , and not on d or on the field in which c is regarded.*
2. *Two distinct codewords agree on at most $d-1$ points of L , so $\Delta(c, c') \geq 1 - (d-1)/M > 1 - \rho$ for all distinct $c, c' \in \mathcal{C}$.*
3. *If $2\delta < 1 - (d-1)/M$, in particular if $\delta \leq (1 - \rho)/2$, then every word $w \in \mathbb{F}^L$ is within distance δ of at most one codeword.*

Proof. Items 1 and 2 follow from Lemma 2.1, since $d \leq M$ and the points of L are distinct. For item 3, two codewords within distance δ of w would be within distance $2\delta < 1 - (d-1)/M$ of each other (Lemma 2.18), contradicting item 2. Finally $(1 - \rho)/2 = (1 - d/M)/2 < (1 - (d-1)/M)/2$. $\square$

Fact 2.20 (Efficient unique decoding [WB86, Gao03]). *There is an algorithm that, given L , d and a word $w \in \mathbb{F}^L$, outputs the unique $c \in \text{RS}_{\mathbb{F}}[L, d]$ with $\Delta(w, c) < \frac{1}{2}(1 - (d-1)/M)$ whenever it exists, using a number of operations in $\mathbb{F}$ polynomial in M . Checking the distance of the output to w then decides whether such a codeword exists, at the cost of $O(M)$ further comparisons.*

Lemma 2.21 (Decoding over an extension field). *Let $\mathbb{K} \supseteq \mathbb{F}_q$ be a finite field, L a smooth domain of order M , $1 \leq d \leq M$, and $w \in \mathbb{F}_q^L$. If $c \in \text{RS}_{\mathbb{K}}[L, d]$ satisfies $\Delta(w, c) < \frac{1}{2}(1 - (d-1)/M)$, then $c \in \mathbb{F}_q^L$ and $P_c \in \mathbb{F}_q[X]_{<d}$; in particular $c \in \text{RS}[L, d]$.*

Proof. Let $\text{Frob} : \mathbb{K} \rightarrow \mathbb{K}$, $x \mapsto x^q$, be the Frobenius map; it is an injective ring homomorphism, since q is a power of the characteristic p and $(x+y)^p = x^p + y^p$. Its fixed points are the roots of $X^q - X$, of which there are at most q by Lemma 2.1; every element of $\mathbb{F}_q$ is one (since $x^{q-1} = 1$ for $x \in \mathbb{F}_q^\times$), so the fixed points are exactly the elements of $\mathbb{F}_q$. Apply Frob entrywise to words, and coefficientwise to polynomials. Since $L \subseteq \mathbb{F}_q$, we have $\text{Frob}(P(\xi)) = \text{Frob}(P)(\xi)$ for $\xi \in L$, so $\text{Frob}(c) = \text{ev}_L(\text{Frob}(P_c)) \in \text{RS}_{\mathbb{K}}[L, d]$. Since Frob is injective and $\text{Frob}(w) = w$, $\Delta(w, \text{Frob}(c)) = \Delta(\text{Frob}(w), \text{Frob}(c)) = \Delta(w, c)$. By Lemma 2.19(3), $\text{Frob}(c) = c$, so $c \in \mathbb{F}_q^L$. Then $\text{Frob}(P_c)$ and P_c both lie in $\mathbb{K}[X]_{<d}$ and evaluate to c , so they are equal by Lemma 2.19(1), and the coefficients of P_c lie in $\mathbb{F}_q$. $\square$

Even–odd split of words. Let L be a smooth domain of order $M \geq 2$, and $\mathbb{F} \supseteq \mathbb{F}_q$ a finite field. For a word $w \in \mathbb{F}^L$, define the words $w_e, w_o \in \mathbb{F}^{L^2}$ by

$$w_e(\xi^2) = \frac{w(\xi) + w(-\xi)}{2}, \quad w_o(\xi^2) = \frac{w(\xi) - w(-\xi)}{2\xi} \quad (\xi \in L). \quad (1)$$

Lemma 2.22 (Even–odd split of words). *Let $w \in \mathbb{F}^L$ with $|L| = M \geq 2$.*

1. *The words w_e and w_o are well defined, and $w \mapsto (w_e, w_o)$ is $\mathbb{F}$ -linear.*
2. *(Locality.) For $\eta \in L^2$, the values $w_e(\eta)$ and $w_o(\eta)$ depend only on the values of w on the fibre over η .*
3. *For every $\xi \in L$,*

$$w(\xi) = w_e(\xi^2) + \xi w_o(\xi^2), \quad w(-\xi) = w_e(\xi^2) - \xi w_o(\xi^2). \quad (2)$$

Consequently, w vanishes on the fibre over η if and only if $w_e(\eta) = w_o(\eta) = 0$.

4. If $1 \leq d \leq M/2$, $P \in \mathbb{F}[X]_{<2d}$ and $w = \text{ev}_L(P)$, then $w_e = \text{ev}_{L^2}(P_e)$ and $w_o = \text{ev}_{L^2}(P_o)$, where (P_e, P_o) is the even-odd split of P (Lemma 2.3); in particular $w_e, w_o \in \text{RS}_{\mathbb{F}}[L^2, d]$.

Proof. (1) The denominators are nonzero because the characteristic is odd and $\xi \neq 0$. Replacing ξ by $-\xi$ leaves both right-hand sides of (1) unchanged, and ξ and $-\xi$ are the only square roots of ξ^2 in L (Lemma 2.15(4)), so the words are well defined. Linearity is clear. (2) is visible in (1). (3) Add and subtract the two formulas of (1). If $w_e(\eta) = w_o(\eta) = 0$, then (2) gives $w(\pm\xi) = 0$; the converse follows from (1). (4) For $\xi \in L$, $P(\pm\xi) = P_e(\xi^2) \pm \xi P_o(\xi^2)$; substituting into (1) gives $w_e(\xi^2) = P_e(\xi^2)$ and $w_o(\xi^2) = P_o(\xi^2)$. Moreover $P_e, P_o \in \mathbb{F}[X]_{<d}$, and $d \leq M/2 = |L^2|$ by Lemma 2.15(2). $\square$

2.5 Proximity gaps

The external result used in our soundness analysis is the correlated-agreement theorem of Ben-Sasson, Carmon, Ishai, Kopparty and Saraf in the unique-decoding regime. In their notation, $\text{RS}[\mathbb{F}, D, k]$ is the code of polynomials of degree at most k evaluated on a set $D \subseteq \mathbb{F}$, with rate $(k+1)/|D|$. The statement below is theirs with $D = L$ and $d = k+1$. It holds over any finite field $\mathbb{F}$; we apply it with $\mathbb{F} \supseteq \mathbb{F}_q$ and $L \subseteq \mathbb{F}_q^\times \subseteq \mathbb{F}$.

Theorem 2.23 ([BSCI⁺23, Theorem 4.1]). *Let $\mathbb{F} \supseteq \mathbb{F}_q$ be a finite field, $\mathcal{C} = \text{RS}_{\mathbb{F}}[L, d]$ with $|L| = M$ and rate $\rho = d/M$, and $0 < \delta \leq (1 - \rho)/2$. Let $u^0, u^1 \in \mathbb{F}^L$ and*

$$S = \{r \in \mathbb{F} : \Delta(u^0 + r u^1, \mathcal{C}) \leq \delta\}.$$

If $|S| > M$, then there are a set $L' \subseteq L$ with $|L'| \geq (1 - \delta)M$ and codewords $\hat{c}^0, \hat{c}^1 \in \mathcal{C}$ such that u^0 agrees with $\hat{c}^0$, and u^1 agrees with $\hat{c}^1$, on L' .

3 Interactive Oracle Proofs and Polynomial Commitments

This section recalls the proof-system notions used in Sections 6 and 7: interactive oracle proofs, their probability space and soundness notions, the sumcheck protocol, polynomial commitment schemes, and the compilation of interactive oracle proofs into non-interactive arguments. Everything in Sections 3.1 to 3.4 is proved. Section 3.5 concerns the compiled, non-interactive scheme; it restates the definitions of Chiesa, Di, Hu and Zheng [CDHZ25] that we need and quotes their theorem (Theorem 3.16), and nothing in Sections 3.1 to 3.4 or in Sections 4 to 6 depends on it.

3.1 Interactive oracle proofs

A *relation* $\mathcal{R}$ is a set of pairs $(\mathbf{x}, \mathbf{w})$ of an *instance* $\mathbf{x}$ and a *witness* $\mathbf{w}$; its *language* is $L_{\mathcal{R}} = \{\mathbf{x} : \exists \mathbf{w}, (\mathbf{x}, \mathbf{w}) \in \mathcal{R}\}$. An instance may have an *oracle part*: a word to which the verifier has query access only.

Definition 3.1 (Interactive oracle proof). A *public-coin interactive oracle proof* (IOP) [BSCS16] with $\nu \geq 1$ rounds and *challenge sets* $\text{Ch}_1, \dots, \text{Ch}_\nu$ (finite and nonempty) is a pair (P, V) of a prover and a verifier. On a common instance $\mathbf{x}$ (and a witness given to P only), the interaction proceeds as follows. For $i = 1, \dots, \nu$:

1. P sends a message $\mathbf{m}_i$. Parts of $\mathbf{m}_i$ may be designated as *oracles*: V does not read them, but may later query individual entries.
2. V replies with a *challenge* $c_i \in \text{Ch}_i$.

After the last round, V reads the non-oracle parts of the instance and of the messages, queries entries of the oracles (including the oracle part of the instance) at positions that are determined by these non-oracle parts and by the challenges, and outputs *accept* or *reject* as a deterministic function of everything it has read.

A *partial transcript* of i rounds is $\tau = (\mathbf{x}, \mathbf{m}_1, \mathbf{c}_1, \dots, \mathbf{m}_i, \mathbf{c}_i)$; for $i = 0$ it is the *empty transcript* $(\mathbf{x})$, and for $i = \nu$ it is a *full transcript*. Since the verifier's decision is a deterministic function of the full transcript (with oracles), we say that V *accepts* or *rejects* a full transcript. When the verifier makes checks “during” a round, as in the protocols below, these checks are predicates on the transcript that are evaluated at the end; the interaction always runs all ν rounds.

Provers and the probability space. A *deterministic prover* P^* is a function that maps each partial transcript of $i - 1$ rounds to a message $\mathbf{m}_i$. Its interaction with V on an instance $\mathbf{x}$ is determined by the challenges. All probabilities are over the *challenge space* $\Omega = \text{Ch}_1 \times \dots \times \text{Ch}_\nu$ with the uniform distribution, that is, with independent uniform challenges. We write τ_i for the partial transcript after i rounds, which is a function of $(\mathbf{c}_1, \dots, \mathbf{c}_i)$, and $\mathbf{m}_i$ for the i -th message, which is a function of $(\mathbf{c}_1, \dots, \mathbf{c}_{i-1})$. A *randomised prover* is a finitely supported probability distribution over deterministic provers, independent of the challenges.

Lemma 3.2 (Sequential challenges). *For $i \in \llbracket 1, \nu \rrbracket$, let $E_i \subseteq \text{Ch}_1 \times \dots \times \text{Ch}_i$, and suppose that for every $(\mathbf{c}_1, \dots, \mathbf{c}_{i-1})$ there are at most k_i values $\mathbf{c}_i \in \text{Ch}_i$ with $(\mathbf{c}_1, \dots, \mathbf{c}_i) \in E_i$. Then*

$$\Pr_{\Omega}[\exists i \in \llbracket 1, \nu \rrbracket : (\mathbf{c}_1, \dots, \mathbf{c}_i) \in E_i] \leq \sum_{i=1}^{\nu} \frac{k_i}{|\text{Ch}_i|}.$$

Proof. Under the uniform distribution on Ω , the prefix $(\mathbf{c}_1, \dots, \mathbf{c}_i)$ is uniform on $\text{Ch}_1 \times \dots \times \text{Ch}_i$. Counting the elements of E_i prefix by prefix gives $|E_i| \leq k_i \prod_{i' < i} |\text{Ch}_{i'}|$, so the probability of the i -th event is at most $k_i/|\text{Ch}_i|$. Conclude by the union bound. $\square$

Lemma 3.3 (Independent queries). *Let Ω' be a finite nonempty set with the uniform distribution, L a finite nonempty set, $\kappa \geq 1$, and let $\varpi \mapsto \mathcal{S}(\varpi) \subseteq L$ and $\mathcal{A} \subseteq \Omega'$ be given. On $\Omega' \times L^{\times \kappa}$ with the uniform distribution, where $L^{\times \kappa}$ denotes the Cartesian power (the set of κ -tuples of elements of L),*

$$\Pr[\varpi \in \mathcal{A} \text{ and } \xi^{(1)}, \dots, \xi^{(\kappa)} \in \mathcal{S}(\varpi)] = \mathbb{E}_{\varpi} \left[\mathbf{1}_{\mathcal{A}}(\varpi) \cdot \left(\frac{|\mathcal{S}(\varpi)|}{|L|} \right)^{\kappa} \right],$$

where $\mathbf{1}_{\mathcal{A}}$ is the indicator function of $\mathcal{A}$. In particular, if $\pi \geq 0$ and $|\mathcal{S}(\varpi)| \leq \pi|L|$ for all $\varpi \in \mathcal{A}$, the probability is at most $\Pr[\mathcal{A}] \pi^{\kappa} \leq \pi^{\kappa}$.

Proof. For fixed $\varpi \in \mathcal{A}$, the number of κ -tuples of elements of $\mathcal{S}(\varpi)$ is $|\mathcal{S}(\varpi)|^{\kappa}$; divide by $|\Omega'| |L|^{\kappa}$ and sum over ϖ . $\square$

Lemma 3.4 (Randomised provers). *Fix an instance $\mathbf{x}$, and let E be a set of full transcripts of $\mathbf{x}$ (for instance, the accepted ones). If $\Pr_{\Omega}[\tau_{\nu} \in E] \leq \varepsilon$ for every deterministic prover, then the same bound holds for every randomised prover. Equivalently, if a randomised prover achieves $\Pr[\tau_{\nu} \in E] > \varepsilon$, so does some deterministic prover in its support.*

Proof. Since the prover's randomness is independent of the challenges, the probability for a randomised prover is the average, over its distribution, of the probabilities for deterministic provers. $\square$

Definition 3.5 (Completeness and soundness). The IOP is *complete* for $\mathcal{R}$ if, for every $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, the honest prover makes $\mathbf{V}$ accept with probability 1. It has *soundness error* ε if, for every $\mathbf{x} \notin L_{\mathcal{R}}$ and every deterministic prover $\mathbf{P}^*$, $\mathbf{V}$ accepts with probability at most ε .

By Lemma 3.4, soundness also holds against randomised provers.

3.2 Round-by-round soundness

For the Fiat–Shamir compilation, the relevant notions are round by round: every partial transcript is classified as *doomed* or not, and no round can turn a doomed transcript into a non-doomed one except with small probability. The following definitions are those of [BGTZ24, Definitions 3.11 and 3.12]; the soundness notion was introduced in [CCH⁺19], where it is phrased with a *state function*, which amounts to specifying a set $\mathcal{D}$ of doomed transcripts, and its knowledge analogue in [CMS19]. We place the verifier’s query positions in the last challenge $\mathbf{c}_\nu$.

Definition 3.6 (Round-by-round soundness). An IOP with ν rounds has *round-by-round soundness error* $(\varepsilon_1, \dots, \varepsilon_\nu) \in [0, 1]^\nu$ for $\mathcal{R}$ if there is a set $\mathcal{D}$ of partial transcripts, the *doomed* ones, such that:

1. if $\mathbf{x} \notin L_{\mathcal{R}}$, then the empty transcript $(\mathbf{x})$ is doomed;
2. for every $i \in \llbracket 1, \nu \rrbracket$, every doomed partial transcript τ of $i - 1$ rounds and every message $\mathbf{m}_i$,

$$\Pr_{\mathbf{c}_i \leftarrow \text{Ch}_i} [(\tau, \mathbf{m}_i, \mathbf{c}_i) \notin \mathcal{D}] \leq \varepsilon_i;$$

3. $\mathbf{V}$ rejects every doomed full transcript.

We write $\varepsilon_{\text{rbr}} = \max_i \varepsilon_i$.

Definition 3.7 (Round-by-round knowledge soundness). An IOP with ν rounds is *round-by-round knowledge sound* for $\mathcal{R}$ with error $(\varepsilon_1, \dots, \varepsilon_\nu) \in [0, 1]^\nu$ if there are a set $\mathcal{D}$ of partial transcripts and an algorithm Ext , the *extractor*, running in time polynomial in the size of the instance (including its oracle part), defined on all pairs of a partial transcript and a next message, such that:

1. the empty transcript $(\mathbf{x})$ is doomed for every instance $\mathbf{x}$;
2. for every $i \in \llbracket 1, \nu \rrbracket$, every doomed partial transcript τ of $i - 1$ rounds and every message $\mathbf{m}_i$,
if

$$\Pr_{\mathbf{c}_i \leftarrow \text{Ch}_i} [(\tau, \mathbf{m}_i, \mathbf{c}_i) \notin \mathcal{D}] > \varepsilon_i,$$

then $(\mathbf{x}, \text{Ext}(\tau, \mathbf{m}_i)) \in \mathcal{R}$, where $\mathbf{x}$ is the instance of τ ;

3. $\mathbf{V}$ rejects every doomed full transcript.

We write $\varepsilon_{\text{rbr}} = \max_i \varepsilon_i$.

The polynomial-time requirement on Ext matters only for the compiled scheme of Section 3.5; the results of Sections 3.2 and 3.4 use conditions 1–3 but not the running time of Ext .

Lemma 3.8 (Round-by-round implies overall). *Let $\mathbf{P}^*$ be a deterministic prover and $\mathbf{x}$ an instance, and let τ_i and $\mathbf{m}_i$ be as in Section 3.1.*

1. If the IOP has round-by-round soundness error $(\varepsilon_i)_i$ and $\mathbf{x} \notin L_{\mathcal{R}}$, then $\mathbf{V}$ accepts with probability at most $\sum_{i=1}^{\nu} \varepsilon_i$.
2. If the IOP is round-by-round knowledge sound with error $(\varepsilon_i)_i$, then

$$\Pr[\mathbf{V} \text{ accepts}] \leq \sum_{i=1}^{\nu} \varepsilon_i + \Pr[\exists i \in [1, \nu] : \tau_{i-1} \in \mathcal{D} \text{ and } (\mathbf{x}, \text{Ext}(\tau_{i-1}, \mathbf{m}_i)) \in \mathcal{R}].$$

In particular, if $\mathbf{V}$ accepts with probability greater than $\sum_i \varepsilon_i$, then there exist $i \in [1, \nu]$ and challenges $\mathbf{c}_1, \dots, \mathbf{c}_{i-1}$ such that $\tau_{i-1} \in \mathcal{D}$ and $(\mathbf{x}, \text{Ext}(\tau_{i-1}, \mathbf{m}_i)) \in \mathcal{R}$.

Proof. In both cases τ_0 is doomed, and an accepted full transcript τ_{ν} is not doomed (condition 3). So if $\mathbf{V}$ accepts, there is a first round i with $\tau_{i-1} \in \mathcal{D}$ and $\tau_i \notin \mathcal{D}$. For (2), let E_i be the set of $(\mathbf{c}_1, \dots, \mathbf{c}_i)$ such that $\tau_{i-1} \in \mathcal{D}$, $(\mathbf{x}, \text{Ext}(\tau_{i-1}, \mathbf{m}_i)) \notin \mathcal{R}$ and $\tau_i \notin \mathcal{D}$. For fixed $(\mathbf{c}_1, \dots, \mathbf{c}_{i-1})$, the transcript τ_{i-1} and the message $\mathbf{m}_i$ are fixed; if the first two conditions hold, condition 2 of Definition 3.7 (in contrapositive form) shows that at most $\varepsilon_i |\text{Ch}_i|$ values of $\mathbf{c}_i$ satisfy the third. Acceptance implies that some E_i occurs or that the event in the displayed probability occurs, and Lemma 3.2 bounds the probability of the former by $\sum_i \varepsilon_i$. For (1), the same argument applies with E_i defined without the condition on Ext , using condition 2 of Definition 3.6. $\square$

3.3 The sumcheck protocol

The sumcheck protocol of Lund, Fortnow, Karloff and Nisan [LFKN92] reduces a claim about the sum of a polynomial over the Boolean hypercube to a claim about one evaluation. Our commitment scheme runs it on a product of two multilinear polynomials, so we state it for polynomials of degree at most 2 in each variable.

Let $\mathbb{F} \supseteq \mathbb{F}_q$ be a finite field, and let $F \in \mathbb{F}[x_1, \dots, x_n]$ have degree at most 2 in each variable, with $n \geq 1$. For $j \in [1, n]$ and $r_1, \dots, r_{j-1} \in \mathbb{F}$, the *honest round polynomial* is

$$s_j^*(X) = \sum_{b=0}^{2^{n-j}-1} F(r_1, \dots, r_{j-1}, X, b_1, \dots, b_{n-j}) \in \mathbb{F}[X]_{<3}.$$

To prove the claim $\sum_{b=0}^{N-1} F(b) = v_0$, the prover sends, in round $j = 1, \dots, n$, a round polynomial $s_j \in \mathbb{F}[X]_{<3}$ (honestly $s_j = s_j^*$). The verifier samples $r_j \in \mathbb{F}$ uniformly at random and sets $v_j = s_j(r_j)$; it accepts if $s_j(0) + s_j(1) = v_{j-1}$ for every $j \in [1, n]$ and $v_n = F(r_1, \dots, r_n)$.

Lemma 3.9 (Sumcheck). *If $\sum_{b=0}^{N-1} F(b) = v_0$, the honest prover is accepted with probability 1. If $\sum_b F(b) \neq v_0$, then for every deterministic prover the verifier accepts with probability at most $2n/|\mathbb{F}|$. More precisely, in that case the verifier accepts only if there is a round j with $s_j \neq s_j^*$ and $s_j(r_j) = s_j^*(r_j)$.*

Proof. Put $v_0^* = \sum_b F(b)$ and $v_j^* = s_j^*(r_j)$ for $j \in [1, n]$. By definition, $s_j^*(0) + s_j^*(1) = v_{j-1}^*$ for every j , and $v_n^* = F(r_1, \dots, r_n)$. Completeness follows, since the honest prover has $v_j = v_j^*$ for all j .

Soundness: suppose that $v_0 \neq v_0^*$, that the verifier accepts, and that $s_j(r_j) \neq s_j^*(r_j)$ whenever $s_j \neq s_j^*$. We show by induction that $v_j \neq v_j^*$ for all $j \in [0, n]$; for $j = n$ this contradicts the final check. If $v_{j-1} \neq v_{j-1}^*$, the check of round j gives $s_j(0) + s_j(1) = v_{j-1} \neq v_{j-1}^* = s_j^*(0) + s_j^*(1)$, so $s_j \neq s_j^*$, hence $v_j = s_j(r_j) \neq s_j^*(r_j) = v_j^*$ by assumption.

For the probability, let E_j be the set of $(r_1, \dots, r_j)$ with $s_j \neq s_j^*$ and $s_j(r_j) = s_j^*(r_j)$. For fixed $r_1, \dots, r_{j-1}$, both s_j and s_j^* are fixed, and their difference, when nonzero, has degree at most 2, so at most 2 values of r_j lie in E_j (Lemma 2.1). Lemma 3.2 gives the bound $2n/|\mathbb{F}|$. $\square$

3.4 Polynomial commitment schemes

A *multilinear polynomial commitment scheme* lets a prover commit to a table $f : \llbracket 0, N-1 \rrbracket \rightarrow \mathbb{F}$, over a finite field $\mathbb{F} \supseteq \mathbb{F}_q$, and later prove claims of the form $\tilde{f}(z) = v$. We describe it in the IOP model, where the commitment is an oracle.

Definition 3.10 (Polynomial commitment in the IOP model). A multilinear polynomial commitment scheme in the IOP model consists of:

- an *encoding* Enc that maps each table f to a word $\text{Enc}(f)$, the *commitment*;
- an *evaluation protocol*: a public-coin IOP whose instance is $(w; z, v)$, with a word w (the commitment) as oracle part, a point $z \in \mathbb{F}^n$ and a value $v \in \mathbb{F}$.

It is *complete* if the evaluation protocol accepts with probability 1 when the prover is honest, $w = \text{Enc}(f)$ and $\tilde{f}(z) = v$.

Every claim (z, v) is true for *some* table (for instance the constant table with value v , since $\sum_b \text{eq}(b, z) = \prod_{k=1}^n (z_k + 1 - z_k) = 1$), so the relation $\{((z, v), f) : \tilde{f}(z) = v\}$ says nothing about the commitment. The meaningful requirement is that the commitment determines the table. To express it, fix a function dist from pairs of words to real numbers, and a threshold δ , such that

$$\text{for every word } w, \text{ there is at most one table } f \text{ with } \text{dist}(w, \text{Enc}(f)) \leq \delta, \quad (3)$$

and consider the relation

$$\mathcal{R}_{\text{eval}}^\delta = \{((w; z, v), f) : \text{dist}(w, \text{Enc}(f)) \leq \delta \text{ and } \tilde{f}(z) = v\}.$$

Definition 3.11 (Evaluation binding). The evaluation protocol is ε -*evaluation binding* if for every word w , point z and values $v \neq v'$, it is not the case that some deterministic prover makes the verifier accept $(w; z, v)$ with probability greater than ε and some deterministic prover makes it accept $(w; z, v')$ with probability greater than ε .

By Lemma 3.4, ε -evaluation binding also holds when randomised provers are allowed.

Lemma 3.12 (Knowledge soundness implies binding). *If the evaluation protocol is round-by-round knowledge sound for $\mathcal{R}_{\text{eval}}^\delta$ with error $(\varepsilon_i)_i$, and (3) holds, then it is ε -evaluation binding with $\varepsilon = \sum_i \varepsilon_i$.*

Proof. Suppose that provers are accepted on $(w; z, v)$ and on $(w; z, v')$, each with probability greater than ε . By Lemma 3.8(2), the extractor outputs, on some partial transcript of each interaction, tables f and f' with $((w; z, v), f) \in \mathcal{R}_{\text{eval}}^\delta$ and $((w; z, v'), f') \in \mathcal{R}_{\text{eval}}^\delta$. Both tables satisfy $\text{dist}(w, \text{Enc}(\cdot)) \leq \delta$, so $f = f'$ by (3), and $v = f(z) = f'(z) = v'$. $\square$

3.5 Non-interactive arguments

This subsection concerns only the non-interactive scheme (Corollary 7.26).

Merkle trees. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_H}$ be a hash function with output length λ_H , and let $\parallel$ denote concatenation of bit strings. The *Merkle tree* [Mer89] over a vector $\mathbf{d} = (\mathbf{d}_0, \dots, \mathbf{d}_{T-1})$ of bit strings, with T a power of 2, is the complete binary tree whose leaves are the hashes $H(0 \parallel \mathbf{d}_i)$ and whose internal nodes are $H(1 \parallel \text{left child} \parallel \text{right child})$; its *root* $\text{MT}(\mathbf{d})$ is a commitment to the vector. An *opening* of position i consists of $\mathbf{d}_i$ and its *authentication path*, the $\log_2 T$ siblings on the path from the leaf to the root; it is checked by recomputing the root. The entries $\mathbf{d}_i$ may be encodings of several field elements; in Section 8 each leaf holds the two values of a word on one fibre.

Lemma 3.13 (Merkle binding). *1. Given two accepting openings of the same position to different values under the same root, one can compute a collision of H , that is, two distinct inputs with the same hash, with $O(\log_2 T)$ hash evaluations.*

2. Given two vectors $\mathbf{d} \neq \mathbf{d}'$ of length T with $\text{MT}(\mathbf{d}) = \text{MT}(\mathbf{d}')$, one can compute a collision of H with $O(T)$ hash evaluations.

Proof. (1) Number the levels of the tree from 0 (leaf hashes) to $\log_2 T$ (root). Node values are strings of the fixed length λ_H , and the position i fixes, at every level, on which side the path node lies; so the input $1 \parallel \text{left} \parallel \text{right}$ hashed at a level determines the path node below it and its sibling. Recompute both paths from the leaves to the root, and let t be the first level at which the two recomputed path nodes are equal; it exists because both paths end at the same root. If $t = 0$, the hashed inputs $0 \parallel \mathbf{d}_i$ and $0 \parallel \mathbf{d}'_i$ are distinct. If $t > 0$, the inputs hashed at level t are distinct, because they contain the path nodes of level $t - 1$, which differ by the choice of t . In both cases these two inputs have the same hash. (2) Compute both trees, and apply (1) at a position where $\mathbf{d}$ and $\mathbf{d}'$ differ, taking the authentication paths from the two trees. $\square$

Random oracles. In the *random oracle model*, H is modelled as a uniformly random function to which all parties have oracle access; in the *quantum random oracle model* (QROM) [BDF⁺11], the adversary is a quantum algorithm that may query H in superposition. The *BCS transformation* of Ben-Sasson, Chiesa and Spooner [BCS16] compiles a public-coin IOP into a non-interactive argument: each oracle message is replaced by a Merkle root, each challenge is derived by hashing the instance and the transcript so far (the Fiat–Shamir paradigm), and the proof contains the non-oracle messages, the roots, and an opening of every queried oracle entry. The BCS theorem in the QROM of Chiesa, Manohar and Spooner [CMS19, Theorem 8.6] covers IOPs whose instance has no oracle part. An evaluation protocol has an oracle part, the commitment, which in the non-interactive scheme is replaced by its Merkle root and is chosen by the adversary. This setting is covered by the extension of the BCS transformation to *interactive oracle reductions* (IORs) of Chiesa, Di, Hu and Zheng [CDHZ25], which we now recall in the special case that we need. All definitions below are those of [CDHZ25]; we restate them to fix notation, and we indicate the specialisation. We write ε for the error terms that [CDHZ25] denotes by κ , since κ is our number of queries.

Relations with an implicit instance [CDHZ25, §3.3]. Fix a finite alphabet Σ and a length $l_y \geq 1$. A *partial string* is an element of $(\Sigma \sqcup \{\perp\})^{l_y}$, the symbol $\perp$ marking an undefined entry; $\text{Dom}(y)$ is the set of positions where y is defined, and y is *full* if $\text{Dom}(y)$ is everything. The fractional Hamming distance $\Delta(y, y')$ of two partial strings is the fraction of positions at which they differ, $\perp$ being different from every symbol of Σ . A *relation with implicit instance* is a set $\mathcal{R}$ of triples $(\mathbf{x}, y, \mathbf{w})$ of an *explicit instance* $\mathbf{x}$, a full string $y \in \Sigma^{l_y}$, the *implicit instance*, to which the

verifier has oracle access only, and a witness $\mathbf{w}$. For $\delta \in [0, 1]$, the *relaxed relation* is

$$\mathcal{R}_{\leq \delta} = \{ (\mathbf{x}, y, \mathbf{w}) : y \in (\Sigma \sqcup \{\perp\})^{l_y}, \exists y^* \in \Sigma^{l_y} \text{ with } (\mathbf{x}, y^*, \mathbf{w}) \in \mathcal{R} \text{ and } \Delta(y, y^*) \leq \delta \}.$$

IOPs as interactive oracle reductions [CDHZ25, §3.4]. Consider an IOP (Definition 3.1) with ν rounds whose instance is $(\mathbf{x}; y)$ with oracle part y , in which every prover message is a string Π_i over Σ , whose challenges are uniformly random bit strings $\mathbf{vr}_i \in \{0, 1\}^{\beta_i}$, and whose verifier reads (by queries) positions of y and of the Π_i that are determined by $\mathbf{x}$ and the challenges only; a message part that the verifier reads entirely is a string all of whose positions are read. Let $\mathcal{R}_{\text{acc}} = \{(\top, (), \mathbf{w})\}$, where $\top$ is a fixed symbol, $()$ is the empty implicit instance, and $\mathbf{w}$ is arbitrary; then $(\mathcal{R}_{\text{acc}})_{\leq \delta} = \mathcal{R}_{\text{acc}}$ for every δ . If the verifier outputs the instance $(\top, ())$ when it accepts and rejects otherwise, the IOP is an IOR from $\mathcal{R}$ to $\mathcal{R}_{\text{acc}}$ in the sense of [CDHZ25, §3.4], with $n_y = 1$ input implicit instance and $n'_y = 0$ output implicit instances. $\mathcal{R}_{\text{acc}}$ is *monotone* in the sense of [CDHZ25, Definition 11.1], because it has no implicit instance. A *transcript* is a sequence $\text{tr} = (\Pi_1, \mathbf{vr}_1, \Pi_2, \dots)$; the empty transcript is written $\text{tr}_\emptyset$.

In the proof of the theorem below, the verifier and the knowledge state function are evaluated on strings produced by an extractor, whose entries may be undefined [CDHZ25, Constructions 11.8 and 11.9]. We therefore require the verifier to be defined when y and the Π_i are partial strings, and we take the statements $(\mathbf{x}, y)$ and the messages Π_i in Definition 3.14 to range over partial strings. For the protocols of this paper, the verifier rejects as soon as one of the entries it reads is $\perp$.

Definition 3.14 (Relaxed round-by-round knowledge soundness [CDHZ25]). This is [CDHZ25, Definitions 3.5 and 3.6] with output relation $\mathcal{R}_{\text{acc}}$. A *knowledge state function* for an IOR from $\mathcal{R}$ to $\mathcal{R}_{\text{acc}}$ is a deterministic function KState that, on input a proximity bound $\delta \in [0, 1]$, a statement $(\mathbf{x}, y)$, a transcript tr and a witness $\mathbf{w}$, outputs a bit $\text{KState}_\delta(\mathbf{x}, y, \text{tr}, \mathbf{w})$, such that:

1. (empty transcript) $\text{KState}_\delta(\mathbf{x}, y, \text{tr}_\emptyset, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, y, \mathbf{w}) \in \mathcal{R}_{\leq \delta}$;
2. (prover moves) if the prover is about to move after tr and $\text{KState}_\delta(\mathbf{x}, y, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}_\delta(\mathbf{x}, y, \text{tr} \parallel \Pi, \mathbf{w}) = 0$ for every message Π ;
3. (full transcript) for a full transcript, $\text{KState}_\delta(\mathbf{x}, y, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier accepts tr .

The IOR has *relaxed round-by-round knowledge soundness errors* $(\varepsilon_1^{\text{rr}}, \dots, \varepsilon_\nu^{\text{rr}})$, functions of $(\mathbf{x}, y, \delta)$, if there are a knowledge state function KState and a deterministic polynomial-time algorithm E , which does not depend on δ , such that for every $\delta \in [0, 1]$, every statement $(\mathbf{x}, y)$, every $i \in \llbracket 1, \nu \rrbracket$ and every transcript $\text{tr} = (\Pi_1, \mathbf{vr}_1, \dots, \Pi_{i-1}, \mathbf{vr}_{i-1}, \Pi_i)$,

$$\Pr_{\mathbf{vr}_i \leftarrow \{0, 1\}^{\beta_i}} \left[\exists \mathbf{w} : \text{KState}_\delta(\mathbf{x}, y, \text{tr}, \text{E}(\mathbf{x}, y, \text{tr} \parallel \mathbf{vr}_i, \mathbf{w})) = 0 \right. \\ \left. \text{and } \text{KState}_\delta(\mathbf{x}, y, \text{tr} \parallel \mathbf{vr}_i, \mathbf{w}) = 1 \right] \leq \varepsilon_i^{\text{rr}}(\mathbf{x}, y, \delta).$$

For $\mathcal{R}_{\text{acc}}$, condition 3 is the specialisation of [CDHZ25, Definition 3.5], which asks that the verifier output an instance $(\mathbf{x}', y')$ with $(\mathbf{x}', y', \mathbf{w}) \in (\mathcal{R}_{\text{acc}})_{\leq \delta}$, that is, that it accept.

Salted Merkle commitments [CDHZ25, Definition 8.2]. The compiler of [CDHZ25] uses a salted variant of the Merkle tree: to commit to $\mathbf{a} = (a_1, \dots, a_l) \in \Sigma^l$ (l a power of 2, after padding), it samples uniformly random salts $\gamma_1, \dots, \gamma_l$ of a fixed length, sets the leaves to $\text{H}(a_h, \gamma_h)$ and the

internal nodes to $H(\text{left child}, \text{right child})$, where H is a random oracle with output length λ_H ; the root is the commitment cm , and the salts and the tree form the trapdoor td . An opening of a set I of positions consists of (a_h, γ_h) and the authentication path of every $h \in I$, and is checked by recomputing the root. The proof of Lemma 3.13(1) applies verbatim, with the input (a_h, γ_h) in place of $0\|\mathbf{d}_i$ and $\text{left}\|\text{right}$ in place of $1\|\text{left}\|\text{right}$: *two accepting openings of the same position to different values under the same root and the same oracle yield a collision of that oracle.*

Definition 3.15 (Committed relations [CDHZ25, Definition 11.2]). Let $\mathcal{R}$ be a relation with implicit instance and $\delta \in [0, 1]$. A *committed instance* is $\mathbf{cx} = (\mathbf{x}, \text{cm})$, where cm is a Merkle root, and a *committed witness* is $\mathbf{cw} = (y, \mathbf{w}, \text{td})$.

- $(\mathbf{cx}, \mathbf{cw}) \in \text{CR}[\mathcal{R}]$ if $(\mathbf{x}, y, \mathbf{w}) \in \mathcal{R}$ and cm and td are the root and the trapdoor of the salted Merkle tree of y with the salts recorded in td ;
- $(\mathbf{cx}, \mathbf{cw}) \in \widehat{\text{CR}}[\mathcal{R}, \delta]$ (the *relaxed committed relation*) if y is a partial string with $(\mathbf{x}, y, \mathbf{w}) \in \mathcal{R}_{\leq \delta}$, and the opening of all positions of $\text{Dom}(y)$ computed from td is accepted for y under the root cm .

In [CDHZ25] the committed instance also carries an index that selects, by domain separation, the random oracle used by the commitment. We fix this index in the scheme (the verifier rejects any other value) and omit it from the notation. This extra check only removes accepted proofs and makes no oracle query, so the extraction event of [CDHZ25, Definition 10.3] (acceptance and failed extraction) can only become smaller, and Theorem 3.16 still applies.

The compiler. $\text{BCS}[\Pi]$ denotes the non-interactive reduction of [CDHZ25, Construction 11.7] applied to an IOR Π , with salted Merkle commitments of output length λ_H . Its prover receives $(\mathbf{cx}, \mathbf{cw}) \in \text{CR}[\mathcal{R}]$; in round i it commits to Π_i by a salted Merkle tree, samples a fresh random salt, and derives vr_i by querying a random oracle on $\mathbf{x}$, the root cm of y , the roots of $\Pi_1, \dots, \Pi_i$ and the salts of rounds $1, \dots, i$. The proof consists of the roots, the salts, and the openings of every position of y and of the Π_i that the IOR verifier reads; the verifier recomputes the challenges, checks the openings, and runs the IOR verifier on the opened values. All random oracles are derived from one random oracle by domain separation.

Knowledge soundness. The security notion is post-quantum straightline knowledge soundness [CDHZ25, Definition 10.3]. It is stated for an *emulated oracle*, which answers the adversary's queries and gives the extractor an additional extraction interface. It has three error terms: a simulation error ε_{sim} , bounding the distinguishing advantage between the emulated oracle and a true random oracle; a commutativity error ε_{com} ; and the extraction error ε_{ext} , which bounds, for a Q -query quantum adversary $\tilde{P}$ that outputs a committed instance $\mathbf{cx}$ of its choice, a proof, and a witness for the output relation, the probability that

the verifier accepts, and the output $\mathbf{cw}$ of the extractor satisfies $(\mathbf{cx}, \mathbf{cw}) \notin \widehat{\text{CR}}[\mathcal{R}, \delta]$.

(For $\mathcal{R}_{\text{acc}}$ the output relation holds exactly when the verifier accepts, and its decider makes no oracle queries.) The extractor is a polynomial-time quantum algorithm and is *straightline*: it does not rewind the adversary.

Theorem 3.16 ([CDHZ25, Theorems 6.10, 8.3 and 11.3], specialised). *Let Π be an IOR from $\mathcal{R}$ to $\mathcal{R}_{\text{acc}}$ as above, with ν rounds, challenge lengths $\beta_1, \dots, \beta_\nu$ and $\beta_{\min} = \min_i \beta_i$, relaxed round-by-round knowledge soundness errors $(\varepsilon_i^{\text{rr}})_i$ (Definition 3.14), and let $\varepsilon_{\text{rr}}(\delta) = \max_{i, (\mathbf{x}, y)} \varepsilon_i^{\text{rr}}(\mathbf{x}, y, \delta)$,*

the maximum being over the statements of bounded size under consideration. Let $l_{\max}$ be the largest length of y and of the strings Π_i , q_V the number of random-oracle queries of the verifier of $\text{BCS}[\Pi]$, $q_{\widehat{\text{CR}}}$ that of the decider of $\widehat{\text{CR}}[\mathcal{R}, \delta]$, q_s the number of positions of y and of the Π_i that the IOR verifier reads, and $T_1 = Q + 1 + q_V + \nu + q_{\widehat{\text{CR}}}$. Then there is a polynomial-time quantum extractor such that, for every $\delta \in [0, 1]$ and every query bound Q , $\text{BCS}[\Pi]$ is a non-interactive reduction from $(\text{CR}[\mathcal{R}], \widehat{\text{CR}}[\mathcal{R}, \delta])$ to $(\text{CR}[\mathcal{R}_{\text{acc}}], \widehat{\text{CR}}[\mathcal{R}_{\text{acc}}, \delta])$ with post-quantum straightline knowledge soundness errors $(0, \varepsilon_{\text{com}}, \varepsilon_{\text{ext}})$, where $\varepsilon_{\text{com}}(q) \leq 240(q + 1)/2^{\lambda_H}$ and

$$\begin{aligned} \varepsilon_{\text{ext}}(Q) \leq & 320(Q + \nu + 1)(Q + \nu) \varepsilon_{\text{rr}}(\delta) + \frac{4(Q + \nu + 1)\nu}{2^{\beta_{\min}}} \\ & + \frac{32l_{\max} + 1280Q(2Q + 1)^2 + 128q_s \lceil \log_2 l_{\max} \rceil}{2^{\lambda_H}} \\ & + \frac{960(Q + 1)T_1^2 + 480q_V^2(Q + \nu + q_V + 2)}{2^{\lambda_H}}. \end{aligned}$$

Derivation from [CDHZ25]. By [CDHZ25, Theorem 6.10], Π has post-quantum state-restoration straightline knowledge soundness error $\varepsilon^{\text{sr}} \leq 80(Q + \nu + 1)(\mathbf{m}_{\text{FS}} + \nu) \varepsilon_{\text{rr}}(\delta) + (Q + \nu + 1)\nu/2^{\beta_{\min}}$ for a Q -query prover with total query mass $\mathbf{m}_{\text{FS}}$ to the challenge oracle. By [CDHZ25, Theorem 8.3], the salted Merkle commitments are post-quantum multi-extractable with errors, at query bound Q' ,

$$\begin{aligned} \varepsilon_{\text{com}}(Q') &\leq \frac{240(Q' + 1)}{2^{\lambda_H}}, & \varepsilon_{\text{extract}} &\leq \frac{8n_y l_{\max}}{2^{\lambda_H}}, \\ \varepsilon_{\text{offline}} &\leq \frac{160\mathbf{m}Q'^2 + 16q_s \log_2 l_{\max}}{2^{\lambda_H}}, & \varepsilon_{\text{online}} &\leq \frac{240\mathbf{m}'Q'^2}{2^{\lambda_H}}, \end{aligned}$$

where $\mathbf{m}, \mathbf{m}'$ are the total query masses in the third argument of these functions in [CDHZ25]. Substitute these bounds into [CDHZ25, Theorem 11.3] with $n_y = 1$, $n'_y = 0$ and $q_{\widehat{\text{CR}}}' = 0$. Every total query mass is at most the number of queries [CDHZ25, Definition 4.3 and Remark 4.2], so the masses $\mathbf{m}_{\text{FS}}, \mathbf{m}_{\text{VC}}, \mathbf{m}_{\text{MultiExtract}}$ over which the maximum in [CDHZ25, Theorem 11.3] is taken are at most Q . This gives the displayed terms in order: $4\varepsilon^{\text{sr}}$; $4\varepsilon_{\text{extract}}$ with $n_y = 1$; $8\varepsilon_{\text{offline}}$ at query bound $2Q + 1$ with mass $\mathbf{m}_{\text{VC}} \leq Q$ (by [CDHZ25, Remark 11.5] this term vanishes when $n'_y = 0$, and we keep it as an upper bound); $4\varepsilon_{\text{online}}$ at query bound T_1 with mass $\mathbf{m}_{\text{MultiExtract}} + 1 \leq Q + 1$; and $2q_V^2 \varepsilon_{\text{com}}$ at query bound $Q + 1 + \nu + q_V$. $\square$

We use the constants of [CDHZ25, Theorem 8.3] as stated there (version of 2 March 2026); they follow from [CDHZ25, Theorem 8.4, Lemmas 8.7 and 8.8] with the query counts of [CDHZ25, §8.3]. A classical adversary is a special case of a quantum one, so Theorem 3.16 also applies to classical adversaries in the random oracle model, with the same quantum extractor. For classical adversaries, Bünz, Mishra, Nguyen and Wang [BMNW24, Theorem 5.9] give an asymptotic statement of the same shape for their notion of round-by-round knowledge [BMNW24, Definition 5.5]; we do not use it.

4 The Boolean-Kernel Basis

This section introduces the kernel polynomials and shows that their Boolean members form a basis of $\mathbb{F}[X]_{<N}$, with a Kronecker-structured change of basis. It then characterises low degree in the new coordinates and expresses univariate evaluation as a scaled multilinear evaluation. Throughout, $\mathbb{F} \supseteq \mathbb{F}_q$ is a finite field (in the applications, $\mathbb{F} = \mathbb{F}_q$ or an extension field $\mathbb{K}$), and $n \geq 0$. When the number of variables matters we write $K_y^{(n)}$, and otherwise K_y .

4.1 Kernel polynomials

Definition 4.1 (Kernel polynomials). For $y = (y_1, \dots, y_n) \in \mathbb{F}^n$, the *kernel polynomial* of y is

$$K_y(X) = \prod_{k=1}^n (X^{2^{k-1}} + y_k) \in \mathbb{F}[X],$$

with the empty product equal to 1. For $b \in \llbracket 0, N-1 \rrbracket$ we write $K_b = K_{(b_1, \dots, b_n)}$, where $(b_1, \dots, b_n)$ is the bit vector of b . The family $(K_b)_{b \in \llbracket 0, N-1 \rrbracket}$ is the *Boolean kernel family*; its members have coefficients in $\{0, 1\}$, so they do not depend on $\mathbb{F}$. For $n = 0$ the only kernel polynomial is $K_0^{(0)} = 1$.

Every K_y is monic of degree $\sum_{k=1}^n 2^{k-1} = N - 1$.

Example 4.2. For $n = 2$ (so $N = 4$), $K_b(X) = (X + b_1)(X^2 + b_2)$, and $K_0 = X^3$, $K_1 = X^3 + X^2$, $K_2 = X^3 + X$, $K_3 = X^3 + X^2 + X + 1$.

Lemma 4.3 (Coefficient rule). For every $y \in \mathbb{F}^n$ and $a \in \llbracket 0, N-1 \rrbracket$,

$$[X^a] K_y = \prod_{k \in \llbracket 1, n \rrbracket : a_k = 0} y_k,$$

with the empty product equal to 1.

Proof. Expanding the product, each term chooses from factor k either $X^{2^{k-1}}$ or y_k . Encode the choice by $a_k = 1$ for $X^{2^{k-1}}$ and $a_k = 0$ for y_k . The term is then $(\prod_{k: a_k=0} y_k) X^{\sum_k a_k 2^{k-1}} = (\prod_{k: a_k=0} y_k) X^a$. Distinct choices give distinct exponents a by uniqueness of the binary decomposition, so no two terms contribute to the same monomial. $\square$

Corollary 4.4 (Boolean coefficients). For $a, b \in \llbracket 0, N-1 \rrbracket$,

$$[X^a] K_b = \begin{cases} 1 & \text{if } \bar{b} \preceq a, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. By Lemma 4.3, $[X^a] K_b = \prod_{k: a_k=0} b_k$, which is 1 exactly when $a_k = 0 \Rightarrow b_k = 1$ for all k , that is, when $\bar{b}_k = 1 \Rightarrow a_k = 1$ for all k . $\square$

Lemma 4.5 (Recursion). Let $t \in \llbracket 0, n \rrbracket$, and write $b = c + 2^t h$ with $c \in \llbracket 0, 2^t - 1 \rrbracket$ and $h \in \llbracket 0, 2^{n-t} - 1 \rrbracket$ (Section 2.1). Then

$$K_{c+2^t h}^{(n)}(X) = K_c^{(t)}(X) K_h^{(n-t)}(X^{2^t}).$$

In particular, for $n \geq 1$ and $b = b_1 + 2b'$, $K_b^{(n)}(X) = (X + b_1) K_{b'}^{(n-1)}(X^2)$.

Proof. The bits of b are $b_k = c_k$ for $k \leq t$ and $b_k = h_{k-t}$ for $k > t$. Split the product of Definition 4.1 into the factors $k \leq t$, whose product is $K_c^{(t)}(X)$, and the factors $k > t$. For $k > t$, $X^{2^{k-1}} + h_{k-t} = (X^{2^t})^{2^{(k-t)-1}} + h_{k-t}$, and the product of these factors over $k \in \llbracket t+1, n \rrbracket$ is $K_h^{(n-t)}(X^{2^t})$. The last statement is the case $t = 1$, since $K_{b_1}^{(1)} = X + b_1$. $\square$

4.2 Kronecker products

Matrices below have entries in $\mathbb{F}$, and their rows and columns are indexed by $\{0, 1\}$ (for 2×2 matrices) or by $\llbracket 0, N-1 \rrbracket$ (for $N \times N$ matrices).

Definition 4.6 (Kronecker product). For 2×2 matrices $S_1, \dots, S_n$, the $N \times N$ matrix $S_1 \otimes \dots \otimes S_n$ has entries

$$(S_1 \otimes \dots \otimes S_n)[a, b] = \prod_{k=1}^n S_k[a_k, b_k], \quad a, b \in \llbracket 0, N-1 \rrbracket,$$

where factor k acts on bit k . For $n = 0$ it is the 1×1 identity matrix. We write $S^{\otimes n}$ when all factors equal S , and $S_{(k)} = I \otimes \dots \otimes S \otimes \dots \otimes I$ (with S in position k and the 2×2 identity I elsewhere).

Lemma 4.7 (Mixed-product rule). For 2×2 matrices $S_1, \dots, S_n$ and $T_1, \dots, T_n$,

$$(S_1 \otimes \dots \otimes S_n)(T_1 \otimes \dots \otimes T_n) = (S_1 T_1) \otimes \dots \otimes (S_n T_n).$$

In particular $I^{\otimes n}$ is the identity, $S^{\otimes n} T^{\otimes n} = (ST)^{\otimes n}$, and $S^{\otimes n} = S_{(1)} S_{(2)} \dots S_{(n)}$, where the factors $S_{(k)}$ commute.

Proof. Entry $[a, b]$ of the left-hand side is

$$\sum_{c=0}^{N-1} \prod_{k=1}^n S_k[a_k, c_k] T_k[c_k, b_k] = \prod_{k=1}^n \sum_{\beta \in \{0,1\}} S_k[a_k, \beta] T_k[\beta, b_k],$$

by distributivity, since $c \mapsto (c_1, \dots, c_n)$ is a bijection from $\llbracket 0, N-1 \rrbracket$ onto $\{0, 1\}^n$. The right-hand side is $\prod_k (S_k T_k)[a_k, b_k]$, which is the same. For the particular cases: $I^{\otimes n}[a, b] = \prod_k [a_k = b_k]$ equals 1 if $a = b$ and 0 otherwise, by uniqueness of the bit vector; $S_{(1)} \dots S_{(n)}$ is, by induction on the number of factors, the Kronecker product whose factor in position k is the product of n matrices of which exactly one is S and the others are I ; and any two $S_{(k)}, S_{(k')}$ commute because both products equal the Kronecker product with S in positions k and k' . $\square$

4.3 The basis theorem

Let

$$C = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad C^{-1} = \begin{pmatrix} -1 & 1 \\ 1 & 0 \end{pmatrix},$$

so that $C[0, \beta] = \beta$ and $C[1, \beta] = 1$ for $\beta \in \{0, 1\}$; one checks $C^{-1}C = CC^{-1} = I$. For $U \in \mathbb{F}[X]_{<N}$ we write $\text{coef}(U) = ([X^a] U)_{a \in \llbracket 0, N-1 \rrbracket}$ for its *coefficient vector*; the map coef is an $\mathbb{F}$ -linear bijection from $\mathbb{F}[X]_{<N}$ onto $\mathbb{F}^{\llbracket 0, N-1 \rrbracket}$.

Theorem 4.8 (Boolean-kernel basis). Let $n \geq 0$, and let $\mathbf{K}_n$ be the $N \times N$ matrix with entries $\mathbf{K}_n[a, b] = [X^a] K_b$ for $a, b \in \llbracket 0, N-1 \rrbracket$ (rows indexed by coefficient positions, columns by kernel indices).

1. $\mathbf{K}_n = C^{\otimes n}$.
2. $\mathbf{K}_n$ is invertible over every field, with inverse $(C^{-1})^{\otimes n}$.
3. $(K_b)_{b \in \llbracket 0, N-1 \rrbracket}$ is a basis of $\mathbb{F}[X]_{<N}$.

Proof. (1) By Lemma 4.3, $[X^a] K_b = \prod_{k:a_k=0} b_k = \prod_k C[a_k, b_k]$. For $n = 0$ both sides are the 1×1 identity. (2) By Lemma 4.7, $(C^{-1})^{\otimes n} C^{\otimes n} = (C^{-1}C)^{\otimes n} = I^{\otimes n}$, and likewise in the other order. (3) Each K_b has degree $N - 1$ and so lies in $\mathbb{F}[X]_{<N}$, and its coefficient vector is column b of $\mathbf{K}_n$. These N columns are linearly independent by (2), and $\dim \mathbb{F}[X]_{<N} = N$. $\square$

The proof uses no property of $\mathbb{F}$; so $(K_b)_b$ is a basis of $\mathbb{F}[X]_{<N}$ over every field $\mathbb{F}$, of any characteristic.

Remark 4.9 (Determinant). For $n \geq 1$, $\det \mathbf{K}_n = (-1)^{n2^{n-1}}$, and $\det \mathbf{K}_0 = 1$. Indeed, $C^{\otimes n} = C_{(1)} \cdots C_{(n)}$ by Lemma 4.7, and each $C_{(k)}$ is, after a simultaneous permutation of rows and columns grouping the indices into the $N/2$ pairs that differ only in bit k , block diagonal with $N/2$ blocks equal to C ; so $\det C_{(k)} = (\det C)^{N/2} = (-1)^{2^{n-1}}$, since $\det C = -1$. Multiplying over k gives the claim. This fact is not used below.

Definition 4.10 (Kernel coordinates). For $U \in \mathbb{F}[X]_{<N}$, its *kernel coordinates* are the unique table $\lambda : \llbracket 0, N-1 \rrbracket \rightarrow \mathbb{F}$ with $U = \sum_{b=0}^{N-1} \lambda(b) K_b$ (Theorem 4.8). With the coefficient vector $\text{coef}(U)$, we have $\text{coef}(U) = C^{\otimes n} \lambda$ and $\lambda = (C^{-1})^{\otimes n} \text{coef}(U)$.

Since $(C^{-1})^{\otimes n}$ has integer entries, the kernel coordinates of a polynomial with coefficients in a subfield $\mathbb{F}_0 \subseteq \mathbb{F}$ lie in $\mathbb{F}_0$, and do not depend on whether it is regarded in $\mathbb{F}_0[X]$ or in $\mathbb{F}[X]$.

Corollary 4.11 (Explicit transforms). For $U = \sum_b \lambda(b) K_b \in \mathbb{F}[X]_{<N}$:

$$[X^a] U = \sum_{\substack{b \in \llbracket 0, N-1 \rrbracket \\ \bar{b} \leq a}} \lambda(b), \quad \lambda(b) = \sum_{\substack{a \in \llbracket 0, N-1 \rrbracket \\ a \leq \bar{b}}} (-1)^{\text{wt}(\bar{b}) - \text{wt}(a)} [X^a] U.$$

Each of the two transforms is computed with exactly $\frac{n}{2}N$ field additions or subtractions and no multiplications.

Proof. The first formula is Corollary 4.4. For the second, $C^{-1}[\beta, \beta']$ is nonzero only when $(\beta, \beta') \neq (1, 1)$, and then equals -1 if $\beta = \beta' = 0$ and 1 otherwise. Hence $(C^{-1})^{\otimes n}[b, a] = \prod_k C^{-1}[b_k, a_k]$ is nonzero only if $a \leq \bar{b}$, and then equals $(-1)^{|\{k:a_k=b_k=0\}|} = (-1)^{\text{wt}(\bar{b}) - \text{wt}(a)}$. For the cost, $C^{\otimes n} = C_{(1)} \cdots C_{(n)}$ by Lemma 4.7. The matrix $C_{(k)}$ acts on the $N/2$ pairs of entries whose indices differ only in bit k : a pair with values (x, x') , where x sits at the index with bit k equal to 0 , becomes $(x', x + x')$, which costs one addition. Likewise $(C^{-1})_{(k)}$ maps (x, x') to $(x' - x, x)$, and $(C^{-1})^{\otimes n} = (C^{-1})_{(1)} \cdots (C^{-1})_{(n)}$. $\square$

4.4 Low degree in kernel coordinates

Lemma 4.12 (The constant polynomial). For every $n \geq 0$,

$$1 = \sum_{h=0}^{2^n-1} (-1)^{n-\text{wt}(h)} K_h^{(n)}(X).$$

Proof. Expand $1 = \prod_{k=1}^n ((X^{2^{k-1}} + 1) - X^{2^{k-1}})$ by choosing, in factor k , either $X^{2^{k-1}} + 1$ (bit $h_k = 1$) or $-X^{2^{k-1}}$ (bit $h_k = 0$). The term indexed by h is $(-1)^{n-\text{wt}(h)} K_h^{(n)}(X)$. $\square$

Theorem 4.13 (Low-degree criterion). Let $t \in \llbracket 0, n \rrbracket$ and $U = \sum_{b=0}^{N-1} \lambda(b) K_b^{(n)} \in \mathbb{F}[X]_{<N}$. Then $\deg U < 2^t$ if and only if there is a table $\varphi : \llbracket 0, 2^t - 1 \rrbracket \rightarrow \mathbb{F}$ such that

$$\lambda(c + 2^t h) = (-1)^{(n-t)-\text{wt}(h)} \varphi(c) \quad \text{for all } c \in \llbracket 0, 2^t - 1 \rrbracket, h \in \llbracket 0, 2^{n-t} - 1 \rrbracket. \quad (4)$$

In that case φ is unique, $\varphi(c) = \lambda(c + N - 2^t)$, and φ is the table of kernel coordinates of U with t variables: $U = \sum_{c=0}^{2^t-1} \varphi(c) K_c^{(t)}$.

Proof. By Lemma 4.5,

$$U(X) = \sum_{c=0}^{2^t-1} K_c^{(t)}(X) Y_c(X^{2^t}), \quad Y_c(Z) = \sum_{h=0}^{2^{n-t}-1} \lambda(c + 2^t h) K_h^{(n-t)}(Z) \in \mathbb{F}[Z]_{<2^{n-t}}.$$

Put $R_i = \sum_c [X^i] Y_c K_c^{(t)} \in \mathbb{F}[X]_{<2^t}$ for $i \in \llbracket 0, 2^{n-t} - 1 \rrbracket$, so that $U(X) = \sum_i X^{2^t i} R_i(X)$. By Lemma 2.3(2) (with $m = 2^{n-t}$), this is the block split of U , so $\deg U < 2^t$ if and only if $R_i = 0$ for all $i \geq 1$. By Theorem 4.8 with t variables, this holds if and only if $[X^i] Y_c = 0$ for all c and all $i \geq 1$, that is, if and only if every Y_c is a constant $\varphi(c) \in \mathbb{F}$.

By Lemma 4.12 applied with $n - t$ variables, the constant $\varphi(c)$ has kernel coordinates $h \mapsto (-1)^{(n-t)-\text{wt}(h)} \varphi(c)$. By uniqueness of kernel coordinates (Theorem 4.8 with $n - t$ variables), $Y_c = \varphi(c)$ holds if and only if $\lambda(c + 2^t h) = (-1)^{(n-t)-\text{wt}(h)} \varphi(c)$ for all h , which is (4). Taking $h = 2^{n-t} - 1$ (so that $\text{wt}(h) = n - t$) gives $\varphi(c) = \lambda(c + N - 2^t)$, and then $U = \sum_c \varphi(c) K_c^{(t)}$. $\square$

Corollary 4.14 (Reed–Solomon codes in kernel coordinates). *Let $t \in \llbracket 0, n \rrbracket$, and let L be a smooth domain of order $M \geq 2^t$. Then*

$$\text{RS}_{\mathbb{F}}[L, 2^t] = \left\{ \text{ev}_L \left(\sum_b \lambda(b) K_b \right) : \lambda : \llbracket 0, N - 1 \rrbracket \rightarrow \mathbb{F} \text{ satisfies (4) for some } \varphi \right\}.$$

Proof. Immediate from Theorem 4.13, since every polynomial of degree $< 2^t$ lies in $\mathbb{F}[X]_{<N}$ and so has kernel coordinates. $\square$

4.5 Evaluation as a scaled multilinear evaluation

The next result shows that evaluating U at a point $\zeta \in \mathbb{F}$ amounts, up to an explicit scalar, to evaluating the multilinear extension $\tilde{\lambda}$ of its kernel coordinates (Proposition 2.6) at a point $\psi(\zeta)$ of an explicit curve.

Proposition 4.15 (Evaluation identity). *Let $U = \sum_b \lambda(b) K_b \in \mathbb{F}[X]_{<N}$, and let $\zeta \in \mathbb{F}$ satisfy $\gamma_k(\zeta) := 2\zeta^{2^{k-1}} + 1 \neq 0$ for all $k \in \llbracket 1, n \rrbracket$. Define $\psi(\zeta) = (\psi_1(\zeta), \dots, \psi_n(\zeta)) \in \mathbb{F}^n$ by*

$$\psi_k(\zeta) = \frac{\zeta^{2^{k-1}} + 1}{2\zeta^{2^{k-1}} + 1}, \quad k \in \llbracket 1, n \rrbracket.$$

Then

$$U(\zeta) = \left(\prod_{k=1}^n \gamma_k(\zeta) \right) \cdot \tilde{\lambda}(\psi(\zeta)).$$

Proof. Fix k and $\beta \in \{0, 1\}$, and write $\gamma = \gamma_k(\zeta)$ and $\psi = \psi_k(\zeta)$, so that $\gamma\psi = \zeta^{2^{k-1}} + 1$. Then $\gamma \text{eq}_1(\beta, \psi) = \gamma(\beta\psi + (1-\beta)(1-\psi))$ equals $\gamma\psi = \zeta^{2^{k-1}} + 1$ for $\beta = 1$, and $\gamma - \gamma\psi = \zeta^{2^{k-1}}$ for $\beta = 0$; in both cases it equals $\zeta^{2^{k-1}} + \beta$. Taking the product over k gives $K_b(\zeta) = (\prod_k \gamma_k(\zeta)) \text{eq}(b, \psi(\zeta))$ for every $b \in \llbracket 0, N - 1 \rrbracket$. Multiplying by $\lambda(b)$ and summing over b gives the claim by Proposition 2.6. $\square$

Remark 4.16. The condition $\gamma_k(\zeta) \neq 0$ for all k excludes at most $N - 1$ points of $\mathbb{F}$, namely the roots of $\prod_k (2X^{2^{k-1}} + 1)$, a polynomial of degree $N - 1$ with leading coefficient $2^n \neq 0$ (Lemma 2.1). At an excluded point, $U(\zeta) = \sum_b \lambda(b) K_b(\zeta)$ still holds, but the scalar vanishes and the identity carries no information about $\tilde{\lambda}$.

5 The Fold Dictionary

This section proves the central structural fact of the paper. In kernel coordinates, a suitably normalised FRI fold is *exactly* the restriction of the first variable of the multilinear extension of the coordinate table (Definition 2.10 and Lemma 2.11). Consequently, folding a Reed–Solomon codeword n times evaluates the multilinear extension of its kernel coordinates, with no change of representation. As in Section 4, $\mathbb{F} \supseteq \mathbb{F}_q$ is a finite field. We define two folds: pfold_r acts on polynomials and fold_r acts on words; Proposition 5.8 relates them.

5.1 Even–odd split in kernel coordinates

Proposition 5.1 (Even–odd split). *Let $n \geq 1$ and $U = \sum_{b=0}^{N-1} \lambda(b) K_b^{(n)} \in \mathbb{F}[X]_{<N}$, and let $U(X) = U_e(X^2) + X U_o(X^2)$ be its even–odd split, with $U_e, U_o \in \mathbb{F}[X]_{<N/2}$ (Lemma 2.3). Then*

$$U_e = \sum_{b'=0}^{N/2-1} \lambda(2b'+1) K_{b'}^{(n-1)}, \quad U_o = \sum_{b'=0}^{N/2-1} (\lambda(2b') + \lambda(2b'+1)) K_{b'}^{(n-1)}.$$

Proof. By Lemma 4.5, $K_{2b'}^{(n)}(X) = X K_{b'}^{(n-1)}(X^2)$ and $K_{2b'+1}^{(n)}(X) = K_{b'}^{(n-1)}(X^2) + X K_{b'}^{(n-1)}(X^2)$. Hence

$$U(X) = \sum_{b'} \lambda(2b'+1) K_{b'}^{(n-1)}(X^2) + X \sum_{b'} (\lambda(2b') + \lambda(2b'+1)) K_{b'}^{(n-1)}(X^2).$$

Both sums over b' are polynomials of degree $< N/2$ evaluated at X^2 , so the claim follows from the uniqueness in Lemma 2.3(1). $\square$

5.2 The kernel fold of a polynomial

Definition 5.2 (Kernel fold of a polynomial). Let $d \geq 1$. For $r \in \mathbb{F}$ and $U \in \mathbb{F}[X]_{<2d}$ with even–odd split $U = U_e(X^2) + X U_o(X^2)$, define

$$\text{pfold}_r(U) = (2r-1)U_e + (1-r)U_o \in \mathbb{F}[X]_{<d}.$$

The map $U \mapsto \text{pfold}_r(U)$ is $\mathbb{F}$ -linear, by Lemma 2.3. By the coefficient formulas of Lemma 2.3(1), U_e and U_o , and hence $\text{pfold}_r(U)$, do not depend on the choice of d with $U \in \mathbb{F}[X]_{<2d}$: equivalently, $[X^i]U_e = [X^{2i}]U$ and $[X^i]U_o = [X^{2i+1}]U$ for all $U \in \mathbb{F}[X]$ and $i \in \mathbb{N}$.

Theorem 5.3 (Fold dictionary). *Let $n \geq 1$, $U = \sum_{b=0}^{N-1} \lambda(b) K_b^{(n)} \in \mathbb{F}[X]_{<N}$ and $r \in \mathbb{F}$. Then the kernel coordinates of $\text{pfold}_r(U)$, with $n-1$ variables, are the restriction $\text{res}_r(\lambda)$ (Definition 2.10):*

$$\text{pfold}_r(U) = \sum_{b'=0}^{N/2-1} \text{res}_r(\lambda)(b') K_{b'}^{(n-1)}, \quad \text{res}_r(\lambda)(b') = (1-r)\lambda(2b') + r\lambda(2b'+1).$$

Consequently, $\widetilde{\text{res}_r(\lambda)}(y) = \tilde{\lambda}(r, y)$ for $y = (y_1, \dots, y_{n-1})$ (Lemma 2.11).

Proof. By Proposition 5.1, the coordinate of $K_{b'}^{(n-1)}$ in $\text{pfold}_r(U)$ is $(2r-1)\lambda(2b'+1) + (1-r)(\lambda(2b') + \lambda(2b'+1)) = (1-r)\lambda(2b') + r\lambda(2b'+1)$. $\square$

Corollary 5.4 (Iterated folding evaluates the multilinear extension). *Let $U = \sum_{b=0}^{N-1} \lambda(b) K_b^{(n)} \in \mathbb{F}[X]_{<N}$ and $r = (r_1, \dots, r_n) \in \mathbb{F}^n$. Define $U_0 = U$ and $U_j = \text{pfold}_{r_j}(U_{j-1})$ for $j \in \llbracket 1, n \rrbracket$. Then, for $j \in \llbracket 0, n \rrbracket$, the kernel coordinates of U_j (with $n - j$ variables) are the iterated restriction $\text{res}_{r_{\leq j}}(\lambda)$, that is,*

$$U_j = \sum_{b=0}^{2^{n-j}-1} \tilde{\lambda}(r_{\leq j}, b) K_b^{(n-j)}.$$

In particular, U_n is the constant polynomial $\tilde{\lambda}(r_1, \dots, r_n)$.

Proof. Induction on j , using Theorem 5.3 for the step and Definition 2.10 for the iterated restriction; the displayed formula is Lemma 2.11(2). For $j = n$, $K_0^{(0)} = 1$. $\square$

5.3 The kernel fold of a word

Let L be any smooth domain of order $M \geq 2$; the definitions below are used for all such domains, in particular for the domains L^{2^j} . For a word $w \in \mathbb{F}^L$, let $w_e, w_o \in \mathbb{F}^{L^2}$ be its even and odd parts (1).

Definition 5.5 (Kernel fold of a word). For $r \in \mathbb{F}$ and $w \in \mathbb{F}^L$, define $\text{fold}_r(w) \in \mathbb{F}^{L^2}$ by

$$\text{fold}_r(w) = (2r - 1)w_e + (1 - r)w_o.$$

Lemma 5.6 (Properties of the word fold). *Let $w \in \mathbb{F}^L$ and $r \in \mathbb{F}$.*

1. (Explicit formula.) *For every $\xi \in L$,*

$$\text{fold}_r(w)(\xi^2) = \frac{((2r - 1)\xi + (1 - r))w(\xi) + ((2r - 1)\xi - (1 - r))w(-\xi)}{2\xi}. \quad (5)$$

2. (Linearity.) *$w \mapsto \text{fold}_r(w)$ is $\mathbb{F}$ -linear.*

3. (Locality.) *For $\eta \in L^2$, $\text{fold}_r(w)(\eta)$ depends only on the values of w on the fibre over η (Definition 2.16); a verifier computes it from two queries.*

Proof. (1) Substitute (1) into the definition and put everything over the denominator 2ξ : $(2r - 1) \frac{w(\xi) + w(-\xi)}{2} = \frac{(2r-1)\xi(w(\xi) + w(-\xi))}{2\xi}$ and $(1 - r) \frac{w(\xi) - w(-\xi)}{2\xi}$; collect the coefficients of $w(\xi)$ and $w(-\xi)$. (2) and (3) follow from Lemma 2.22(1,2). $\square$

Lemma 5.7 (Line form). *For $w \in \mathbb{F}^L$, put $u^0 = w_o - w_e$ and $u^1 = 2w_e - w_o$, both in $\mathbb{F}^{L^2}$. Then $\text{fold}_r(w) = u^0 + r u^1$ for every $r \in \mathbb{F}$. Conversely, $w_e = u^0 + u^1$ and $w_o = 2u^0 + u^1$.*

Proof. $(2r - 1)w_e + (1 - r)w_o = (w_o - w_e) + r(2w_e - w_o)$; and $u^0 + u^1 = w_e$, $2u^0 + u^1 = w_o$. $\square$

Proposition 5.8 (Consistency with the polynomial fold). *Let $1 \leq d \leq M/2$, $r \in \mathbb{F}$ and $U \in \mathbb{F}[X]_{<2d}$. Then*

$$\text{fold}_r(\text{ev}_L(U)) = \text{ev}_{L^2}(\text{pfold}_r(U)).$$

In particular, fold_r maps $\text{RS}_{\mathbb{F}}[L, 2d]$ into $\text{RS}_{\mathbb{F}}[L^2, d]$, and $P_{\text{fold}_r(c)} = \text{pfold}_r(P_c)$ for every $c \in \text{RS}_{\mathbb{F}}[L, 2d]$.

Proof. By Lemma 2.22(4), $\text{ev}_L(U)_e = \text{ev}_{L^2}(U_e)$ and $\text{ev}_L(U)_o = \text{ev}_{L^2}(U_o)$; since ev_{L^2} is linear, $\text{fold}_r(\text{ev}_L(U)) = \text{ev}_{L^2}((2r - 1)U_e + (1 - r)U_o)$. Moreover $\text{pfold}_r(U) \in \mathbb{F}[X]_{<d}$ and $d \leq |L^2|$, so the last statement follows from Lemma 2.19(1). $\square$

Corollary 5.9 (Folding a committed codeword). *Let L be a smooth domain of order $M = 2^\mu$ with $\mu \geq n$, let $\lambda : \llbracket 0, N-1 \rrbracket \rightarrow \mathbb{F}$, and let $w_0 = \text{ev}_L(U)$ with $U = \sum_b \lambda(b)K_b$. For $r \in \mathbb{F}^n$, define $w_j = \text{fold}_{r_j}(w_{j-1}) \in \mathbb{F}^{L^{2^j}}$ for $j \in \llbracket 1, n \rrbracket$. Then $w_j = \text{ev}_{L^{2^j}}(U_j)$, with U_j as in Corollary 5.4; in particular w_n is the constant word on L^{2^n} with value $\tilde{\lambda}(r_1, \dots, r_n)$.*

Proof. Induction on j . For the step, apply Proposition 5.8 on the domain $L^{2^{j-1}}$, of order $M/2^{j-1} \geq 2$ (Lemma 2.15), with $2d = N/2^{j-1} \leq M/2^{j-1}$ and $(L^{2^{j-1}})^2 = L^{2^j}$. Conclude with Corollary 5.4. $\square$

5.4 Relation to classical FRI folding

Definition 5.10 (Classical FRI fold). The classical FRI fold [BSBHR18] with challenge $\theta \in \mathbb{F}$ is $\text{cfold}_\theta(U) = U_e + \theta U_o$ for $U \in \mathbb{F}[X]$ (which, as for pfold , does not depend on a degree bound), and $\text{cfold}_\theta(w) = w_e + \theta w_o$ for a word w on a smooth domain of order at least 2.

Proposition 5.11 (Classical versus kernel fold). *1. Let $n \geq 1$, and let $U \in \mathbb{F}[X]_{<N}$ have kernel coordinates λ . Then the kernel coordinates of $\text{cfold}_\theta(U)$, with $n-1$ variables, are*

$$b' \mapsto \theta \lambda(2b') + (1 + \theta) \lambda(2b' + 1).$$

2. As maps $\mathbb{F}[X]_{<2d} \rightarrow \mathbb{F}[X]_{<d}$, for every $d \geq 1$: if $\theta \neq -1/2$, then $\text{cfold}_\theta = (1 + 2\theta) \text{pfold}_r$ with $r = (1 + \theta)/(1 + 2\theta)$. Conversely, if $r \neq 1/2$, then $\text{pfold}_r = (2r - 1) \text{cfold}_\theta$ with $\theta = (1 - r)/(2r - 1)$.

3. Let $U = \sum_b \lambda(b)K_b \in \mathbb{F}[X]_{<N}$ and $\theta_1, \dots, \theta_n \in \mathbb{F} \setminus \{-1/2\}$. Applying $\text{cfold}_{\theta_1}, \dots, \text{cfold}_{\theta_n}$ successively to U gives the constant

$$\prod_{j=1}^n (1 + 2\theta_j) \cdot \tilde{\lambda}\left(\frac{1+\theta_1}{1+2\theta_1}, \dots, \frac{1+\theta_n}{1+2\theta_n}\right).$$

4. For $\zeta \in \mathbb{F}$, applying $\text{cfold}_{\theta_1}, \dots, \text{cfold}_{\theta_n}$ with $\theta_j = \zeta^{2^{j-1}}$ to U gives the constant $U(\zeta)$.

Proof. (1) By Proposition 5.1, the coordinate of $K_{b'}^{(n-1)}$ in $U_e + \theta U_o$ is $\lambda(2b' + 1) + \theta(\lambda(2b') + \lambda(2b' + 1))$. (2) With $r = (1 + \theta)/(1 + 2\theta)$ we have $2r - 1 = 1/(1 + 2\theta)$ and $1 - r = \theta/(1 + 2\theta)$, so $(1 + 2\theta)((2r - 1)U_e + (1 - r)U_o) = U_e + \theta U_o$. The converse is the same computation with $\theta = (1 - r)/(2r - 1)$, for which $1 + 2\theta = 1/(2r - 1)$ and $(1 + \theta)/(1 + 2\theta) = r$. (3) By (2) and the linearity of cfold and pfold , the composition equals $\prod_j (1 + 2\theta_j)$ times $\text{pfold}_{r_n} \circ \dots \circ \text{pfold}_{r_1}(U)$ with $r_j = (1 + \theta_j)/(1 + 2\theta_j)$; conclude with Corollary 5.4. (4) Let $V_0 = U$ and $V_j = \text{cfold}_{\theta_j}(V_{j-1})$. We show $V_{j-1}(\zeta^{2^{j-1}}) = U(\zeta)$ for $j \in \llbracket 1, n+1 \rrbracket$ by induction: the case $j = 1$ is trivial, and $V_{j-1}(X) = (V_{j-1})_e(X^2) + X(V_{j-1})_o(X^2)$ gives $V_{j-1}(\zeta^{2^{j-1}}) = (V_{j-1})_e(\zeta^{2^j}) + \zeta^{2^{j-1}}(V_{j-1})_o(\zeta^{2^j}) = V_j(\zeta^{2^j})$. For $j = n + 1$, V_n is a constant, equal to $U(\zeta)$. $\square$

For ζ with $\gamma_j(\zeta) \neq 0$ for all j , items 3 and 4 with $\theta_j = \zeta^{2^{j-1}}$ give a second proof of Proposition 4.15: $1 + 2\theta_j = \gamma_j(\zeta)$ and $(1 + \theta_j)/(1 + 2\theta_j) = \psi_j(\zeta)$.

Remark 5.12 (A consistency pitfall). By Proposition 5.11(2), the two folds differ by a scalar and by a reparametrisation of the challenge. An implementation must therefore use *the same* fold on the committed word and on the table that the verifier tracks. Folding the word with cfold_θ while restricting the table with $r = \theta$ (or folding with pfold_θ while tracking cfold_θ) yields in general

a final constant different from $\tilde{\lambda}(r_1, \dots, r_n)$, even for an honest prover; see Example 5.13. In an early version of our implementation this mismatch caused honest proofs to fail the final consistency check; Definition 5.5 together with Theorem 5.3 fixes the convention.

Example 5.13. Over $\mathbb{F}_{17}$ with $n = 1$, take $\lambda = (\lambda(0), \lambda(1)) = (1, 0)$, so $U = K_0 = X$, $U_e = 0$ and $U_o = 1$. With $\theta = 2$, the classical fold gives $\text{cfold}_2(U) = 2$, while the restriction at $r = 2$ gives $\text{res}_2(\lambda)(0) = (1 - 2) \cdot 1 + 2 \cdot 0 = -1 = 16$. The kernel fold agrees with the restriction: $\text{pfold}_2(U) = 3 \cdot 0 + (1 - 2) \cdot 1 = 16$. So the classical final constant 2 differs from $\tilde{\lambda}(2) = 16$.

Remark 5.14 (Comparison with coefficient-form folding). In the monomial basis, cfold_θ acts on coefficients by $[X^{a'}] \text{cfold}_\theta(U) = [X^{2a'}] U + \theta [X^{2a'+1}] U$ (Lemma 2.3(1)). By induction on n , applying $\text{cfold}_{\theta_1}, \dots, \text{cfold}_{\theta_n}$ gives $\sum_a [X^a] U \prod_{k:a_k=1} \theta_k = \sum_a [X^a] U m_a(\theta_1, \dots, \theta_n)$, the evaluation at $(\theta_1, \dots, \theta_n)$ of the multilinear polynomial whose *coefficient form* (Definition 2.8) is $\text{coef}(U)$. This is the mechanism behind Reed–Solomon instantiations of BaseFold [ZCF24] and behind DeepFold [GLH⁺25]. Corollary 5.9 is its counterpart for the *table form*: if the prover encodes a table f as $U = \sum_b f(b) K_b$, folding computes $\tilde{f}(r_1, \dots, r_n)$ directly, without converting f to coefficient form.

6 The Polynomial Commitment Scheme

We now use the fold dictionary to build a multilinear polynomial commitment scheme for tables, in the sense of Definition 3.10. The prover encodes a table f as the Reed–Solomon codeword of $U_f = \sum_b f(b) K_b$. An evaluation claim $\tilde{f}(z) = v$ is proved by a sumcheck (Section 3.3) whose round challenges are also the fold challenges, exactly as in BaseFold [ZCF24]. By Corollary 5.9, no change of representation is needed: the folds of the committed word track the partially restricted table that the sumcheck prover already holds. We present the scheme as a public-coin IOP (Definition 3.1); its compilation into a non-interactive scheme is discussed in Sections 3.5 and 7.7.

6.1 Parameters and encoding

Fields. The smooth domain lies in the base field $\mathbb{F}_q$, while tables, words, the evaluation point and the challenges live in a finite field $\mathbb{F} \supseteq \mathbb{F}_q$, the *challenge field*. Taking $\mathbb{F} = \mathbb{F}_q$ gives the plain scheme; taking $\mathbb{F} = \mathbb{K}$, an extension of $\mathbb{F}_q$, gives the scheme with extension-field challenges used in Section 8, in which honest tables have entries in $\mathbb{F}_q$. All results of Sections 6 and 7 hold for every such $\mathbb{F}$, with $|\mathbb{F}|$ in the error bounds.

Parameters.

- the number of variables $n \geq 1$, and $N = 2^n$;
- a rate $\rho = 2^{-R}$ with $R \geq 1$, and a smooth domain $L \leq \mathbb{F}_q^\times$ of order $M = N/\rho = 2^{n+R}$ (Definition 2.14; it exists if and only if $2^{n+R} \mid q - 1$, by Lemma 2.15);
- a number of folding rounds $\ell \in \llbracket 0, n \rrbracket$;
- a number of queries $\kappa \geq 1$.

For $j \in \llbracket 0, \ell \rrbracket$ let $L_j = L^{2^j}$, the smooth domain of order $M_j = M/2^j \geq 2^R \geq 2$ (Lemma 2.15), let $N_j = N/2^j$, and let $\mathcal{C}_j = \text{RS}_{\mathbb{F}}[L_j, N_j]$. Every code $\mathcal{C}_j$ has the same rate $N_j/M_j = \rho$, and $L_{j+1} = (L_j)^2$.

Definition 6.1 (Encoding). The *encoding* of a table $f : \llbracket 0, N-1 \rrbracket \rightarrow \mathbb{F}$ is the word

$$\text{Enc}(f) = \text{ev}_L(U_f) \in \mathcal{C}_0, \quad U_f = \sum_{b=0}^{N-1} f(b) K_b \in \mathbb{F}[X]_{<N}.$$

Lemma 6.2 (Encoding). *Enc is an $\mathbb{F}$ -linear bijection from the tables over $\mathbb{F}$ onto $\mathcal{C}_0$, and for $c \in \mathcal{C}_0$, the table f with $\text{Enc}(f) = c$ is the table of kernel coordinates of P_c .*

Proof. $f \mapsto U_f$ is a linear bijection onto $\mathbb{F}[X]_{<N}$ by Theorem 4.8, and ev_L is a linear bijection from $\mathbb{F}[X]_{<N}$ onto $\mathcal{C}_0$ by Lemma 2.19(1), with inverse $c \mapsto P_c$. $\square$

Enc is computed by the transform of Corollary 4.11 ($\frac{n}{2}N$ additions), followed by one number-theoretic transform (NTT, the fast evaluation of a polynomial on a smooth domain) of size M on the zero-padded coefficient vector.

6.2 The evaluation protocol

The commitment to f is the word $w_0 = \text{Enc}(f)$. The evaluation protocol Π_{eval} proves the claim $\tilde{f}(z) = v$ for a public point $z = (z_1, \dots, z_n) \in \mathbb{F}^n$ and a public value $v \in \mathbb{F}$. As in Definition 3.10, its instance is $(w_0; z, v)$, and the verifier has oracle access to w_0 .

Round structure. Π_{eval} is a public-coin IOP with $\ell + 1$ rounds.

- In round $j \in \llbracket 1, \ell \rrbracket$, the prover sends a *round polynomial* $s_j \in \mathbb{F}[X]_{<3}$, together with, if $j \geq 2$, an oracle $w_{j-1} \in \mathbb{F}^{L_{j-1}}$; the verifier replies with a uniformly random challenge $r_j \in \mathbb{F}$.
- In round $\ell + 1$, the prover sends a *final table* $g : \llbracket 0, N_\ell - 1 \rrbracket \rightarrow \mathbb{F}$ in the clear; the verifier replies with κ independent uniformly random *query points* $\xi_0^{(1)}, \dots, \xi_0^{(\kappa)} \in L$, and then decides.

Thus the challenge sets are $\text{Ch}_j = \mathbb{F}$ for $j \in \llbracket 1, \ell \rrbracket$ and $\text{Ch}_{\ell+1} = L^{\times \kappa}$, the set of κ -tuples of elements of L (a Cartesian power, not the image of a power map). A round polynomial is transmitted as its values at 0, 1 and 2, which are distinct because the characteristic is odd. The map $\mathbb{F}[X]_{<3} \rightarrow \mathbb{F}^3$, $s \mapsto (s(0), s(1), s(2))$, is injective by Lemma 2.1, hence bijective since both spaces have dimension 3; so every triple sent by a prover identifies a unique $s_j \in \mathbb{F}[X]_{<3}$. As in Section 3.1, the checks below are predicates on the full transcript.

Honest prover. The honest prover keeps two tables over $\mathbb{F}$, initialised to

$$f_0 = f \quad \text{and} \quad \chi_0 = (b \mapsto \text{eq}(b, z)), \quad b \in \llbracket 0, N-1 \rrbracket,$$

and after round j replaces them by their restrictions (Definition 2.10) at r_j : $f_j = \text{res}_{r_j}(f_{j-1})$ and $\chi_j = \text{res}_{r_j}(\chi_{j-1})$, tables on $\llbracket 0, N_j - 1 \rrbracket$. In round j it sends

$$s_j(X) = \sum_{b=0}^{N_j-1} ((1-X)f_{j-1}(2b) + Xf_{j-1}(2b+1)) ((1-X)\chi_{j-1}(2b) + X\chi_{j-1}(2b+1)), \quad (6)$$

and, if $j \geq 2$, the oracle $w_{j-1} = \text{fold}_{r_{j-1}}(w_{j-2})$ (Definition 5.5). In round $\ell + 1$ it sends $g = f_\ell$.

Verifier. Set $v_0 = v$ and $v_j = s_j(r_j)$ for $j \in \llbracket 1, \ell \rrbracket$. Let $G = \sum_{b=0}^{N_\ell-1} g(b) K_b^{(n-\ell)} \in \mathbb{F}[X]_{<N_\ell}$ and $w_\ell = \text{ev}_{L_\ell}(G)$; the verifier computes values of w_ℓ itself. The verifier accepts if and only if all of the following checks pass.

1. *Sumcheck.* $s_j(0) + s_j(1) = v_{j-1}$ for every $j \in \llbracket 1, \ell \rrbracket$.

2. *Closure.*

$$v_\ell = \left(\prod_{j=1}^{\ell} \text{eq}_1(r_j, z_j) \right) \cdot \tilde{g}(z_{\ell+1}, \dots, z_n).$$

3. *Queries.* For every $i \in \llbracket 1, \kappa \rrbracket$, with $\xi_0 = \xi_0^{(i)}$ and $\xi_j = \xi_0^{2^j}$ for $j \in \llbracket 1, \ell \rrbracket$:

- if $\ell \geq 1$: for every $j \in \llbracket 1, \ell \rrbracket$, the value $\text{fold}_{r_j}(w_{j-1})(\xi_j)$, computed by (5) from the two queried values $w_{j-1}(\pm \xi_{j-1})$, equals $w_j(\xi_j)$ (a query to the oracle w_j if $j < \ell$, and $G(\xi_\ell)$ if $j = \ell$);
- if $\ell = 0$: $w_0(\xi_0) = G(\xi_0)$.

Remark 6.3 (Stopping parameter). With $\ell = n$, the final table is the single value $\tilde{f}(r_1, \dots, r_n)$ and G is a constant; with $\ell = 0$, the prover sends f in the clear. Intermediate values trade the $N_\ell = 2^{n-\ell}$ field elements of the final message against ℓ folding rounds.

6.3 Committing to fewer words

In Π_{eval} the prover commits to every folded word $w_1, \dots, w_{\ell-1}$, and each query opens one fibre of each of them. Because the fold is local (Lemma 5.6(3)), the verifier can instead read a larger piece of one word and compute several folds itself; the prover then commits only to some of the words.

Committed levels. Let $1 \leq \ell \leq n$ and let $\mathcal{J} \subseteq \llbracket 0, \ell-1 \rrbracket$ with $0 \in \mathcal{J}$, the set of *committed levels*. The set $\mathcal{J}$ is a public parameter of the scheme, fixed with the commitment, like ℓ . For $j \in \mathcal{J}$, let j^+ be the next element of $\mathcal{J}$ after j , or ℓ if there is none, and let $d_j = j^+ - j \geq 1$. For $\xi \in L_j$, the *coset* of ξ at level j is

$$S_j(\xi) = \{ \zeta \in L_j : \zeta^{2^{d_j}} = \xi^{2^{d_j}} \},$$

the 2^{d_j} preimages of $\xi^{2^{d_j}} \in L_{j^+}$ under the 2^{d_j} -th power map; it has 2^{d_j} elements because L_j is cyclic of order $M_j = 2^{n+R-j}$, a multiple of 2^{d_j} since $d_j \leq \ell - j < n + R - j$, and it is a union of 2^{d_j-1} fibres, since $-1 \in L_j$ and $(-\zeta)^{2^{d_j}} = \zeta^{2^{d_j}}$. For $j \in \mathcal{J}$, challenges $r = (r_1, \dots, r_\ell)$ and a word $w \in \mathbb{F}^{L_j}$, write

$$\text{fold}_r^{(j)}(w) = \text{fold}_{r_{j^+}}(\dots \text{fold}_{r_{j+2}}(\text{fold}_{r_{j+1}}(w)) \dots) \in \mathbb{F}^{L_{j^+}}.$$

Lemma 6.4 (Local folding). *Let $j \in \mathcal{J}$, $w \in \mathbb{F}^{L_j}$ and $\xi \in L_j$. The value $\text{fold}_r^{(j)}(w)(\xi^{2^{d_j}})$ depends only on the values of w on $S_j(\xi)$, and a verifier computes it from these 2^{d_j} values by $2^{d_j} - 1$ applications of (5).*

Proof. For $0 \leq i \leq d_j$, the image $T_i = \{ \zeta^{2^i} : \zeta \in S_j(\xi) \}$ is the set of 2^{d_j-i} points of L_{j+i} whose 2^{d_j-i} -th power is $\xi^{2^{d_j}}$ (the power map from L_j onto L_{j+i} is surjective); it is closed under negation for $i < d_j$, so it is the union of the fibres over the points of T_{i+1} . Call w the 0-th fold and $\text{fold}_{r_{j+i}}$ of the $(i-1)$ -th fold the i -th fold. By Lemma 5.6(3), the values of the i -th fold on T_i are computed from the values of the $(i-1)$ -th fold on T_{i-1} , one application of (5) per point of T_i . Induction on i , and $|T_1| + \dots + |T_{d_j}| = 2^{d_j-1} + \dots + 1 = 2^{d_j} - 1$. $\square$

The protocol $\Pi_{\text{eval}}^{\mathcal{J}}$. $\Pi_{\text{eval}}^{\mathcal{J}}$ is Π_{eval} with two changes.

- In round $j \in \llbracket 2, \ell \rrbracket$, the prover sends the oracle w_{j-1} only if $j-1 \in \mathcal{J}$; the honest oracle is, as before, $w_{j-1} = \text{fold}_{r_{j-1}}(\cdots \text{fold}_{r_1}(w_0))$.
- The query check is replaced by: for every query point $\xi_0 = \xi_0^{(i)}$ and every $j \in \mathcal{J}$, with $\xi_j = \xi_0^{2^j}$, the verifier reads w_j on $S_j(\xi_j)$, computes $\text{fold}_r^{(j)}(w_j)(\xi_{j+})$ (Lemma 6.4), and checks that it equals $w_{j+}(\xi_{j+})$, a query to the oracle w_{j+} (which lies in $S_{j+}(\xi_{j+})$) if $j^+ < \ell$, and $G(\xi_\ell)$ if $j^+ = \ell$.

The sumcheck and closure checks are unchanged. For $\mathcal{J} = \llbracket 0, \ell-1 \rrbracket$, every d_j is 1, every coset is a fibre, and $\Pi_{\text{eval}}^{\mathcal{J}} = \Pi_{\text{eval}}$. The honest prover of Π_{eval} , with the oracles of the levels outside $\mathcal{J}$ omitted, makes the verifier of $\Pi_{\text{eval}}^{\mathcal{J}}$ accept with probability 1: its words satisfy $w_{j+} = \text{fold}_r^{(j)}(w_j)$ on all of L_{j+} (proof of Theorem 6.7).

Augmentation. For a transcript τ of $\Pi_{\text{eval}}^{\mathcal{J}}$ (partial or full), let $\text{aug}(\tau)$ be the transcript of Π_{eval} obtained by inserting, in round $j+1$ for every $j \in \llbracket 1, \ell-1 \rrbracket \setminus \mathcal{J}$ that τ reaches, the oracle $w_j = \text{fold}_{r_j}(w_{j-1})$; here w_{j-1} is an oracle of τ , the instance oracle w_0 , or a word inserted before. The inserted word depends only on the part of $\text{aug}(\tau)$ before round $j+1$.

Lemma 6.5 (Skipping commitments). *For every instance and every full transcript τ of $\Pi_{\text{eval}}^{\mathcal{J}}$, if the verifier of $\Pi_{\text{eval}}^{\mathcal{J}}$ accepts τ , the verifier of Π_{eval} accepts $\text{aug}(\tau)$. Consequently, for every prover $\tilde{P}$ of $\Pi_{\text{eval}}^{\mathcal{J}}$, the prover $\tilde{P}'$ of Π_{eval} that sends the messages and oracles of $\tilde{P}$ and, in round $j+1$ for $j \in \llbracket 1, \ell-1 \rrbracket \setminus \mathcal{J}$, the oracle $w_j = \text{fold}_{r_j}(w_{j-1})$, is accepted, for every instance and every choice of the challenges, whenever $\tilde{P}$ is.*

Proof. The round polynomials, the final table and the challenges are the same in τ and $\text{aug}(\tau)$, so the sumcheck and closure checks are the same. Fix a query point ξ_0 and $j \in \mathcal{J}$. For $i \in \llbracket j+1, j^+-1 \rrbracket$, $i \notin \mathcal{J}$, so $w_i = \text{fold}_{r_i}(w_{i-1})$ in $\text{aug}(\tau)$, and by induction $w_{j^+-1} = \text{fold}_{r_{j^+-1}}(\cdots \text{fold}_{r_{j+1}}(w_j))$. The checks of Π_{eval} at the levels $i \in \llbracket j+1, j^+-1 \rrbracket$ compare $\text{fold}_{r_i}(w_{i-1})(\xi_i)$ with $w_i(\xi_i)$ and hold by construction. The check at level j^+ compares $\text{fold}_{r_{j^+}}(w_{j^+-1})(\xi_{j^+}) = \text{fold}_r^{(j)}(w_j)(\xi_{j^+})$ with $w_{j+}(\xi_{j^+})$ (or $G(\xi_\ell)$), which is the check of $\Pi_{\text{eval}}^{\mathcal{J}}$ for ξ_0 and j . Every level $i \in \llbracket 1, \ell \rrbracket$ lies in exactly one interval $\llbracket j+1, j^+ \rrbracket$ with $j \in \mathcal{J}$, so all checks of Π_{eval} pass. For the second statement, the transcript of $\tilde{P}'$ is aug of the transcript of $\tilde{P}$, and $\tilde{P}'$ is a prover: its oracle in round $j+1$ depends only on r_j and on messages already sent. $\square$

Lemma 6.5 transfers soundness and evaluation binding from Π_{eval} to $\Pi_{\text{eval}}^{\mathcal{J}}$ with the same errors; the round-by-round notions are transferred in Section 7.8. Section 8 uses $\mathcal{J} = \{0, k, 2k, \dots\} \cap \llbracket 0, \ell-1 \rrbracket$ with $k = 4$: the prover commits to one word in four, and each query reads $2^{d_j} \leq 2^k$ values of each committed word w_j .

6.4 Completeness

Recall from Section 2.1 the notation $(r_{\leq j}, b) = (r_1, \dots, r_j, b_1, \dots, b_{n-j})$ for an index b with $n-j$ bits.

Lemma 6.6 (Honest prover). *Let f be a table over $\mathbb{F}$, $z \in \mathbb{F}^n$, and $r_1, \dots, r_\ell \in \mathbb{F}$, and let f_j , χ_j and s_j be computed by the honest prover. Then, for $j \in \llbracket 0, \ell \rrbracket$ and $b \in \llbracket 0, N_j-1 \rrbracket$:*

1. $f_j(b) = \tilde{f}(r_{\leq j}, b)$ and $\chi_j(b) = \text{eq}((r_{\leq j}, b), z)$;

2. for $j \geq 1$, s_j is the honest round polynomial s_j^* of Section 3.3 for $F(x) = \tilde{f}(x) \text{eq}(x, z)$, and $s_j \in \mathbb{F}[X]_{<3}$;
3. for $j \geq 1$, $s_j(r_j) = \sum_b f_j(b) \chi_j(b)$, and $s_j(0) + s_j(1) = \sum_b f_{j-1}(b) \chi_{j-1}(b)$;
4. $\sum_b f_0(b) \chi_0(b) = \tilde{f}(z)$ and, for every j ,

$$\sum_{b=0}^{N_j-1} f_j(b) \chi_j(b) = \left(\prod_{i=1}^j \text{eq}_1(r_i, z_i) \right) \cdot \tilde{f}_j(z_{j+1}, \dots, z_n).$$

Proof. (1) Lemma 2.11(2), and Corollary 2.7 for χ_0 . (2) Regard f_{j-1} and χ_{j-1} as tables over the ring $\mathbb{F}[X] \supseteq \mathbb{F}$ (Section 2.3). The two factors in (6) are then $\text{res}_X(f_{j-1})(b)$ and $\text{res}_X(\chi_{j-1})(b)$, and Lemma 2.11(2) over $\mathbb{F}[X]$, with (1), gives

$$s_j(X) = \sum_{b=0}^{N_j-1} \tilde{f}(r_1, \dots, r_{j-1}, X, b_1, \dots, b_{n-j}) \text{eq}((r_1, \dots, r_{j-1}, X, b_1, \dots, b_{n-j}), z) = s_j^*(X).$$

Each summand of (6) is a product of two polynomials of degree ≤ 1 in X , so $s_j \in \mathbb{F}[X]_{<3}$. (3) By Lemma 2.12 applied to the evaluation $X \mapsto r_j$, the factors become $f_j(b)$ and $\chi_j(b)$. Setting $X = 0$ and $X = 1$ in (6) gives $\sum_b (f_{j-1}(2b) \chi_{j-1}(2b) + f_{j-1}(2b+1) \chi_{j-1}(2b+1))$. (4) The first identity is Proposition 2.6. For the second, (1) and Lemma 2.5(3) give

$$\chi_j(b) = \left(\prod_{i=1}^j \text{eq}_1(r_i, z_i) \right) \text{eq}(b, (z_{j+1}, \dots, z_n));$$

conclude with Proposition 2.6. □

Theorem 6.7 (Completeness). *If $w_0 = \text{Enc}(f)$, $v = \tilde{f}(z)$ and the prover is honest, the verifier accepts with probability 1.*

Proof. Sumcheck. By Lemma 6.6(3,4), $v_0 = \tilde{f}(z) = \sum_b f_0(b) \chi_0(b)$ and $v_j = s_j(r_j) = \sum_b f_j(b) \chi_j(b)$, so $s_j(0) + s_j(1) = \sum_b f_{j-1}(b) \chi_{j-1}(b) = v_{j-1}$ for every $j \in \llbracket 1, \ell \rrbracket$.

Closure. By Lemma 6.6(4) with $j = \ell$ and $g = f_\ell$, $v_\ell = \prod_j \text{eq}_1(r_j, z_j) \cdot \tilde{g}(z_{\ell+1}, \dots, z_n)$.

Queries. If $\ell = 0$, then $g = f$, $G = U_f$ and $w_0 = \text{Enc}(f) = \text{ev}_L(G)$. If $\ell \geq 1$, extend $(r_1, \dots, r_\ell)$ arbitrarily to $r \in \mathbb{F}^n$, and let U_j be as in Corollary 5.4 for $U = U_f$; its kernel coordinates are f_j . Since L has order 2^{n+R} with $n+R \geq n$, Corollary 5.9 shows that the successive folds of w_0 satisfy $\text{fold}_{r_j}(\dots \text{fold}_{r_1}(w_0)) = \text{ev}_{L_j}(U_j)$ for $j \in \llbracket 1, \ell \rrbracket$. The honest oracles are these folds for $j < \ell$, and $U_\ell = \sum_b f_\ell(b) K_b^{(n-\ell)} = G$ because $g = f_\ell$. Hence $\text{fold}_{r_j}(w_{j-1}) = w_j$ on all of L_j for every $j \in \llbracket 1, \ell \rrbracket$, and every check of every query passes. □

6.5 Costs

Let t_H denote the cost of one hash evaluation. We count field operations up to constant factors.

- *Commit.* $\frac{n}{2}N$ additions (Corollary 4.11), one NTT of size M , that is, $O(M \log M)$ operations, and a Merkle tree over the $M/2$ fibres of w_0 (one leaf per fibre), $O(M) \cdot t_H$.
- *Prover, evaluation.* Sumcheck: $O(N)$ in total, since each round is linear in the current table size and the sizes halve. Folds: $\text{fold}_{r_j}(w_{j-1})$ costs $O(M_j)$ using precomputed inverses of 2ξ for $\xi \in L$; $O(M)$ in total. Merkle trees for $w_1, \dots, w_{\ell-1}$: $O(M) \cdot t_H$ in total.

- *Verifier.* Sumcheck: $O(\ell)$. Closure: $O(N_\ell)$. Queries: $O(\kappa\ell)$ field operations plus $O(\kappa\ell \log M) \cdot t_H$ for authentication paths, plus κ evaluations of G , $O(\kappa N_\ell)$.
- *Proof size.* For $\ell \geq 1$: 3ℓ field elements for the sumcheck, N_ℓ for the final table, $\ell - 1$ Merkle roots besides the commitment, and $\kappa\ell$ opened leaves ($2\kappa\ell$ values) with their authentication paths. For $\ell = 0$: the table $g = f$ and κ opened values of w_0 .
- *With committed levels $\mathcal{J}$* (Section 6.3): Merkle trees only for the words w_j , $j \in \mathcal{J}$; per query, 2^{d_j} opened values of each w_j , $j \in \mathcal{J}$, and $O(2^{d_j})$ field operations to fold them. With $|\mathcal{J}| = \lceil \ell/k \rceil$ and the openings of one word merged into one path (Section 8.3), this divides the number of opened leaves and of path nodes by about k .

Remark 6.8 (Relation to BaseFold and DeepFold). The interleaving of sumcheck rounds with fold rounds under shared challenges is the BaseFold paradigm [ZCF24], and the asymptotic costs above match Reed–Solomon instantiations of BaseFold and DeepFold [GLH⁺25]. In particular, converting a table to coefficient form by Lemma 2.9 also costs $\frac{n}{2}N$ additions, so the kernel encoding offers no asymptotic advantage. What differs is structural: by Theorem 5.3, the committed word, the sumcheck table and the folds are three views of *one* object, the restricted table f_j , and the consistency between them is an identity rather than a bookkeeping convention (compare Remark 5.12).

Remark 6.9 (Batching and zero knowledge). Because Enc and fold_r are linear, several tables can be opened at the same point by running Π_{eval} on a random linear combination of them, the verifier answering queries to the combined word from queries to each committed word. The soundness analysis of Section 7 covers a single table; batched openings and a zero-knowledge variant are described in Section 10.1.

7 Soundness

We prove that Π_{eval} is sound, evaluation binding, and round-by-round knowledge sound, in the unique-decoding regime. The only external result about codes used is the correlated-agreement theorem (Theorem 2.23) of [BSCI⁺23], which enters through the line form of the kernel fold (Lemma 5.7); the efficiency of the extractor also uses Fact 2.20. Throughout this section we use the notation of Section 6: the challenge field $\mathbb{F} \supseteq \mathbb{F}_q$, the domains L_j of order M_j , and the codes $\mathcal{C}_j = \text{RS}_{\mathbb{F}}[L_j, N_j]$, all of rate ρ . We fix a proximity parameter $0 < \delta \leq (1 - \rho)/2$. We assume $1 \leq \ell \leq n$, except in Section 7.5, which treats $\ell = 0$. Probabilities are over the challenge space $\mathbb{F}^\ell \times L^{\times \kappa}$ of Π_{eval} (Sections 3.1 and 6.2).

7.1 Fibre distance and decoding

For $j \in \llbracket 0, \ell - 1 \rrbracket$, the domain L_j has order $M_j \geq 2$, and its fibres (Definition 2.16) are indexed by the M_{j+1} points of $L_{j+1} = (L_j)^2$.

Definition 7.1 (Fibre distance). Let $j \in \llbracket 0, \ell - 1 \rrbracket$. For $w, c \in \mathbb{F}^{L_j}$,

$$\Delta_j^{\text{fib}}(w, c) = \frac{|\{\eta \in L_{j+1} : w \text{ and } c \text{ differ somewhere on the fibre over } \eta\}|}{M_{j+1}},$$

and $\Delta_j^{\text{fib}}(w, \mathcal{C}_j) = \min_{c \in \mathcal{C}_j} \Delta_j^{\text{fib}}(w, c)$.

Lemma 7.2 (Fibre distance). Let $j \in \llbracket 0, \ell - 1 \rrbracket$ and $w, c \in \mathbb{F}^{L_j}$.

1. $\Delta(w, c) \leq \Delta_j^{\text{fib}}(w, c)$.
2. $\Delta_j^{\text{fib}}(w, c) \leq \delta$ if and only if w and c agree on every fibre over some set of at least $(1 - \delta)M_{j+1}$ points of L_{j+1} .

Proof. (1) Each point of L_j at which w and c differ lies in a fibre over which they differ, and each of these $\Delta_j^{\text{fib}}(w, c) M_{j+1}$ fibres has two points, so $\Delta(w, c) M_j \leq 2 \Delta_j^{\text{fib}}(w, c) M_{j+1} = \Delta_j^{\text{fib}}(w, c) M_j$.
(2) The set of points over whose fibre w and c agree has $(1 - \Delta_j^{\text{fib}}(w, c)) M_{j+1}$ elements. $\square$

Lemma 7.3 (Unique fibre decoding). *Let $j \in \llbracket 0, \ell - 1 \rrbracket$. For every $w \in \mathbb{F}^{L_j}$ there is at most one $c \in \mathcal{C}_j$ with $\Delta_j^{\text{fib}}(w, c) \leq \delta$.*

Proof. Such a c satisfies $\Delta(w, c) \leq \delta \leq (1 - \rho)/2$ by Lemma 7.2(1); apply Lemma 2.19(3). $\square$

Definition 7.4 (Decoded codeword and decoded table). Let $j \in \llbracket 0, \ell - 1 \rrbracket$ and $w \in \mathbb{F}^{L_j}$ with $\Delta_j^{\text{fib}}(w, \mathcal{C}_j) \leq \delta$. The *decoded codeword* $\text{dec}_j(w)$ is the unique $c \in \mathcal{C}_j$ of Lemma 7.3. For $j = 0$, the *decoded table* of w is the unique table f^* over $\mathbb{F}$ with $\text{Enc}(f^*) = \text{dec}_0(w)$, that is, the table of kernel coordinates of $P_{\text{dec}_0(w)}$ (Lemma 6.2).

Remark 7.5 (Computing the decoded table). Since $\Delta(w, \text{dec}_0(w)) \leq \delta < \frac{1}{2}(1 - (N - 1)/M)$ (Lemma 2.19), the algorithm of Fact 2.20 finds $\text{dec}_0(w)$; an inverse NTT gives $P_{\text{dec}_0(w)}$, and Corollary 4.11 gives f^* . Checking whether $\Delta_0^{\text{fib}}(w, \text{dec}_0(w)) \leq \delta$ costs $O(M)$ comparisons. Conversely, for an arbitrary w , if the decoder fails or its output fails this check, then no codeword is within fibre distance δ of w ; so decoding followed by the check decides whether $\Delta_0^{\text{fib}}(w, \mathcal{C}_0) \leq \delta$. If $w \in \mathbb{F}_q^L$ (as for a commitment computed over the base field), then f^* has entries in $\mathbb{F}_q$, by Lemma 2.21 and the integrality of Corollary 4.11.

7.2 Two folding lemmas

Lemma 7.6 (Folding far words). *Let $j \in \llbracket 0, \ell - 1 \rrbracket$ and $w \in \mathbb{F}^{L_j}$ with $\Delta_j^{\text{fib}}(w, \mathcal{C}_j) > \delta$. Then*

$$|\{r \in \mathbb{F} : \Delta(\text{fold}_r(w), \mathcal{C}_{j+1}) \leq \delta\}| \leq M_{j+1}.$$

Proof. Suppose the set has more than M_{j+1} elements. By Lemma 5.7, $\text{fold}_r(w) = u^0 + ru^1$ with $u^0 = w_o - w_e$ and $u^1 = 2w_e - w_o$. Apply Theorem 2.23 to the code $\mathcal{C}_{j+1}$ (length M_{j+1} , rate ρ): there are $L' \subseteq L_{j+1}$ with $|L'| \geq (1 - \delta)M_{j+1}$ and codewords $\hat{c}^0, \hat{c}^1 \in \mathcal{C}_{j+1}$ with $u^0 = \hat{c}^0$ and $u^1 = \hat{c}^1$ on L' . Let $T_e = P_{\hat{c}^0} + P_{\hat{c}^1}$ and $T_o = 2P_{\hat{c}^0} + P_{\hat{c}^1}$ in $\mathbb{F}[X]_{<N_{j+1}}$, and $T(X) = T_e(X^2) + XT_o(X^2) \in \mathbb{F}[X]_{<N_j}$. By Lemma 5.7, $w_e = T_e$ and $w_o = T_o$ on L' , so for every $\eta \in L'$ and $\xi \in L_j$ with $\xi^2 = \eta$, (2) gives $w(\pm\xi) = T_e(\eta) \pm \xi T_o(\eta) = T(\pm\xi)$. Thus w agrees with the codeword $\text{ev}_{L_j}(T) \in \mathcal{C}_j$ on every fibre over L' , so $\Delta_j^{\text{fib}}(w, \mathcal{C}_j) \leq \delta$, a contradiction. $\square$

Lemma 7.7 (Fibre errors survive folding). *Let $j \in \llbracket 0, \ell - 1 \rrbracket$, $w, c \in \mathbb{F}^{L_j}$, and $\eta \in L_{j+1}$ such that w and c differ somewhere on the fibre over η . Then there is at most one $r \in \mathbb{F}$ with $\text{fold}_r(w)(\eta) = \text{fold}_r(c)(\eta)$.*

Proof. Let $y = w - c$. By Lemma 5.6(2) and Lemma 5.7, $\text{fold}_r(w)(\eta) - \text{fold}_r(c)(\eta) = \text{fold}_r(y)(\eta) = (y_o(\eta) - y_e(\eta)) + r(2y_e(\eta) - y_o(\eta))$. This is a polynomial of degree ≤ 1 in r . If it were identically zero, the inverse formulas of Lemma 5.7 would give $y_e(\eta) = y_o(\eta) = 0$, so y would vanish on the fibre over η (Lemma 2.22(3)). Hence it has at most one root. $\square$

7.3 Good challenges

Definition 7.8 (Good challenge). Let $j \in \llbracket 1, \ell \rrbracket$, $w \in \mathbb{F}^{L_{j-1}}$ and $r \in \mathbb{F}$. We say that r is *good* for w at level j , and write $\text{Good}_j(w, r)$, if:

1. in case $\Delta_{j-1}^{\text{fib}}(w, \mathcal{C}_{j-1}) > \delta$: $\Delta(\text{fold}_r(w), \mathcal{C}_j) > \delta$;
2. in case $\Delta_{j-1}^{\text{fib}}(w, \mathcal{C}_{j-1}) \leq \delta$, with $c = \text{dec}_{j-1}(w)$: for every $\eta \in L_j$ such that w and c differ somewhere on the fibre over η , $\text{fold}_r(w)(\eta) \neq \text{fold}_r(c)(\eta)$.

Lemma 7.9 (Few bad challenges). *For every $j \in \llbracket 1, \ell \rrbracket$ and $w \in \mathbb{F}^{L_{j-1}}$, at most M_j values $r \in \mathbb{F}$ are not good for w at level j .*

Proof. In case 1, this is Lemma 7.6. In case 2, by Lemma 7.7, each of the at most M_j points η over which w and c differ excludes at most one r . $\square$

Lemma 7.10 (One folding step). *Let $j \in \llbracket 1, \ell \rrbracket$, $w \in \mathbb{F}^{L_{j-1}}$, and $r \in \mathbb{F}$ good for w at level j . Let $c' \in \mathcal{C}_j$ and $Y = \{\eta \in L_j : \text{fold}_r(w)(\eta) = c'(\eta)\}$, and suppose $|Y| \geq (1 - \delta)M_j$. Then $\Delta_{j-1}^{\text{fib}}(w, \mathcal{C}_{j-1}) \leq \delta$; the codeword $c = \text{dec}_{j-1}(w)$ satisfies $\text{fold}_r(c) = c'$; and $w = c$ on the fibre over every $\eta \in Y$.*

Proof. We have $\Delta(\text{fold}_r(w), c') \leq \delta$, so case 1 of Definition 7.8 is excluded: $\Delta_{j-1}^{\text{fib}}(w, \mathcal{C}_{j-1}) \leq \delta$. By Proposition 5.8 (with $2N_j = N_{j-1} \leq M_{j-1}$), $\text{fold}_r(c) \in \mathcal{C}_j$. By locality (Lemma 5.6(3)), $\text{fold}_r(c)$ agrees with $\text{fold}_r(w)$ at every η over whose fibre $w = c$, hence on at least $(1 - \delta)M_j$ points; and $\text{fold}_r(w)$ agrees with c' on Y . By Lemma 2.18, $\text{fold}_r(c)$ and c' agree on at least $(1 - 2\delta)M_j \geq \rho M_j = N_j$ points, so they are equal (Lemma 2.19(2)). Finally, for $\eta \in Y$, $\text{fold}_r(w)(\eta) = c'(\eta) = \text{fold}_r(c)(\eta)$, so by case 2 of Definition 7.8, $w = c$ on the fibre over η . $\square$

7.4 The soundness theorem

Fix a deterministic prover P^* (Lemma 3.4 extends the results to randomised provers) and an instance $(w_0; z, v)$ with $w_0 \in \mathbb{F}^L$. The prover's messages are functions of the earlier challenges: s_j of $r_{\leq j-1}$, the oracle w_j ($j \in \llbracket 1, \ell - 1 \rrbracket$) of $r_{\leq j}$, and g of $r_{\leq \ell}$. As in Section 6.2, $G = \sum_b g(b)K_b^{(n-\ell)}$ and $w_\ell = \text{ev}_{L_\ell}(G) \in \mathcal{C}_\ell$.

Passing sets. For fixed challenges $r \in \mathbb{F}^\ell$, let $\mathcal{P}_\ell = L_\ell$ and, for $j = \ell - 1, \dots, 0$,

$$\mathcal{P}_j = \{ \xi \in L_j : \xi^2 \in \mathcal{P}_{j+1} \text{ and } \text{fold}_{r_{j+1}}(w_j)(\xi^2) = w_{j+1}(\xi^2) \}.$$

Let $\pi = \pi(r) = |\mathcal{P}_0|/M$. We record these facts.

Lemma 7.11 (Passing sets). *For fixed $r \in \mathbb{F}^\ell$:*

1. for $j \in \llbracket 0, \ell \rrbracket$, a point $\xi_0 \in L$ has $\xi_0^{2^j} \in \mathcal{P}_j$ if and only if the query checks at levels $j + 1, \dots, \ell$ pass for ξ_0 ; in particular ξ_0 passes all the query checks if and only if $\xi_0 \in \mathcal{P}_0$;
2. for $j < \ell$, $\mathcal{P}_j$ is a union of fibres, and the squaring map sends it two-to-one onto a set $\mathcal{P}_j^2 \subseteq \mathcal{P}_{j+1}$;
3. $|\mathcal{P}_j| \geq \pi M_j$ for every $j \in \llbracket 0, \ell \rrbracket$.

Proof. (1) Downward induction on j , from the recursive definition. (2) The defining condition of $\mathcal{P}_j$ depends only on ξ^2 , and squaring is two-to-one on L_j (Lemma 2.15). (3) By (2), $|\mathcal{P}_j|/M_j = |\mathcal{P}_j^2|/M_{j+1} \leq |\mathcal{P}_{j+1}|/M_{j+1}$; iterate from $j = 0$. $\square$

Lemma 7.12 (The codeword chain). *Fix $r \in \mathbb{F}^\ell$ such that r_j is good for w_{j-1} at level j for every $j \in \llbracket 1, \ell \rrbracket$, and $\pi > 1 - \delta$. Then $\Delta_j^{\text{fib}}(w_j, \mathcal{C}_j) \leq \delta$ for every $j \in \llbracket 0, \ell - 1 \rrbracket$, and the codewords $c_j = \text{dec}_j(w_j)$ for $j < \ell$ and $c_\ell = w_\ell$ satisfy $\text{fold}_{r_{j+1}}(c_j) = c_{j+1}$ and $w_j = c_j$ on $\mathcal{P}_j$, for every $j \in \llbracket 0, \ell - 1 \rrbracket$. Moreover, $g = \text{res}_r(f^*)$, where f^* is the decoded table of w_0 .*

Proof. We show by downward induction on $j \in \llbracket 0, \ell \rrbracket$ that $w_j = c_j$ on $\mathcal{P}_j$, together with the other claims for $j < \ell$. For $j = \ell$ this is trivial. Let $j < \ell$. For $\eta \in \mathcal{P}_j^2 \subseteq \mathcal{P}_{j+1}$, $\text{fold}_{r_{j+1}}(w_j)(\eta) = w_{j+1}(\eta) = c_{j+1}(\eta)$ by the definition of $\mathcal{P}_j$ and the induction hypothesis. So the set Y of Lemma 7.10 (with $w = w_j$, $r = r_{j+1}$, $c' = c_{j+1}$, at level $j+1$) contains $\mathcal{P}_j^2$, of size $|\mathcal{P}_j|/2 \geq \pi M_{j+1} > (1 - \delta)M_{j+1}$ (Lemma 7.11). Lemma 7.10 gives $\Delta_j^{\text{fib}}(w_j, \mathcal{C}_j) \leq \delta$, $\text{fold}_{r_{j+1}}(c_j) = c_{j+1}$, and $w_j = c_j$ on every fibre over $\mathcal{P}_j^2$, that is, on $\mathcal{P}_j$.

For the last claim, extend r arbitrarily to a point of $\mathbb{F}^n$. Then $c_0 = \text{Enc}(f^*) = \text{ev}_L(U_{f^*})$, and by Corollary 5.9 (with $\mu = n + R \geq n$), c_ℓ , obtained from c_0 by ℓ folds, is $\text{ev}_{L_\ell}(U_\ell)$, where U_ℓ has kernel coordinates $\text{res}_r(f^*)$ (Corollary 5.4). Since $c_\ell = w_\ell = \text{ev}_{L_\ell}(G)$ and both G and U_ℓ lie in $\mathbb{F}[X]_{<N_\ell}$ with $N_\ell \leq M_\ell$, Lemma 2.19(1) gives $G = U_\ell$, and Theorem 4.8 gives $g = \text{res}_r(f^*)$. $\square$

Theorem 7.13 (Soundness). *Let $1 \leq \ell \leq n$, and put*

$$\varepsilon_{\text{fold}} = \sum_{j=1}^{\ell} \frac{M_j}{|\mathbb{F}|} = \frac{M(1 - 2^{-\ell})}{|\mathbb{F}|} < \frac{M}{|\mathbb{F}|}.$$

1. *If $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) > \delta$, then V accepts with probability at most $\varepsilon_{\text{fold}} + (1 - \delta)^\kappa$.*
2. *If $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$ and the decoded table f^* of w_0 satisfies $\tilde{f}^*(z) \neq v$, then V accepts with probability at most*

$$\varepsilon_{\text{fold}} + \frac{2\ell}{|\mathbb{F}|} + (1 - \delta)^\kappa.$$

Proof. Honest values. In case 2, let $f_j^* = \text{res}_{r_{\leq j}}(f^*)$ and $\chi_j^* = \text{res}_{r_{\leq j}}(\chi_0)$ be the tables of the honest prover for f^* (Section 6.2), let s_j^* be its round polynomials, which are those of Section 3.3 for $F^*(x) = \tilde{f}^*(x) \text{eq}(x, z)$ (Lemma 6.6), and let $\sigma_j^* = \sum_b f_j^*(b) \chi_j^*(b)$ for $j \in \llbracket 0, \ell \rrbracket$. By Lemma 6.6, $\sigma_0^* = \tilde{f}^*(z)$, $s_j^*(0) + s_j^*(1) = \sigma_{j-1}^*$, $s_j^*(r_j) = \sigma_j^*$, and $\sigma_\ell^* = (\prod_j \text{eq}_1(r_j, z_j)) \cdot \tilde{f}^*(z_{\ell+1}, \dots, z_n)$. Note that s_j^* depends on $r_{\leq j-1}$ only.

Bad events. For $j \in \llbracket 1, \ell \rrbracket$, let E_j be the set of $r_{\leq j}$ such that r_j is not good for w_{j-1} at level j , or (in case 2) $s_j \neq s_j^*$ and $s_j(r_j) = s_j^*(r_j)$. For fixed $r_{\leq j-1}$, the word w_{j-1} and the polynomials s_j, s_j^* are fixed, so at most M_j values of r_j (Lemma 7.9) plus, in case 2, at most 2 values (Lemma 2.1) complete $r_{\leq j-1}$ into E_j . By Lemma 3.2, $\Pr[\exists j : r_{\leq j} \in E_j] \leq \varepsilon_{\text{fold}}$ in case 1 and $\leq \varepsilon_{\text{fold}} + 2\ell/|\mathbb{F}|$ in case 2.

No bad event. Suppose that no E_j occurs and that V accepts. We show that $\pi \leq 1 - \delta$. Assume instead $\pi > 1 - \delta$. Then Lemma 7.12 applies. In case 1 it gives $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$, a contradiction. In case 2 it gives $g = f_\ell^*$, so the closure check reads $v_\ell = \sigma_\ell^*$. Since $v_0 = v \neq \tilde{f}^*(z) = \sigma_0^*$, there is a largest $j \in \llbracket 0, \ell - 1 \rrbracket$ with $v_j \neq \sigma_j^*$; then $v_{j+1} = \sigma_{j+1}^*$. The sumcheck check for index $j+1$ gives $s_{j+1}(0) + s_{j+1}(1) = v_j \neq \sigma_j^* = s_{j+1}^*(0) + s_{j+1}^*(1)$, so $s_{j+1} \neq s_{j+1}^*$, while $s_{j+1}(r_{j+1}) = v_{j+1} = \sigma_{j+1}^* = s_{j+1}^*(r_{j+1})$; that is, $r_{\leq j+1} \in E_{j+1}$, a contradiction.

Conclusion. Acceptance implies that some E_j occurs, or that $\pi(r) \leq 1 - \delta$ and all κ query points lie in $\mathcal{P}_0$ (Lemma 7.11(1)). By Lemma 3.3 with $\mathcal{A}' = \mathbb{F}^\ell$, $\mathcal{A} = \{r : \pi(r) \leq 1 - \delta\}$ and $\mathcal{S}(r) = \mathcal{P}_0$, the second event has probability at most $(1 - \delta)^\kappa$. $\square$

Corollary 7.14 (Evaluation binding). *Let $1 \leq \ell \leq n$ and $\varepsilon = \varepsilon_{\text{fold}} + 2\ell/|\mathbb{F}| + (1 - \delta)^\kappa$. For every $w_0 \in \mathbb{F}^L$ and $z \in \mathbb{F}^n$, if some prover makes V accept the instance $(w_0; z, v)$ with probability greater than ε , then $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$ and $v = \tilde{f}^*(z)$ for the decoded table f^* of w_0 . In particular, Π_{eval} is ε -evaluation binding (Definition 3.11).*

Proof. Immediate from Theorem 7.13 and Lemma 3.4; f^* depends on w_0 only. $\square$

7.5 The case $\ell = 0$

For $\ell = 0$ there is no fold, and the fibre distance is replaced by the Hamming distance.

Proposition 7.15 (No folding). *Let $\ell = 0$. If a deterministic prover makes V accept $(w_0; z, v)$ with probability greater than $(1 - \delta)^\kappa$, then $\Delta(w_0, \mathcal{C}_0) < \delta$, and $v = \tilde{f}^H(z)$, where f^H is the table such that $\text{Enc}(f^H)$ is the unique codeword within Hamming distance δ of w_0 (Lemma 2.19(3)).*

Proof. For $\ell = 0$ the prover sends g before any challenge, so g and G are fixed, and the verifier accepts if and only if $v = \tilde{g}(z)$ and all query points lie in $S = \{\xi \in L : w_0(\xi) = G(\xi)\}$. This has probability 0 or $(|S|/M)^\kappa$. If it exceeds $(1 - \delta)^\kappa$, then $v = \tilde{g}(z)$ and $|S| > (1 - \delta)M$, so $\Delta(w_0, \text{ev}_L(G)) < \delta$; hence $\text{ev}_L(G) = \text{Enc}(g)$ is the unique codeword within distance δ of w_0 , and $g = f^H$. $\square$

7.6 Round-by-round knowledge soundness

Theorem 7.13 bounds the soundness of the interactive protocol. We now prove the stronger property of round-by-round knowledge soundness (Definition 3.7); Section 7.7 derives from it the relaxed notion of Definition 3.14, which is the hypothesis of Theorem 3.16.

Relation. Let $1 \leq \ell \leq n$. We view Π_{eval} as an IOP for the relation $\mathcal{R}_{\text{eval}}^\delta$ of Section 3.4, with $\text{dist}(w, \text{Enc}(f)) = \Delta_0^{\text{fib}}(w, \text{Enc}(f))$; condition (3) holds by Lemma 7.3 and the injectivity of Enc . A table f is a witness for $(w_0; z, v)$ if and only if $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$, f is the decoded table of w_0 , and $\tilde{f}(z) = v$.

Notation. The rounds are those of Section 6.2, and τ_j is the partial transcript after j rounds. For $j \in \llbracket 1, \ell \rrbracket$ let $\hat{w}_j = \text{fold}_{r_j}(w_{j-1}) \in \mathbb{F}^{L_j}$, which is determined by τ_j , and for $j \in \llbracket 1, \ell - 1 \rrbracket$ let $\mathcal{B}_j = \{\eta \in L_j : \hat{w}_j(\eta) \neq w_j(\eta)\}$, which is determined by τ_{j+1} . The *sumcheck check of index j* is the equation $s_j(0) + s_j(1) = v_{j-1}$, which is determined by τ_j . For $c \in \mathcal{C}_j$, let $\lambda[c]$ be the table of kernel coordinates, with $n - j$ variables, of P_c (Lemma 2.19 and Definition 4.10).

Witness sets and claimed values. For $c \in \mathcal{C}_0$ let $\mathcal{W}_0(c) = \{\xi \in L : w_0(\xi) = c(\xi) \text{ and } w_0(-\xi) = c(-\xi)\}$, and for $j \in \llbracket 1, \ell \rrbracket$ and $c \in \mathcal{C}_j$ let

$$\mathcal{W}_j(c) = \{ \xi \in L : \xi^{2^i} \notin \mathcal{B}_i \text{ for all } i \in \llbracket 1, j - 1 \rrbracket, \text{ and } \hat{w}_j(\xi^{2^j}) = c(\xi^{2^j}) \},$$

which is determined by τ_j . Let $\text{dens}_j(c) = |\mathcal{W}_j(c)|/M$, and for $j \in \llbracket 0, \ell \rrbracket$ and $c \in \mathcal{C}_j$ let

$$\sigma_j(c) = \left(\prod_{i=1}^j \text{eq}_1(r_i, z_i) \right) \cdot \widetilde{\lambda[c]}(z_{j+1}, \dots, z_n),$$

the value that the running claim v_j takes if the prover is honest with respect to c (compare Lemma 6.6(4)).

Definition 7.16 (Doomed set and extractor). • The empty transcript τ_0 is doomed.

- For $j \in \llbracket 1, \ell \rrbracket$, the partial transcript τ_j is *not doomed* if the sumcheck checks of indices $1, \dots, j$ hold and there is $c \in \mathcal{C}_j$ with

$$\text{dens}_j(c) \geq 1 - \delta \quad \text{and} \quad v_j = \sigma_j(c);$$

otherwise it is *doomed*.

- A full transcript $\tau_{\ell+1}$ is doomed if and only if the verifier rejects it.

Let $\mathcal{D}$ be the set of doomed transcripts. The *extractor* Ext , on every input $(\tau, \mathbf{m})$, reads the oracle part w_0 of the instance of τ , and outputs the decoded table of w_0 if $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$, and the zero table otherwise.

By Remark 7.5, Ext runs in time polynomial in M . Its output is a witness for $(w_0; z, v)$ if and only if $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$ and the decoded table f^* satisfies $\tilde{f}^*(z) = v$: if $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) > \delta$, no table is a witness.

Lemma 7.17 (One round of the chain). *Let $j \in \llbracket 1, \ell \rrbracket$, fix τ_{j-1} and the round- j message (hence w_{j-1}), and let r_j be good for w_{j-1} at level j . If $c' \in \mathcal{C}_j$ satisfies $\text{dens}_j(c') \geq 1 - \delta$, then $\Delta_{j-1}^{\text{fib}}(w_{j-1}, \mathcal{C}_{j-1}) \leq \delta$, the codeword $c = \text{dec}_{j-1}(w_{j-1})$ satisfies $\text{fold}_{r_j}(c) = c'$, and $\mathcal{W}_j(c') \subseteq \mathcal{W}_{j-1}(c)$; in particular $\text{dens}_{j-1}(c) \geq 1 - \delta$.*

Proof. Let $Y = \{\eta \in L_j : \hat{w}_j(\eta) = c'(\eta)\}$. Every $\xi \in \mathcal{W}_j(c')$ has $\xi^{2^j} \in Y$, and the map $\xi \mapsto \xi^{2^j}$ is 2^j -to-1 from L onto L_j (Lemma 2.15); so $|Y| \geq |\mathcal{W}_j(c')|/2^j \geq (1 - \delta)M_j$. Lemma 7.10 gives the first two claims, and $w_{j-1} = c$ on the fibre over every $\eta \in Y$. Let $\xi \in \mathcal{W}_j(c')$, and $\eta = \xi^{2^j} \in Y$; the fibre over η is $\{\pm \xi^{2^{j-1}}\}$, on which $w_{j-1} = c$. If $j = 1$, this says $\xi \in \mathcal{W}_0(c)$. If $j \geq 2$, the condition $\xi^{2^{j-1}} \notin \mathcal{B}_{j-1}$ (part of the definition of $\mathcal{W}_j(c')$) gives $\hat{w}_{j-1}(\xi^{2^{j-1}}) = w_{j-1}(\xi^{2^{j-1}}) = c(\xi^{2^{j-1}})$, and the conditions $\xi^{2^i} \notin \mathcal{B}_i$ for $i \leq j - 2$ are common to both sets; hence $\xi \in \mathcal{W}_{j-1}(c)$. $\square$

Lemma 7.18 (Honest round polynomial for a codeword). *Let $j \in \llbracket 1, \ell \rrbracket$, $c \in \mathcal{C}_{j-1}$ and $r_1, \dots, r_{j-1} \in \mathbb{F}$, and define*

$$s_j^c(X) = \left(\prod_{i=1}^{j-1} \text{eq}_1(r_i, z_i) \right) \sum_{b=0}^{N_j-1} \widetilde{\lambda[c]}(X, b_1, \dots, b_{n-j}) \text{eq}((X, b_1, \dots, b_{n-j}), (z_j, \dots, z_n)).$$

Then $s_j^c \in \mathbb{F}[X]_{<3}$, $s_j^c(0) + s_j^c(1) = \sigma_{j-1}(c)$, and $s_j^c(r) = \sigma_j(\text{fold}_r(c))$ for every $r \in \mathbb{F}$, where σ_j is computed with $r_j = r$.

Proof. Each summand is a product of two polynomials of degree ≤ 1 in X . Summing over $X \in \{0, 1\}$ and b amounts to summing over $a = X + 2b \in \llbracket 0, N_{j-1} - 1 \rrbracket$, whose bit vector is $(X, b_1, \dots, b_{n-j})$; this gives $\sum_a \lambda[c](a) \text{eq}(a, (z_j, \dots, z_n)) = \widetilde{\lambda[c]}(z_j, \dots, z_n)$ by Proposition 2.6, times the prefix, that is, $\sigma_{j-1}(c)$. By Proposition 5.8 and Theorem 5.3, $\lambda[\text{fold}_r(c)] = \text{res}_r(\lambda[c])$, whose multilinear extension is $\widetilde{\lambda[c]}(r, \cdot)$ (Lemma 2.11). Since $\text{eq}((r, b), (z_j, \dots, z_n)) = \text{eq}_1(r, z_j) \text{eq}(b, (z_{j+1}, \dots, z_n))$ (Lemma 2.5(3)), evaluating at $X = r$ (Lemma 2.12) gives

$$s_j^c(r) = \left(\prod_{i=1}^{j-1} \text{eq}_1(r_i, z_i) \right) \text{eq}_1(r, z_j) \widetilde{\lambda[\text{fold}_r(c)]}(z_{j+1}, \dots, z_n) = \sigma_j(\text{fold}_r(c)). \quad \square$$

Theorem 7.19 (Round-by-round knowledge soundness). *Let $1 \leq \ell \leq n$, and let $\mathcal{D}$ and Ext be as in Definition 7.16.*

1. (Round 1.) If the output of Ext is not a witness for $(w_0; z, v)$, then for every round-1 message s_1 , τ_1 is not doomed with probability at most $(M_1 + 2)/|\mathbb{F}|$ over r_1 .
2. (Round j , $2 \leq j \leq \ell$.) If τ_{j-1} is doomed, then for every round- j message (w_{j-1}, s_j) , τ_j is not doomed with probability at most $(M_j + 2)/|\mathbb{F}|$ over r_j .
3. (Round $\ell + 1$.) If τ_ℓ is doomed, then for every final message g , the verifier accepts with probability at most $(1 - \delta)^\kappa$ over the query points.

Consequently, Π_{eval} is round-by-round knowledge sound for $\mathcal{R}_{\text{eval}}^\delta$ (Definition 3.7) with errors $\varepsilon_j = (M_j + 2)/|\mathbb{F}|$ for $j \in [1, \ell]$ and $\varepsilon_{\ell+1} = (1 - \delta)^\kappa$, and in particular with

$$\varepsilon_{\text{rbr}} = \max\left\{\frac{M_1 + 2}{|\mathbb{F}|}, (1 - \delta)^\kappa\right\}.$$

Proof. Items 1 and 2. Fix $j \in [1, \ell]$, the instance, τ_{j-1} and the round- j message. Then w_{j-1} , s_j and, if $\Delta_{j-1}^{\text{fb}}(w_{j-1}, \mathcal{C}_{j-1}) \leq \delta$, the codeword $c = \text{dec}_{j-1}(w_{j-1})$ and the polynomial s_j^c of Lemma 7.18 are fixed. Call r_j *bad* if it is not good for w_{j-1} at level j , or if $\Delta_{j-1}^{\text{fb}}(w_{j-1}, \mathcal{C}_{j-1}) \leq \delta$, $s_j \neq s_j^c$ and $s_j(r_j) = s_j^c(r_j)$. By Lemmas 2.1 and 7.9, at most $M_j + 2$ values of r_j are bad. We show that if r_j is not bad and τ_j is not doomed, then the hypothesis of the item fails.

Let $c' \in \mathcal{C}_j$ witness that τ_j is not doomed: $\text{dens}_j(c') \geq 1 - \delta$, $v_j = \sigma_j(c')$, and the sumcheck checks of indices $1, \dots, j$ hold. By Lemma 7.17, c exists, $\text{fold}_{r_j}(c) = c'$ and $\text{dens}_{j-1}(c) \geq 1 - \delta$. By Lemma 7.18, $s_j(r_j) = v_j = \sigma_j(\text{fold}_{r_j}(c)) = s_j^c(r_j)$, so $s_j = s_j^c$ because r_j is not bad. The sumcheck check of index j then gives $v_{j-1} = s_j(0) + s_j(1) = s_j^c(0) + s_j^c(1) = \sigma_{j-1}(c)$.

For $j \geq 2$, the codeword c therefore witnesses that τ_{j-1} is not doomed, contradicting the hypothesis of item 2. For $j = 1$, $\Delta_0^{\text{fb}}(w_0, \mathcal{C}_0) \leq \delta$ and $c = \text{dec}_0(w_0) = \text{Enc}(f^*)$ for the decoded table f^* , so $\lambda[c] = f^*$ and $v = v_0 = \sigma_0(c) = \hat{f}^*(z)$; thus the output f^* of Ext is a witness, contradicting the hypothesis of item 1.

Item 3. If a sumcheck check or the closure check fails, the verifier rejects. Otherwise, since $\lambda[w_\ell] = g$, the closure check reads $v_\ell = \sigma_\ell(w_\ell)$; as τ_ℓ is doomed, $\text{dens}_\ell(w_\ell) < 1 - \delta$. A query point ξ passes all its checks exactly when $\hat{w}_i(\xi^{2^i}) = w_i(\xi^{2^i})$ for all $i \in [1, \ell]$, that is, when $\xi \in \mathcal{W}_\ell(w_\ell)$. The κ query points are independent and uniform, so the verifier accepts with probability at most $\text{dens}_\ell(w_\ell)^\kappa < (1 - \delta)^\kappa$ (Lemma 3.3).

Round-by-round knowledge soundness. We check the three conditions of Definition 3.7. Condition 1 holds because τ_0 is doomed, and condition 3 because a full transcript is doomed exactly when it is rejected. For condition 2 in round 1, item 1 says that an escape probability above ε_1 forces Ext to output a witness. In rounds $2, \dots, \ell + 1$, items 2 and 3 bound the escape probability from a doomed transcript by ε_j outright. Finally, $M_j \leq M_1$ for all $j \geq 1$. $\square$

Remark 7.20 (Relation to the definition of [BGTZ24]). Definition 3.7 is Definition 3.12 of Block, Garreta, Tiwari and Zajac [BGTZ24], with the query positions placed in the challenge of the last round. Because the relation $\mathcal{R}_{\text{eval}}^\delta$ requires the witness to be the table decoded from the commitment, the extracted witness is bound to the commitment; by Lemma 3.12, Theorem 7.19 implies ε -evaluation binding with $\varepsilon = \sum_{j=1}^\ell (M_j + 2)/|\mathbb{F}| + (1 - \delta)^\kappa$, which is also the bound of Corollary 7.14.

Proposition 7.21 (Round-by-round knowledge soundness for $\ell = 0$). *Let $\ell = 0$, and define $\mathcal{R}_{\text{eval}}^\delta$ with $\text{dist} = \Delta$; condition (3) holds by Lemma 2.19(3). Let Ext output the table f^H of Proposition 7.15 if $\Delta(w_0, \mathcal{C}_0) \leq \delta$ (computed with Fact 2.20), and the zero table otherwise, and let $\mathcal{D}$ consist of τ_0 and the rejected full transcripts. Then Π_{eval} is round-by-round knowledge sound for $\mathcal{R}_{\text{eval}}^\delta$ with error $\varepsilon_1 = (1 - \delta)^\kappa$.*

Proof. Conditions 1 and 3 of Definition 3.7 hold by construction. For condition 2, fix the message g : if the acceptance probability exceeds $(1 - \delta)^\kappa$, the proof of Proposition 7.15 gives $\Delta(w_0, \mathcal{C}_0) < \delta$ and $v = \tilde{f}^H(z)$, so the output of Ext is a witness. $\square$

7.7 The non-interactive scheme

We now show that Π_{eval} , written as an interactive oracle reduction, satisfies Definition 3.14, and apply Theorem 3.16. Let $1 \leq \ell \leq n$ and $\delta^* = (1 - \rho)/2$. Everything proved so far in this section holds for every $\delta \in (0, \delta^*]$; below, δ varies in $[0, 1]$.

Fibre strings. For each $j \in \llbracket 0, \ell - 1 \rrbracket$ and $\eta \in L_{j+1}$, fix one of the two square roots $\xi_\eta \in L_j$ of η (Lemma 2.15); the other is $-\xi_\eta$. Let $\Sigma = \mathbb{F}^2$, and for $w \in \mathbb{F}^{L_j}$ let $\text{fib}_j(w) \in \Sigma^{L_{j+1}}$ be the string $\eta \mapsto (w(\xi_\eta), w(-\xi_\eta))$, indexed by L_{j+1} in a fixed order. The map fib_j is a bijection from $\mathbb{F}^{L_j}$ onto $\Sigma^{M_{j+1}}$, and by Definition 7.1,

$$\Delta(\text{fib}_j(w), \text{fib}_j(c)) = \Delta_j^{\text{fib}}(w, c) \quad (w, c \in \mathbb{F}^{L_j}). \quad (7)$$

For a partial string u over Σ , the *filling* $\bar{u}$ replaces every $\perp$ by $(0, 0)$, and $\mathcal{U}(u)$ is the set of positions where u is $\perp$. Since $\bar{u}$ and u agree on $\text{Dom}(u)$, $\Delta(\bar{u}, u') \leq \Delta(u, u')$ for every full string u' .

The relation. Let

$$\mathcal{R}_{\text{eval}} = \{ ((z, v), y, f) : y = \text{fib}_0(\text{Enc}(f)) \text{ and } \tilde{f}(z) = v \},$$

with explicit instance $(z, v) \in \mathbb{F}^n \times \mathbb{F}$, implicit instance $y \in \Sigma^{M/2}$ and witness a table f over $\mathbb{F}$. For a partial string y and $\delta \in [0, 1]$, $((z, v), y, f) \in (\mathcal{R}_{\text{eval}})_{\leq \delta}$ if and only if $\Delta(y, \text{fib}_0(\text{Enc}(f))) \leq \delta$ and $\tilde{f}(z) = v$. For a full string $y = \text{fib}_0(w_0)$, by (7), this says $((w_0; z, v), f) \in \mathcal{R}_{\text{eval}}^\delta$ in the notation of Section 7.6.

The protocol as an IOR. Let Π_{eval}^Σ be Π_{eval} with instance $((z, v); y)$, and with messages and challenges encoded as follows.

- For $j \in \llbracket 1, \ell \rrbracket$, the round- j message is the string over Σ formed by $(s_j(0), s_j(1))$ and $(s_j(2), 0)$, followed, if $j \geq 2$, by the oracle part $\text{fib}_{j-1}(w_{j-1})$. The final message is g , written as $\lceil N_\ell/2 \rceil$ symbols, padded with a zero if $N_\ell = 1$. Padding entries are ignored.
- Fix an integer $\beta \geq \log_2 |\mathbb{F}|$ and a bijection $\text{num} : \llbracket 0, |\mathbb{F}| - 1 \rrbracket \rightarrow \mathbb{F}$. The challenge of round $j \in \llbracket 1, \ell \rrbracket$ is $\text{vr}_j \in \{0, 1\}^\beta$, and $r_j = \phi(\text{vr}_j)$, where $\phi(\text{vr}) = \text{num}(\text{int}(\text{vr}) \bmod |\mathbb{F}|)$ and int reads a bit string as a binary integer.
- Fix a bijection $\text{pos} : \{0, 1\}^{n+R} \rightarrow L$ (recall $M = 2^{n+R}$). The challenge of round $\ell + 1$ is $\text{vr}_{\ell+1} \in \{0, 1\}^{\kappa(n+R)}$, cut into κ blocks of $n + R$ bits, whose images under pos are the query points.
- On partial strings, the verifier rejects if an entry that it reads is $\perp$. Otherwise it runs the verifier of Π_{eval} on the words $w_0 = \text{fib}_0^{-1}(\bar{y})$ and $w_{j-1} = \text{fib}_{j-1}^{-1}(\overline{\text{oracle part of round } j})$, the round polynomials and the final table read from the filled messages; it outputs $(\top, ())$ if that verifier accepts, and rejects otherwise.

Every string of the prescribed length over Σ encodes a message of Π_{eval} , and filled strings are such strings. The query point $\xi_0 = \xi_0^{(i)}$ reads, for each level $i' \in \llbracket 1, \ell \rrbracket$, the entry at position $\xi_0^{2^{i'}}$ of the string y (if $i' = 1$) or of the oracle part of round i' (if $i' \geq 2$): this entry contains the two values $w_{i'-1}(\pm \xi_0^{2^{i'-1}})$ of the fold check of level i' , and, for $i' \geq 2$, one of them is the value $w_{i'-1}(\xi_0^{2^{i'-1}})$ compared at level $i' - 1$. The positions read are determined by the challenges. So Π_{eval}^Σ is an IOR from $\mathcal{R}_{\text{eval}}$ to $\mathcal{R}_{\text{acc}}$ with $\nu = \ell + 1$ rounds, $\beta_{\min} = \min(\beta, \kappa(n + R))$ and $l_{\max} = M/2$.

Lemma 7.22 (Sampling). *For every $S \subseteq \mathbb{F}$, $\Pr_{\mathbf{vr} \leftarrow \{0,1\}^\beta}[\phi(\mathbf{vr}) \in S] \leq |S| (1/|\mathbb{F}| + 2^{-\beta})$. The query points of round $\ell + 1$ are independent and uniform in L .*

Proof. Each $a \in \mathbb{F}$ has at most $\lceil 2^\beta/|\mathbb{F}| \rceil < 2^\beta/|\mathbb{F}| + 1$ preimages under ϕ ; divide by 2^β and sum over S . The map $\{0,1\}^{\kappa(n+R)} \rightarrow L^{\times \kappa}$ induced by pos is a bijection. $\square$

Erasures. For a statement and messages given as partial strings, let $\mathcal{U}_0 = \mathcal{U}(y) \subseteq L_1$ and, for $i \in \llbracket 1, \ell - 1 \rrbracket$, let $\mathcal{U}_i \subseteq L_{i+1}$ be the set of erased positions of the oracle part of round $i + 1$. We apply the notation of Section 7.6 to the filled words $w_0, \dots, w_{\ell-1}$, round polynomials and final table defined above, and, for $j \in \llbracket 0, \ell \rrbracket$ and $c \in \mathcal{C}_j$, we put

$$\mathcal{W}_j^\perp(c) = \{ \xi \in \mathcal{W}_j(c) : \xi^{2^i} \notin \mathcal{U}_{i-1} \text{ for all } i \in \llbracket 1, \max(j, 1) \rrbracket \}, \quad \text{dens}_j^\perp(c) = |\mathcal{W}_j^\perp(c)|/M.$$

For $\delta \in (0, \delta^*]$ and the statement at hand, let $\mathcal{D}_\delta^\perp$ be the set of partial transcripts obtained from Definition 7.16 with $\text{dens}_j^\perp$ in place of dens_j ; it depends on the erasures of y and of the messages, not only on the filled words. Without erasures, $\mathcal{W}_j^\perp = \mathcal{W}_j$ and $\mathcal{D}_\delta^\perp$ is the doomed set of Definition 7.16.

Definition 7.23 (Knowledge state function of Π_{eval}^Σ). The algorithm **E**, on every input, reads y , lets $w_0 = \text{fib}_0^{-1}(\bar{y})$, and outputs the decoded table of w_0 at radius δ^* (Definition 7.4 with $\delta = \delta^*$) if $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta^*$, and the zero table otherwise. For $\delta \in [0, 1]$, a statement $((z, v), y)$, a transcript tr and a table f , let $\text{KState}_\delta((z, v), y, \text{tr}, f)$ be:

1. 1 if and only if $((z, v), y, f) \in (\mathcal{R}_{\text{eval}})_{\leq \delta}$, if $\text{tr} = \text{tr}_\emptyset$;
2. 1 if and only if the verifier of Π_{eval}^Σ accepts, if tr is a full transcript;
3. for $\text{tr} = \tau_j$ the partial transcript after $j \in \llbracket 1, \ell \rrbracket$ rounds: 1 if and only if $\tau_j \notin \mathcal{D}_\delta^\perp$, when $\delta \in (0, \delta^*]$; and 1 when $\delta \notin (0, \delta^*]$;
4. $\text{KState}_\delta((z, v), y, \text{tr}', f)$, if $\text{tr} = \text{tr}' \parallel \Pi$ ends with a prover message.

E runs in time polynomial in M by Remark 7.5, and it does not depend on δ . Conditions 1 and 3 of Definition 3.14 hold by items 1 and 2, and condition 2 by item 4.

Lemma 7.24 (Round-by-round steps with erasures). *Let $\delta \in (0, \delta^*]$, fix a statement given by a partial string y , and let $j \in \llbracket 1, \ell + 1 \rrbracket$. Fix the partial transcript τ_{j-1} and the round- j message (partial strings).*

1. If $j = 1$ and $((z, v), y, \text{E}(y)) \notin (\mathcal{R}_{\text{eval}})_{\leq \delta}$, then at most $M_1 + 2$ values of r_1 make $\tau_1 \notin \mathcal{D}_\delta^\perp$.
2. If $2 \leq j \leq \ell$ and $\tau_{j-1} \in \mathcal{D}_\delta^\perp$, then at most $M_j + 2$ values of r_j make $\tau_j \notin \mathcal{D}_\delta^\perp$.
3. If $j = \ell + 1$ and $\tau_\ell \in \mathcal{D}_\delta^\perp$, then the verifier of Π_{eval}^Σ accepts with probability at most $(1 - \delta)^\kappa$ over the query points.

Proof. We follow the proof of Theorem 7.19, applied to the filled words, which are words of Π_{eval} . First, Lemma 7.17 holds with $\mathcal{W}^\perp$: if $\xi \in \mathcal{W}_j^\perp(c')$, then $\xi \in \mathcal{W}_j(c')$, so $\xi \in \mathcal{W}_{j-1}(c)$ by Lemma 7.17; the erasure conditions defining $\mathcal{W}_{j-1}^\perp(c)$, for $i \leq \max(j-1, 1) \leq j$, are among those defining $\mathcal{W}_j^\perp(c')$. So $\mathcal{W}_j^\perp(c') \subseteq \mathcal{W}_{j-1}^\perp(c)$, and the size bound on Y in that proof still holds because $\mathcal{W}_j^\perp(c') \subseteq \mathcal{W}_j(c')$. Lemma 7.18 does not involve witness sets.

(1) and (2). Call r_j *bad* as in the proof of Theorem 7.19; at most $M_j + 2$ values are bad. As there, if r_j is not bad and $\tau_j \notin \mathcal{D}_\delta^\perp$, then $c = \text{dec}_{j-1}(w_{j-1})$ exists, $\text{dens}_{j-1}^\perp(c) \geq 1 - \delta$ and $v_{j-1} = \sigma_{j-1}(c)$, and the sumcheck checks of indices $1, \dots, j-1$ hold. For $j \geq 2$ this says $\tau_{j-1} \notin \mathcal{D}_\delta^\perp$, contradicting the hypothesis of (2). For $j = 1$: $\mathcal{W}_0^\perp(c)$ is the set of $\xi \in L$ with $\xi^2 \notin \mathcal{U}_0$ and $w_0 = c$ on $\{\pm\xi\}$, so its size is twice the number of fibres $\eta \in L_1$ with $y(\eta) = \text{fib}_0(c)(\eta)$ (in particular $y(\eta) \neq \perp$). So $\text{dens}_0^\perp(c) \geq 1 - \delta$ gives $\Delta(y, \text{fib}_0(c)) \leq \delta$. Hence $\Delta_0^{\text{fib}}(w_0, c) = \Delta(\bar{y}, \text{fib}_0(c)) \leq \delta \leq \delta^*$, so $E(y) = \lambda[c]$ by Lemma 7.3 at radius δ^* and Lemma 6.2; and $v = v_0 = \sigma_0(c) = \lambda[c](z)$. Thus $((z, v), y, E(y)) \in (\mathcal{R}_{\text{eval}})_{\leq \delta}$, contradicting the hypothesis of (1).

(3). If the verifier accepts, no entry it reads is $\perp$, its round polynomials and final table are those of the filled messages, and the checks of Π_{eval} pass on the filled words. As in item 3 of Theorem 7.19, the closure check gives $v_\ell = \sigma_\ell(w_\ell)$, so $\text{dens}_\ell^\perp(w_\ell) < 1 - \delta$ because $\tau_\ell \in \mathcal{D}_\delta^\perp$. A query point ξ that passes its checks lies in $\mathcal{W}_\ell(w_\ell)$, and it reads the positions ξ^{2^i} of the strings with erasure sets $\mathcal{U}_{i-1}$, $i \in \llbracket 1, \ell \rrbracket$, which are therefore not erased; so $\xi \in \mathcal{W}_\ell^\perp(w_\ell)$. By Lemma 7.22 and Lemma 3.3, the verifier accepts with probability at most $\text{dens}_\ell^\perp(w_\ell)^\kappa < (1 - \delta)^\kappa$. $\square$

Theorem 7.25 (Relaxed round-by-round knowledge soundness). Π_{eval}^Σ has relaxed round-by-round knowledge soundness errors (Definition 3.14), with the knowledge state function and the algorithm E of Definition 7.23,

$$\varepsilon_j^{\text{rr}}(\delta) = (M_j + 2) \left(\frac{1}{|\mathbb{F}|} + \frac{1}{2^\beta} \right) \quad (j \in \llbracket 1, \ell \rrbracket), \quad \varepsilon_{\ell+1}^{\text{rr}}(\delta) = (1 - \delta)^\kappa,$$

for $\delta \in (0, \delta^*]$, and $\varepsilon_j^{\text{rr}}(\delta) = 1$ for $\delta \notin (0, \delta^*]$.

Proof. For $\delta \notin (0, \delta^*]$ there is nothing to prove. Fix $\delta \in (0, \delta^*]$, a statement, a round $j \in \llbracket 1, \ell + 1 \rrbracket$ and $\text{tr} = \tau_{j-1} \parallel \Pi_j$, where $\tau_0 = \text{tr}_\emptyset$. By item 4 of Definition 7.23, $\text{KState}_\delta(\cdot, \text{tr}, \cdot) = \text{KState}_\delta(\cdot, \tau_{j-1}, \cdot)$, and by items 2 and 3, $\text{KState}_\delta(\cdot, \text{tr} \parallel \text{vr}_j, f)$ does not depend on f . So the event of Definition 3.14 is contained in the conjunction of $\text{KState}_\delta(\cdot, \tau_{j-1}, E(\cdot)) = 0$, which does not depend on vr_j , and $\text{KState}_\delta(\cdot, \tau_j, \cdot) = 1$. If the first condition fails, the event is empty. Otherwise: for $j = 1$, $((z, v), y, E(y)) \notin (\mathcal{R}_{\text{eval}})_{\leq \delta}$; for $2 \leq j \leq \ell + 1$, $\tau_{j-1} \in \mathcal{D}_\delta^\perp$. For $j \leq \ell$, Lemma 7.24(1,2) and Lemma 7.22 with $|S| \leq M_j + 2$ bound the probability that $\tau_j \notin \mathcal{D}_\delta^\perp$ by $\varepsilon_j^{\text{rr}}(\delta)$. For $j = \ell + 1$, $\text{KState}_\delta(\cdot, \tau_{\ell+1}, \cdot) = 1$ means acceptance, whose probability is at most $(1 - \delta)^\kappa$ by Lemma 7.24(3). $\square$

Corollary 7.26 (Post-quantum knowledge soundness). Let $\delta \in (0, \delta^*]$ and $\varepsilon_{\text{rr}}(\delta) = \max\{(M_1 + 2)(1/|\mathbb{F}| + 2^{-\beta}), (1 - \delta)^\kappa\}$. The non-interactive scheme $\text{BCS}[\Pi_{\text{eval}}^\Sigma]$ (Section 3.5) satisfies the conclusion of Theorem 3.16 with $\nu = \ell + 1$, $\beta_{\min} = \min(\beta, \kappa(n + R))$, $l_{\max} = M/2$ and this $\varepsilon_{\text{rr}}(\delta)$. In particular, for every quantum adversary that makes at most Q queries to the random oracle and outputs a Merkle root cm , a claim (z, v) and a proof, except with probability $\varepsilon_{\text{ext}}(Q)$, if the verifier accepts then the extractor outputs a partial string $y \in (\Sigma \sqcup \{\perp\})^{M/2}$, a table f and a trapdoor such that: the openings of all positions of $\text{Dom}(y)$ are accepted under cm ; y agrees with $\text{fib}_0(\text{Enc}(f))$ on at least $(1 - \delta)M/2$ fibres; and $\tilde{f}(z) = v$.

Proof. Theorem 7.25 and Theorem 3.16; the last sentence unfolds Definition 3.15 for $\mathcal{R}_{\text{eval}}$ ($\perp$ entries count as disagreements). $\square$

The extracted table is bound to the root: two extracted witnesses for the same root with different tables expose a hash collision.

Lemma 7.27 (Uniqueness of relaxed committed witnesses). *Let $\delta \in (0, \delta^*]$ and let cm be a Merkle root. Let $\text{cx} = ((z, v), \text{cm})$ and $\text{cx}' = ((z', v'), \text{cm})$, and suppose that $(\text{cx}, (y, f, \text{td}))$ and $(\text{cx}', (y', f', \text{td}'))$ belong to $\widehat{\text{CR}}[\mathcal{R}_{\text{eval}}, \delta]$. If $f \neq f'$, then td and td' yield two accepting openings, under cm , of one position to different values, hence a collision of the random oracle. In particular, if $z = z'$ and $v \neq v'$, such a collision is obtained.*

Proof. Let $A = \{\eta \in L_1 : y(\eta) = \text{fib}_0(\text{Enc}(f))(\eta)\}$ and $A' = \{\eta \in L_1 : y'(\eta) = \text{fib}_0(\text{Enc}(f'))(\eta)\}$. Then $A \subseteq \text{Dom}(y)$ and $A' \subseteq \text{Dom}(y')$, and $|A|, |A'| \geq (1 - \delta)M/2$, so $|A \cap A'| \geq (1 - 2\delta)M/2 \geq \rho M/2 = N/2$. If $y = y'$ on $A \cap A'$, then $\text{Enc}(f)$ and $\text{Enc}(f')$ agree on the at least N points of the fibres over $A \cap A'$, hence are equal (Lemma 2.19(2)), and $f = f'$ (Lemma 6.2). So if $f \neq f'$, some $\eta \in A \cap A'$ has $y(\eta) \neq y'(\eta)$, and the openings of η computed from td and td' are both accepted under cm (Definition 3.15), with the same random oracle since the commitment index is fixed by the scheme; apply the salted form of Lemma 3.13(1) (Section 3.5). If $z = z'$ and $v \neq v'$, then $\tilde{f}(z) = v \neq v' = \tilde{f}'(z)$, so $f \neq f'$. $\square$

Lemma 7.27 is deterministic. A probability bound for an adversary that produces two accepting proofs for one root would require running the extractor of Theorem 3.16 on two proofs of one adversary, which lies outside the statement of that theorem. For the interactive oracle proof, evaluation binding is Corollary 7.14.

Remark 7.28 (Post-quantum parameters). Take $\rho = 1/4$, $\delta = \delta^* = 3/8$, $M \leq 2^{32}$ (so $\ell \leq n \leq 30$), the quartic extension $\mathbb{F} = \mathbb{F}_p[\iota]/(\iota^4 - 7)$ of the Goldilocks field, of size $p^4 > 2^{255.99}$ ($\iota^4 - 7$ is irreducible over $\mathbb{F}_p$ because 7 is a non-square in $\mathbb{F}_p$ and $p \equiv 1 \pmod{4}$ [LN97, Theorem 3.75]), $\beta = 320$, hash output length $\lambda_H = 256$, and $\kappa = 248$ queries, so that $(5/8)^\kappa < 2^{-168}$ and $(M_1 + 2)(1/|\mathbb{F}| + 2^{-\beta}) < 2^{-224}$. The quantities of Theorem 3.16 satisfy $q_s \leq \kappa(\ell + 1) + 2\ell + \lceil N_\ell/2 \rceil \leq 2^{30}$ (positions read: one per query and level, the sumcheck symbols, and the final table), $q_V \leq \nu + q_s(\lceil \log_2 l_{\max} \rceil + 1) \leq 2^{36}$ (one challenge query per round and one authentication path, of at most $\lceil \log_2 l_{\max} \rceil + 1$ hashes, per position read), and $q_{\widehat{\text{CR}}} \leq (M/2) \log_2 M \leq 2^{36}$ (one leaf hash and $\log_2(M/2)$ internal hashes per position of y). For an adversary making $Q = 2^{64}$ queries, Theorem 3.16 then gives $\varepsilon_{\text{ext}}(2^{64}) < 2^{-31.8}$: the first term is $2^{-31.84}$ and the other terms together are below 2^{-51} (computed exactly in rational arithmetic by `rust/examples/pq_params.rs` and `checks/pq_params.py` in the companion repository). The dominant term is $320 Q^2 \varepsilon_{\text{rr}}(\delta)$, so each additional factor 2 in Q costs about $2/\log_2(8/5) \approx 3$ further queries. The parameters of Section 9 (quadratic extension, $\kappa = 148$) give $\varepsilon_{\text{rr}}(\delta) \approx 2^{-100}$ and are sized for classical adversaries only.

Remark 7.29 (Parameters). For $\rho = 1/4$ and $\delta = (1 - \rho)/2 = 3/8$, one query passes with probability at most $5/8$; $\kappa = 148$ queries give $(5/8)^\kappa < 2^{-100}$. With $M = 2^{24}$ and $|\mathbb{F}| = p^2 > 2^{127.9}$ (the quadratic extension of Section 8), $\varepsilon_{\text{fold}} + 2\ell/|\mathbb{F}| < 2^{-103}$. These unique-decoding parameters are conservative; list-decoding analyses of FRI-type folding [BSCI⁺23] allow fewer queries (Section 10.1).

Remark 7.30 (Scope). • The analysis concerns one commitment opened at one point z per proof. Reusing a commitment across several openings, batching, or choosing z after the commitment inside a larger Fiat–Shamir transcript is not analysed here.

- Corollary 7.26 applies to the compiler BCS of [CDHZ25, Construction 11.7], with salted Merkle trees and the challenge derivation described in Section 3.5. The implementation of Section 8 follows the structure of this compiler (Section 8.3).

- For $\ell = 0$ the same argument applies with $\Sigma = \mathbb{F}$, the implicit instance $y = w_0$ and the Hamming distance, giving the single error $(1 - \delta)^\kappa$: the verifier rejects on $\perp$, E decodes the filled word at radius δ^* , and, for a fixed final table g , the accepted query points lie in $\{\xi : y(\xi) \neq \perp, y(\xi) = G(\xi)\}$, whose density is less than $1 - \delta$ unless $\Delta(y, \text{ev}_L(G)) \leq \delta$ and $v = \tilde{g}(z)$, in which case $E(y) = g$ is a witness (as in Proposition 7.15).
- The bounds also hold against classical adversaries in the random oracle model, with the same (quantum) extractor (Section 3.5).

7.8 Committed levels

We transfer the results of this section to the protocol $\Pi_{\text{eval}}^{\mathcal{J}}$ of Section 6.3, for any set $\mathcal{J} \subseteq \llbracket 0, \ell - 1 \rrbracket$ of committed levels with $0 \in \mathcal{J}$, where $1 \leq \ell \leq n$, using the augmentation aug of Section 6.3. It is computable in time polynomial in M .

The IOR with committed levels. $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ is $\Pi_{\text{eval}}^{\mathcal{J}}$ encoded as an IOR as in Section 7.7, with the following changes.

- The alphabet is the disjoint union $\Sigma_{\mathcal{J}} = \mathbb{F}^2 \sqcup \bigsqcup_{d \in D} \mathbb{F}^{2^d}$, where $D = \{d_j : j \in \mathcal{J} \setminus \{0\}\} \setminus \{1\}$: a symbol is a fibre, as in $\Pi_{\text{eval}}^{\Sigma}$, or a coset of one of the sizes used.
- The implicit instance is $y = \pi(\text{fib}_0(w_0))$, where π is the fixed permutation of the $M/2$ positions that makes the 2^{d_0-1} fibres of each coset $S_0(\xi)$ consecutive (the order of Section 8.3). The relation is $\mathcal{R}_{\text{eval}}^{\mathcal{J}} = \{((z, v), y, f) : y = \pi(\text{fib}_0(\text{Enc}(f))), \tilde{f}(z) = v\}$; a symbol of y that is not in $\mathbb{F}^2$ differs from every fibre, so $((z, v), y, f) \in (\mathcal{R}_{\text{eval}}^{\mathcal{J}})_{\leq \delta}$ if and only if $((z, v), \pi^{-1}(y), f) \in (\mathcal{R}_{\text{eval}})_{\leq \delta}$ when symbols outside $\mathbb{F}^2$ are replaced by $\perp$.
- The round polynomials and the final table are written over $\mathbb{F}^2$ as in Section 7.7. For $j \in \mathcal{J} \setminus \{0\}$, the oracle part of the round- $(j+1)$ message is the string of the $M_j/2^{d_j}$ cosets of w_j , one symbol of $\mathbb{F}^{2^{d_j}}$ per coset, the values of a coset being listed in the order of Section 8.3.
- A query point reads the 2^{d_0-1} symbols of y that form its coset at level 0, and one symbol of each other committed word. The verifier rejects if a symbol that it reads is $\perp$ or does not lie in the expected component of $\Sigma_{\mathcal{J}}$.

The map aug^{Σ} sends a transcript of $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ to one of $\Pi_{\text{eval}}^{\Sigma}$: it applies π^{-1} to y ; it replaces every symbol outside the expected component by $\perp$; it unpacks each coset symbol of w_j into its 2^{d_j-1} fibre symbols, a $\perp$ coset giving $\perp$ fibres; and it inserts, in round $j+1$ for $j \in \llbracket 1, \ell - 1 \rrbracket \setminus \mathcal{J}$, the full word $\text{fib}_j(\text{fold}_{r_j}(\bar{w}_{j-1}))$, where $\bar{w}_{j-1}$ is the filled word of level $j-1$ (Section 7.7). The inserted word depends only on the part of $\text{aug}^{\Sigma}(\tau)$ before round $j+1$.

Theorem 7.31 (Committed levels). *Let $\mathcal{J} \subseteq \llbracket 0, \ell - 1 \rrbracket$ with $0 \in \mathcal{J}$.*

1. *Theorem 7.13 and Corollary 7.14 hold for $\Pi_{\text{eval}}^{\mathcal{J}}$ with the same errors.*
2. *$\Pi_{\text{eval}}^{\mathcal{J}}$ is round-by-round knowledge sound for $\mathcal{R}_{\text{eval}}^{\delta}$ with the errors of Theorem 7.19.*
3. *$\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ is an IOR from $\mathcal{R}_{\text{eval}}^{\mathcal{J}}$ to $\mathcal{R}_{\text{acc}}$ with the relaxed round-by-round knowledge soundness errors of Theorem 7.25.*
4. *Corollary 7.26 holds for $\text{BCS}[\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}]$, with y read through π^{-1} , with the same ν , $\beta_{\min}$ and $\varepsilon_{\text{rr}}(\delta)$, with $l_{\max} = M/2$, and with $q_s \leq \kappa(2^{d_0-1} + |\mathcal{J}| - 1) + 2\ell + \lceil N_{\ell}/2 \rceil$ positions read.*

Proof. (1) By Lemma 6.5, for every instance the acceptance probability of a prover $\tilde{P}$ of $\Pi_{\text{eval}}^{\mathcal{J}}$ is at most that of the prover $\tilde{P}'$ of Π_{eval} , over the same challenges. Theorem 7.13 and Corollary 7.14 quantify over all provers of Π_{eval} , and apply to $\tilde{P}'$.

(2) Let $\mathcal{D}$ and Ext be as in Theorem 7.19. Declare a partial transcript τ of $\Pi_{\text{eval}}^{\mathcal{J}}$ doomed if $\text{aug}(\tau) \in \mathcal{D}$, and let $\text{Ext}^{\mathcal{J}}(\tau, \mathbf{m}) = \text{Ext}(\text{aug}(\tau), \mathbf{m}')$, where $\mathbf{m}'$ is $\mathbf{m}$ together with the word that aug inserts in that round, if any. Condition 1 of Definition 3.7 holds because aug does not change the empty transcript. For condition 2, $\text{aug}(\tau, \mathbf{m}, \mathbf{c}) = (\text{aug}(\tau), \mathbf{m}', \mathbf{c})$, because the inserted word depends only on $\text{aug}(\tau)$; so the probability in condition 2 for $\Pi_{\text{eval}}^{\mathcal{J}}$ at $(\tau, \mathbf{m})$ equals the one for Π_{eval} at $(\text{aug}(\tau), \mathbf{m}')$, and the conclusion is the same. For condition 3, if the verifier of $\Pi_{\text{eval}}^{\mathcal{J}}$ accepted a doomed full transcript τ , the verifier of Π_{eval} would accept the doomed transcript $\text{aug}(\tau)$, by Lemma 6.5.

(3) Let KState and $\mathbf{E}$ be as in Definition 7.23. Define

$$\text{KState}_{\delta}^{\mathcal{J}}(\mathbf{x}, y, \text{tr}, f) = \text{KState}_{\delta}(\mathbf{x}, y', \text{aug}^{\Sigma}(\text{tr}), f)$$

if tr is not full, where y' is y after π^{-1} and the replacement of symbols outside $\mathbb{F}^2$ by $\perp$, and as acceptance by the verifier of $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ if tr is full; and let $\mathbf{E}^{\mathcal{J}}$ run $\mathbf{E}$ on y' . Condition 1 of Definition 3.14 holds by the description of $(\mathcal{R}_{\text{eval}}^{\mathcal{J}})_{\leq \delta}$ above, condition 2 because aug^{Σ} of a transcript followed by a prover message is aug^{Σ} of the transcript followed by a message, and condition 3 by definition. Fix a round j and $\text{tr} = \tau_{j-1} \parallel \Pi_j$. For $j \leq \ell$, $\text{tr} \parallel \text{vr}_j$ is not full and $\text{aug}^{\Sigma}(\text{tr} \parallel \text{vr}_j) = \text{aug}^{\Sigma}(\text{tr}) \parallel \text{vr}_j$, so the event of Definition 3.14 for $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ is the event for $\Pi_{\text{eval}}^{\Sigma}$ at $\text{aug}^{\Sigma}(\text{tr})$, bounded by Theorem 7.25, which holds for every transcript. For $j = \ell + 1$, suppose that the verifier of $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ accepts. It read no $\perp$ and no symbol outside its component, so the values it read are those of the filled words, and the verifier of $\Pi_{\text{eval}}^{\mathcal{J}}$ accepts the transcript with the filled words. By Lemma 6.5, the verifier of Π_{eval} accepts its augmentation, whose inserted words are those of aug^{Σ} . The symbols that the verifier of $\Pi_{\text{eval}}^{\Sigma}$ reads in $\text{aug}^{\Sigma}(\text{tr} \parallel \text{vr}_{\ell+1})$ are fibres of the cosets read above or of inserted full words, so none is $\perp$ and they agree with the filled words; hence the verifier of $\Pi_{\text{eval}}^{\Sigma}$ accepts. The event is therefore contained in the event for $\Pi_{\text{eval}}^{\Sigma}$, bounded by $(1 - \delta)^{\kappa}$.

(4) By (3) and Theorem 3.16, as in Corollary 7.26. The rounds and challenges are those of $\Pi_{\text{eval}}^{\Sigma}$, the query positions are determined by the challenges, the longest string is y , and a query point reads 2^{d_0-1} symbols of y and one symbol of each of the $|\mathcal{J}| - 1$ other committed words. $\square$

Remark 7.32 (Parameters with committed levels). For $\mathcal{J} = \{0, k, 2k, \dots\} \cap [0, \ell - 1]$ with $k = 4$ and the parameters of Remark 7.28, $q_s \leq \kappa(2^{\min(4, \ell)-1} + \lceil \ell/4 \rceil - 1) + 2\ell + \lceil N_{\ell}/2 \rceil \leq 2^{30}$, and the other quantities of Remark 7.28 are unchanged or smaller; so the bound $\varepsilon_{\text{ext}}(2^{64}) < 2^{-31.8}$ of Remark 7.28 holds for $\text{BCS}[\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}]$. The implementation encodes the openings of the positions read in one committed word by a merged authentication path (Section 8.3): the verifier hashes the same salted leaves and nodes, in the same tree, as with separate paths, and rejects unless every node of the merged path is used exactly once. An adversary that outputs proofs in this form is turned into one that outputs separate paths by recomputing, for each position read, its leaf hash and the nodes of its path, at most $q_s(\lceil \log_2 l_{\max} \rceil + 1)$ additional oracle queries; Theorem 3.16 applied to this adversary gives the bound with Q replaced by $Q + q_s(\lceil \log_2 l_{\max} \rceil + 1) \leq Q + 2^{35}$, which does not change it at $Q = 2^{64}$ to the stated precision.

8 Implementation

We implemented Π_{eval} of Section 6, compiled as in Section 3.5 with salted Merkle trees and Fiat–Shamir, as a self-contained Rust library, `kbfold` (about 2,200 lines including tests and benchmarks;

its dependencies are the BLAKE3 hash function and the operating system’s random number generator, through the `getrandom` crate). The code follows the paper line by line; this section records the concrete choices, and Section 9 reports measurements. The library also has a coefficient-form mode, used only as the baseline of Section 9.

Index conventions. The paper numbers bits, variables and rounds from 1; the code, like most Rust code, numbers them from 0. Bit b_k of the paper is bit $k - 1$ of the code (`(b >> (k-1)) & 1`), the variable x_k and the coordinate z_k are entries $k - 1$ of the corresponding arrays, and the round polynomial s_j and the challenge r_j are stored at index $j - 1$, while the oracle w_j (sent in round $j + 1$) is stored at index j . Tables $f : \llbracket 0, N - 1 \rrbracket \rightarrow \mathbb{F}_q$ are arrays of length N indexed identically in both.

8.1 Fields

Base field. Tables and the initial codeword w_0 live over the Goldilocks field $\mathbb{F}_p$, $p = 2^{64} - 2^{32} + 1$. The largest power of 2 dividing $p - 1$ is 2^{32} , so smooth domains of order up to 2^{32} exist (Lemma 2.15). We take $L = \langle \omega \rangle$ with $\omega = 7^{(p-1)/M}$, where 7 generates $\mathbb{F}_p^\times$: since $p - 1 = 2^{32} \cdot 3 \cdot 5 \cdot 17 \cdot 257 \cdot 65537$, this amounts to $7^{(p-1)/\ell'} \neq 1$ for each of the six prime factors ℓ' , which the test suite checks. Multiplication uses the standard reduction of a 128-bit product modulo p based on $2^{64} \equiv 2^{32} - 1$ and $2^{96} \equiv -1$; elements are kept in canonical form.

Extension fields. The code is generic in the challenge field $\mathbb{F}$ of Section 6.1, with two instances: the quadratic extension $\mathbb{K} = \mathbb{F}_p[\iota]/(\iota^2 - 7)$, of size $p^2 \approx 2^{128}$, for the classical parameters of Remark 7.29 and Section 9; and the quartic extension $\mathbb{F}_p[\iota]/(\iota^4 - 7)$, of size $p^4 \approx 2^{256}$, for the post-quantum parameters of Remark 7.28. The challenges r_j , the evaluation point z , the value v , the sumcheck tables and all folded words w_j ($j \geq 1$) live in $\mathbb{F}$, while tables and w_0 have entries in $\mathbb{F}_p$. The polynomial $\iota^2 - 7$ is irreducible because 7, a generator of $\mathbb{F}_p^\times$, is a non-square; $\iota^4 - 7$ is irreducible by Remark 7.28. The quartic extension is implemented as $\mathbb{K}[\nu]/(\nu^2 - \iota)$, where ι is the generator of $\mathbb{K}$; then $\nu^4 = 7$, and ν is the element written ι in $\mathbb{F}_p[\iota]/(\iota^4 - 7)$. Elements of $\mathbb{F}$ are written in the basis $1, \iota, \dots, \iota^{e-1}$, $e \in \{2, 4\}$ (the bijection num of Section 7.7). Multiplication in $\mathbb{K}$ uses three base-field multiplications (Karatsuba) and one multiplication by the constant 7. Mixing base-field words with extension-field challenges is only needed in the first fold, which the code handles generically.

8.2 Encoding and folding

Commit. $\text{Enc}(f)$ (Definition 6.1) is computed in three steps: the kernel-to-monomial transform $\text{coef}(U_f) = C^{\otimes n} f$ of Corollary 4.11, applied in place one tensor factor at a time as $(x, x') \mapsto (x', x + x')$ (exactly $\frac{n}{2}N$ additions, no multiplications); zero-padding to length M ; and an iterative radix-2 NTT producing $w_0(\omega^i)$ in natural order.

Domain layout. With natural ordering, $L_j = \langle \omega^{2^j} \rangle$ is stored as $(\omega^{2^j i})_{i \in \llbracket 0, M_j - 1 \rrbracket}$. The fibre of index $i \in \llbracket 0, M_{j+1} - 1 \rrbracket$ is the pair of positions $(i, i + M_{j+1})$, since $\omega^{2^j M_{j+1}} = -1$, and its image under squaring is position i of L_{j+1} . Folding therefore reads the two halves of the array and writes one array of half size, with no permutation.

Fold. Both prover and verifier evaluate (5) in the line form of Lemma 5.7. With $a = w(\xi)$ and $a' = w(-\xi)$, they compute $a_e = w_e(\xi^2) = (a + a')/2$ and $a_o = w_o(\xi^2) = (a - a')/(2\xi)$, and then $\text{fold}_r(w)(\xi^2) = (a_o - a_e) + r(2a_e - a_o)$ (Lemma 5.7). This costs one multiplication by r in $\mathbb{K}$ and two multiplications by base-field constants. The inverses $(2\xi)^{-1} = 2^{-1}\omega^{-2^j i}$ are read from a single table of ω^{-i} , $i < M/2$, shared by all levels. The same function is used by the prover on whole words and by the verifier on single fibres, which rules out the parametrisation mismatch of Remark 5.12 by construction.

Sumcheck. The prover keeps the tables f_{j-1} and χ_{j-1} of Section 6.2 as arrays over $\mathbb{K}$. It sends $s_j(0)$, $s_j(1)$ and $s_j(2)$, computed in one pass over the pairs of entries at positions $2c$ and $2c + 1$ of both tables, using

$$s_j(2) = \sum_c (2f_{j-1}(2c + 1) - f_{j-1}(2c)) (2\chi_{j-1}(2c + 1) - \chi_{j-1}(2c)).$$

Both tables are then restricted in place (Definition 2.10). The verifier recovers $s_j(r_j)$ by Lagrange interpolation on $\{0, 1, 2\}$.

8.3 Commitments and transcript

Committed levels. The implementation runs $\Pi_{\text{eval}}^{\mathcal{J}}$ (Section 6.3) with $\mathcal{J} = \{0, k, 2k, \dots\} \cap \llbracket 0, \ell - 1 \rrbracket$; the parameter $k \in \llbracket 1, 6 \rrbracket$ is part of the parameters, $k = 4$ by default, and $k = 1$ gives Π_{eval} . The prover computes every folded word, since each is needed for the next fold, but builds Merkle trees only for w_j , $j \in \mathcal{J}$.

Leaves. The trees are the salted Merkle trees of Section 3.5: a leaf is $\mathbf{H}_{K_0}(\text{values} \parallel \gamma)$ with a uniformly random salt γ of 32 bytes ($= \lambda_{\mathbf{H}}/8$), and an internal node is $\mathbf{H}_{K_1}(\text{left} \parallel \text{right})$, where $\mathbf{H}_K$ is BLAKE3 in keyed mode under one of two distinct fixed keys, which separate the two domains; an internal node is then 64 bytes, one BLAKE3 compression. The commitment tree has one leaf per fibre of w_0 , holding its two values, and its leaves are ordered as in Section 7.8: with the natural ordering of L , the fibre over the point of index $f \in \llbracket 0, M/2 - 1 \rrbracket$ of L_1 is the leaf $t2^{d_0-1} + u$, where $f = t + uM/2^{d_0}$ with $t < M/2^{d_0}$, so the 2^{d_0-1} fibres of a coset are consecutive leaves. For $j \in \mathcal{J} \setminus \{0\}$, the tree of w_j has one leaf per coset, holding its 2^{d_j} values: leaf $t \in \llbracket 0, M_j/2^{d_j} - 1 \rrbracket$ holds w_j at the positions $t + uM_j/2^{d_j}$, $u \in \llbracket 0, 2^{d_j} - 1 \rrbracket$. The salts of a tree are the output of BLAKE3 in keyed mode (as an extendable-output function) under a seed of 32 bytes drawn from the operating system: one seed per commitment and one per opening, so that the salts are fresh and secret.

Openings and local folds. A query point of index i reads, at level $j \in \mathcal{J}$, the coset of index $t = i \bmod (M_j/2^{d_j})$. For each committed word, the proof opens each distinct leaf once, in increasing order, with its values and salt, and gives the authentication paths of all opened leaves as one merged path: the nodes that cannot be recomputed from the opened leaves, level by level from the leaves to the root, each level from left to right. The verifier recomputes the set of opened leaves from the query points, and rejects unless the merged path uses every node exactly once and ends at the root. It then folds each opened coset d_j times (Lemma 6.4), with the inverses $(2\xi)^{-1}$ obtained from one power of ω^{-1} per coset and fold step, and compares the result with the entry of the next committed coset, or with G .

Fiat–Shamir. The transcript is a BLAKE3 hash chain, and follows the rounds of $\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$ (Sections 7.7 and 7.8). It absorbs the parameters (n, R, ℓ, κ) , the basis, the salt length, e and k , the root of w_0 , the point z and the value v . In round $j \in \llbracket 1, \ell \rrbracket$ it absorbs s_j , then (for $j \geq 2$ and $j - 1 \in \mathcal{J}$) the root of w_{j-1} , then a fresh round salt of 32 bytes, and derives r_j . In round $\ell + 1$ it absorbs the final table g and a round salt, and derives the κ query points. A challenge in $\mathbb{F}$ is the image under the map ϕ of Section 7.7 of $\beta = 64(e + 1)$ output bits (192 for $e = 2$, 320 for $e = 4$): the bits are read as an integer, whose first e digits in base p are the coordinates of r_j . The κ query points are κ consecutive blocks of $n + R$ output bits, read as exponents of ω (the map pos).

Relation to the compiler of [CDHZ25]. The implementation has the structure of $\text{BCS}[\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}]$: salted trees, one round salt per round, challenges derived from the roots and salts of all previous rounds, and the maps ϕ and pos; merged paths are an encoding of the separate paths (Remark 7.32). The messages that the verifier reads entirely (the round polynomials and g) are absorbed into the transcript in the clear rather than committed by a tree, which is the usual encoding of such messages. With the quartic extension, $\kappa = 248$ and $\beta = 320$, the implementation uses the parameters of Remarks 7.28 and 7.32.

Proof format. A proof consists of the ℓ round polynomials (three values each), the roots of the words w_j , $j \in \mathcal{J} \setminus \{0\}$, the $\ell + 1$ round salts, the final table g , and, for each committed word, the opened leaves (values and salts) and the merged path. Its encoding has no length prefixes, since every length is fixed by the parameters and the query points, and the verifier rejects a proof in which any length differs. The repository contains test vectors: transcript challenges in both extensions, and complete proofs produced from fixed seeds.

Final message. The verifier converts g to monomial form with the transform $C^{\otimes(n-\ell)}$ and evaluates G at each ξ_ℓ by Horner’s rule. It evaluates $\tilde{g}(z_{\ell+1}, \dots, z_n)$ for the closure check by $n - \ell$ successive restrictions (Lemma 2.11).

8.4 Testing

The test suite checks each component against a direct definition (field arithmetic against 128-bit integer arithmetic, the transforms against the expansion of Corollary 4.4, the NTT against Horner evaluation) and the protocol end to end:

- *Completeness.* Honest proofs verify for both extensions, $k \in \{1, 2, 3, 4\}$, all $1 \leq n \leq 9$, all $0 \leq \ell \leq n$ and rates $\rho \in \{1/2, 1/4, 1/8\}$, and the returned value equals $\tilde{f}(z)$ computed independently.
- *Tampering.* Proofs are rejected after changing the claimed value, the point or the commitment; after every single change of one sumcheck value, one entry of g , one root, one round salt, one opened value, one leaf salt or one node of a merged path, and after reordering opened leaves, for $k \in \{1, 2, 3, 4\}$; after changing the length of any of these lists or strings; and when a proof for a different table is checked against the original commitment. The verifier returns an error and never panics on such inputs, which randomised truncations also check.
- *Fresh salts.* Two commitments to the same table have different roots, and two openings have different round salts; both verify.

- *A consistent cheater.* Consider a prover that commits honestly and claims $v + 1$. In every round it adds half of the current discrepancy to each of $s_j(0)$, $s_j(1)$ and $s_j(2)$, so that every sumcheck check passes; it folds honestly, and modifies $g(0)$ so that the closure check passes. This prover is rejected in every tested configuration, and always in the query phase, as predicted by case 2 of Theorem 7.13.

Independently, each algebraic statement of Sections 4 and 5 and the two lemmas of Section 7.2 were checked numerically over small prime fields ($\mathbb{F}_{17}$ exhaustively where feasible, and $\mathbb{F}_{257}$); these scripts are part of the artefact.

9 Experiments

This section measures the implementation of Section 8. It has four goals: to check that the kernel encoding costs the same as coefficient-form folding at identical engineering (as predicted by Remark 6.8), to measure the cost of the post-quantum parameters and of the salted compiler, to locate where the time goes and quantify the protocol parameters ℓ and ρ , and to place the implementation next to an independent multilinear commitment scheme.

9.1 Setup

Machine and code. All runs use a cloud virtual machine with an Intel Xeon processor at 2.10 GHz (2 vCPUs, 7 GB RAM), Rust 1.95.0, release profile with fat LTO and one codegen unit, and no `target-cpu` flag. The library is `kbfold` 0.3.0; the script `rust/bench/run_all.sh` runs every measurement of this section and writes the data of the tables to `rust/results/`. Unless stated otherwise, runs are single-threaded. Times are medians over 11 runs for $n \leq 16$, 5 for $n \in \{18, 20\}$ and 3 for $n = 22$ and for Tables 4 and 5; verification times are medians over 21 runs. The machine is shared, and timings of identical work vary by up to about 20% between runs (Section 9.4).

Parameters. The *classical parameters* are: rate $\rho = 1/4$ ($R = 2$), $\ell = n - 4$ folding rounds (final table of 16 elements), $\kappa = 148$ queries, the quadratic extension $\mathbb{K}$, 32-byte salts, and committed levels $\mathcal{J} = \{0, 4, 8, \dots\}$ ($k = 4$, Section 6.3). By Theorem 7.13 with $\delta = (1 - \rho)/2 = 3/8$, the query term is $(5/8)^{148} < 2^{-100}$, and $\varepsilon_{\text{fold}} + 2\ell/|\mathbb{K}| < 2^{-100}$ for all sizes tested (Remark 7.29). The *post-quantum parameters* are those of Remarks 7.28 and 7.32: the same rate, ℓ and k , $\kappa = 248$ queries and the quartic extension. Tables f are uniformly random over $\mathbb{F}_p$ and the point z is uniformly random in $\mathbb{F}^n$. Proof sizes are the lengths of the canonical encoding of Section 8.3; they depend slightly on the query points, through the number of distinct opened leaves.

Baseline. To isolate the effect of the encoding, the library has a second mode that differs from Π_{eval} in exactly two places: the table is converted to the monomial coefficients of $\tilde{f}$ by Möbius inversion (Lemma 2.9) instead of the transform of Corollary 4.11, and words are folded with the classical FRI fold $\text{cfold}_\theta(w) = w_e + \theta w_o$ (Definition 5.10) instead of the kernel fold (Remark 5.14). This is the coefficient-form folding used in Reed–Solomon instantiations of BaseFold and in DeepFold; everything else (field, NTT, Merkle trees, transcript, sumcheck, query phase) is shared. The baseline passes the same completeness and tampering tests. It is *not* the BaseFold or DeepFold code base, and Table 1 compares encodings, not implementations.

n	Commit (ms)		Open (ms)		Verify (ms)		Proof (KiB)
	kernel	coeff.	kernel	coeff.	kernel	coeff.	
12	1.9	2.1	0.8	0.8	0.54	0.52	80
14	8.7	8.9	3.0	2.9	0.69	0.66	114
16	35.8	33.4	13.2	11.5	1.16	0.83	146
18	159.5	174.3	46.9	49.8	1.11	1.07	194
20	610.4	647.9	231.0	233.2	1.21	1.23	237
22	3574.1	3273.1	1056.5	1120.1	1.37	1.54	290

Table 1: Kernel encoding versus the coefficient-form baseline, single-threaded, classical parameters ($\rho = 1/4$, $\ell = n - 4$, $\kappa = 148$, $\mathbb{F} = \mathbb{K}$, salted trees, $k = 4$). Proof sizes (kernel encoding) agree within 3% for the two encodings.

n	Classical ($\mathbb{K}$, $\kappa = 148$)			Post-quantum ($\mathbb{F}_{p^4}$, $\kappa = 248$)		
	prover (ms)	verify (ms)	proof (KiB)	prover (ms)	verify (ms)	proof (KiB)
12	2.7	0.54	80	5.3	1.26	123
14	11.7	0.69	114	17.3	1.78	210
16	49.1	1.16	146	74.7	2.00	295
18	206.4	1.11	194	341.8	2.83	396
20	841.5	1.21	237	1319.2	2.70	498
22	4630.5	1.37	290	5915.8	5.13	616

Table 2: Classical and post-quantum parameters (Remarks 7.28 and 7.29), kernel encoding, single-threaded; the prover time is commit plus open.

9.2 Kernel versus coefficient-form encoding

Table 1 reports commit, open and verify times for $n = 12, \dots, 22$ ($N = 2^{12}, \dots, 2^{22}$). Proofs have the same shape in both modes; their sizes differ only through the number of distinct opened leaves. The differences between the two encodings change sign from one size to the next (the kernel encoding is 9% faster to commit at $n = 18$ and 9% slower at $n = 22$), and they are of the same size as the variation of identical work between runs: the kernel commitment at $n = 20$ takes 610 ms in Table 1 and 685 ms in the run of Table 3. The two encodings are therefore indistinguishable at the resolution of this machine. This is the outcome predicted by Remark 6.8: the kernel transform and Möbius inversion both cost $\frac{n}{2}N$ additions, and in line form (Lemma 5.7) the kernel fold costs one multiplication by r per output element, the same as the classical fold. The kernel encoding thus provides its structural property (Theorem 5.3) at no measurable cost.

9.3 Post-quantum parameters, salts and parallelism

Table 2 compares the two parameter sets. The post-quantum parameters cost 1.3–2 times the prover time and 1.5–2.1 times the proof size of the classical ones: the extension doubles the size of every folded value and of the sumcheck arithmetic, and the number of queries grows from 148 to 248. The commitment does not depend on the parameter set, since w_0 lives over $\mathbb{F}_p$ in both cases; the differences in prover time come from opening.

Salts. At $n = 20$ with the classical parameters, the prover takes 809 ms with unsalted trees and 909 ms with 32-byte salts. Salts add 49 KiB to the proof (187 to 236 KiB): one salt per distinct opened leaf (up to 8 fibres per query in the commitment tree and one coset per query in each of

	transform	NTT	Merkle tree	commit	open
kernel	14.0	250.2	501.1	684.8	252.8
coefficient form	13.9	238.3	505.9	649.5	210.0

Table 3: Components of the commitment in ms ($n = 20$, $M = 4N$, classical parameters, single-threaded), measured separately from Table 1. The NTT and Merkle columns execute identical code in both modes.

ℓ	final table N_ℓ	open (ms)	verify (ms)	proof (KiB)
8	4096	244.7	6.8	235
10	1024	249.8	2.5	218
12	256	255.9	1.5	226
14	64	257.5	1.2	235
16	16	275.6	1.3	236
18	4	244.8	1.4	237
20	1	227.9	1.3	241

Table 4: Effect of the number of folding rounds ℓ (Remark 6.3), kernel encoding, $n = 20$, classical parameters otherwise.

the three other trees) and 17 round salts.

Parallelism. With the feature `parallel`, the transforms, the NTT, the Merkle trees, the folds and the sumcheck run on all cores; the proofs are identical to the single-threaded ones, which the test vectors check. On the two vCPUs of this machine, the prover time with the classical parameters drops from 842 to 632 ms at $n = 20$ and from 4631 to 2920 ms at $n = 22$ (speed-ups 1.3 and 1.6), and with the post-quantum parameters from 1319 to 965 ms and from 5916 to 4263 ms (1.4 both). The verifier stays single-threaded.

9.4 Where the time goes

Table 3 splits the commitment into its three steps. The kernel transform takes 2% of the commitment, as long as Möbius inversion: both perform $\frac{n}{2}N$ additions. The commitment is dominated by hashing: BLAKE3 over $M/2$ salted fibre leaves and $M/2 - 1$ internal nodes takes twice as long as the NTT. Opening costs about a third of committing: the prover folds every word, but builds trees only for the committed words w_4, w_8, w_{12} , whose leaves hold 16 values each; the sumcheck ($O(N)$ multiplications in $\mathbb{F}$) is a minor term.

Verification takes 0.5–1.4 ms with the classical parameters and grows slowly with n : each query opens four leaves, and the merged paths share their upper nodes. Proofs take 80–290 KiB; at $n = 20$, about 44% is opened values, 20% salts and 35% path nodes.

9.5 The number of folding rounds

Table 4 varies ℓ at $n = 20$. With committed levels, a folding round adds a committed word only every $k = 4$ rounds, so proof sizes vary little with ℓ (218–241 KiB); stopping earlier adds $16N_\ell$ bytes for the final table, which dominates at $\ell = 8$. Verification time is smallest around $\ell = 14$: for smaller ℓ the verifier’s κ evaluations of G , each of cost $O(N_\ell)$, take over. Opening time varies little with ℓ , since the first folds dominate it.

ρ	δ	κ	commit (ms)	open (ms)	verify (ms)	proof (KiB)
1/2	1/4	241	300.8	143.0	1.8	320
1/4	3/8	148	691.1	243.4	1.3	238
1/8	7/16	121	1309.4	452.0	1.5	216

Table 5: Effect of the rate at 100 bits of query security, $\delta = (1 - \rho)/2$, kernel encoding, $n = 20$, $\ell = 16$.

parameters	k	prover (ms)	verify (ms)	proof (KiB)
classical	1	1271.8	2.06	556
classical	2	954.6	1.65	296
classical	3	1035.1	1.54	233
classical	4	989.1	1.22	237
post-quantum	1	1874.0	5.61	952
post-quantum	4	1220.2	4.14	502

Table 6: Committed levels $\mathcal{J} = \{0, k, 2k, \dots\}$ (Section 6.3), kernel encoding, $n = 20$, $\ell = 16$, single-threaded; $k = 1$ is Π_{eval} with merged paths.

9.6 The rate

Table 5 fixes the query term of Theorem 7.13 at 2^{-100} and varies the rate. Halving the rate doubles the domain and roughly doubles the prover’s work; it also reduces the number of queries, which shortens proofs. Within the unique-decoding analysis of this paper, $\rho = 1/4$ is a reasonable default.

9.7 Committed levels

Table 6 varies the number k of levels between committed words. With $k = 1$ (Π_{eval}), merging the paths alone gives 556 KiB, against 1147 KiB with one authentication path per query and level. Committing to one word in k divides the number of opened leaves by about k , at the cost of 2^k values per leaf; the proof shrinks by a factor 1.9 at $k = 2$ and 2.3–2.4 at $k = 3$ and 4, where the larger leaves offset the fewer paths. The prover gets faster because it builds fewer trees, and the verifier because it hashes fewer paths. We take $k = 4$, which gives the fastest verifier.

9.8 An independent reference point

WHIR [ACFY24] is a multilinear polynomial commitment scheme from Reed–Solomon proximity testing. To place our implementation among existing systems, Table 7 runs its implementation¹ on the same machine with the same field, rate and security level, single-threaded, and without proof of work. We use two configurations: folding one variable per round with the unique-decoding analysis, and WHIR’s default of four variables per round with its analysis up to the Johnson bound, the configuration closest to ours in the number of committed words. The two code bases differ in many engineering choices (field arithmetic, Merkle trees, transcript, proof encoding), so Table 7 is a reference point, not a comparison of protocols.

On this machine, **kbfold** with the classical parameters has a prover 6.6–9 times faster than WHIR with one variable per round, with proofs 1.9–2.3 times smaller and a verifier 3–4 times faster. Against WHIR’s default configuration, the **kbfold** prover is 1.8–3.1 times faster, while

¹Repository <https://github.com/WizardOfMenlo/whir>, commit c03a4a5 of September 2026.

n	kbfold, classical			WHIR, unique, $k = 1$			WHIR, Johnson, $k = 4$		
	prover	verify	proof	prover	verify	proof	prover	verify	proof
16	49	1.16	146	445	4.3	285	100	0.54	75
18	206	1.11	194	1738	3.8	392	377	0.59	88
20	842	1.21	237	7200	4.9	513	2638	0.77	108
22	4631	1.37	290	30700	4.9	657	10800	0.86	121

Table 7: **kbfold** and the WHIR implementation, same machine, single-threaded, Goldilocks with its quadratic extension, rate 1/4, 100 bits of security, no proof of work. Prover and verifier times in ms, proof sizes in KiB. WHIR prints times of one second or more with a resolution of 0.1 s.

WHIR’s proofs are 1.9–2.4 times smaller and its verifier 1.6–2.1 times faster. Both schemes commit to one word in four; the remaining difference in proof size and verification time comes from the number of queries, which WHIR reduces by its analysis up to the Johnson bound. The list-decoding extension of Theorem 7.13 in Section 10.1 addresses exactly this term, and it is independent of the encoding.

10 Conclusion and Future Work

The Boolean-kernel basis $\{K_b\}$ gives an exact dictionary between two operations that are usually treated separately. One is even–odd folding of univariate polynomials over smooth multiplicative domains; the other is restriction of a variable of a multilinear polynomial in table form. In kernel coordinates the normalised fold pfold_r is the restriction (Theorem 5.3). A Reed–Solomon codeword can therefore carry a table directly, and the BaseFold paradigm runs on it with no change of representation. We gave a self-contained proof of soundness, evaluation binding and round-by-round knowledge soundness in the unique-decoding regime, whose only external input about codes is the correlated-agreement theorem for Reed–Solomon codes (with efficient unique decoding for the extractor). With the BCS theorem for interactive oracle reductions of [CDHZ25], this makes the non-interactive scheme knowledge sound, for a single opening, against quantum adversaries in the quantum random oracle model (Corollary 7.26). We also gave an implementation that follows the compiler of [CDHZ25], with classical and post-quantum parameters, in which the kernel encoding and coefficient-form folding are indistinguishable in running time at the resolution of our machine.

10.1 Future work

Soundness.

- *List decoding.* Extend Theorems 7.13 and 7.19 to proximity parameters up to the Johnson bound, in the spirit of [Hab24, Zei26]. The line form of Lemma 5.7 reduces each fold to a Reed–Solomon line, so correlated agreement beyond unique decoding [BSCI⁺23] is the natural tool; the main work is to replace the unique codewords c_j of the passing-set argument by lists. This would reduce the number of queries, in the classical parameters of Remark 7.29 and in the post-quantum parameters of Remark 7.28, where the quadratic loss of Theorem 3.16 makes the query count conservative.
- *Batched and repeated openings.* Prove soundness of the batched opening of Remark 6.9, by correlated agreement applied to the random linear combination of the committed words, and extend Corollary 7.26 to several openings of one commitment.

Zero knowledge. The kernel encoding is compatible with the standard masking techniques for folding-based schemes: committing to $U_f + X^N R$ for a random polynomial R of small degree, masking the sumcheck with a random polynomial in the manner of Libra [XZZ⁺19], and masking the folded words with a random codeword. The task is to prove that the fold dictionary and the soundness argument survive the extra degree, and that the resulting view is simulable.

Implementation.

- *Proof size.* After committed levels and merged paths (Sections 6.3 and 8.3), the remaining difference with the proofs of WHIR in Table 7 comes from the number of queries: the list-decoding analysis above, and a proof of work before the query phase [ACFY24], which trades prover time for queries and needs its own analysis in the setting of Theorem 3.16.
- *Engineering.* SIMD field arithmetic and a parallel verifier. None of them interacts with the encoding.

Formal verification. Formalise Lemma 6.4 and items 2–4 of Theorem 7.31; replace the axiom `bciks.unique` by a formal proof of correlated agreement in the unique-decoding regime, state relaxed round-by-round knowledge soundness for a general interactive oracle reduction in Lean, and link the Rust implementation to the Lean specification, for instance through test vectors extracted from Lean.

Other domains. The dictionary uses a smooth multiplicative subgroup in odd characteristic. Is there a kernel-type basis whose natural fold is table-form restriction on circle domains [HLP24] or on additive subspaces [LCH14, DP24]?

10.2 Artefacts

The Rust library, the Lean files, the benchmark data and the numerical checks are available at <https://github.com/qoosmo/kbifold> under the MIT or Apache-2.0 licences (code) and CC BY 4.0 (paper); this version of the paper corresponds to the release tagged `v0.3.0`, and the library is published on crates.io as `kbifold 0.3.0`.

- *Tests* (Section 8.4): `cargo test --release` in `rust/`.
- *Tables of Section 9:*
`cargo run --release --example bench -- scaling`, and likewise with `stop` and `rate`; the recorded data are in `rust/results/`.
- *Numerical checks* (Section 8.4):
`cargo run --release --example paper_checks`.
- *Post-quantum parameters* (Remark 7.28):
`cargo run --release --example pq_params`.
- *Lean:* `lake build` in `lean/` (Lean 4.23.0, Mathlib v4.23.0); `lean/KBFold/Audit.lean` prints the axioms of the main theorems.

Acknowledgments

The author thanks the developers of Lean, Mathlib and the Rust ecosystem, on which the companion artefacts are built. Generative AI tools (Anthropic’s Claude) were used to assist with writing and editing the text, with the Rust code and the Lean proofs, and with checking computations. The author checked all results and takes full responsibility for the content of the paper and the artefacts; every Lean proof is checked by the Lean kernel.

References

- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. IACR Cryptology ePrint Archive, Paper 2024/1586, 2024.
- [BCHO22] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orrù. Gemini: Elastic SNARKs for diverse environments. In *Advances in Cryptology – EUROCRYPT 2022*, 2022. IACR ePrint 2022/420.
- [BDF⁺11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69, 2011.
- [BGTZ24] Alexander R. Block, Albert Garreta, Pratyush Ranjan Tiwari, and Michał Zając. On soundness notions for interactive oracle proofs. *Journal of Cryptology*, 2024. IACR ePrint 2023/1256.
- [BMNW24] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Arc: Accumulation for Reed–Solomon codes. Cryptology ePrint Archive, Paper 2024/1731, 2024. Version of 25 October 2024.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.
- [BSCI⁺23] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. *Journal of the ACM*, 70(5), 2023. Preliminary version in FOCS 2020. Full version: IACR ePrint 2020/654.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography – TCC 2016-B*, 2016.
- [CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Advances in Cryptology – EUROCRYPT 2023*, 2023.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat–Shamir: From practice to theory. In *51st Annual ACM Symposium on Theory of Computing (STOC 2019)*, 2019.
- [CDHZ25] Alessandro Chiesa, Zijing Di, Zihan Hu, and Yuxi Zheng. How to prove post-quantum security for succinct non-interactive reductions. Cryptology ePrint Archive, Paper 2025/2166, 2025. Version of 2 March 2026.

- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In *Theory of Cryptography – TCC 2019*, 2019. IACR ePrint 2019/834.
- [DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. IACR Cryptology ePrint Archive, Paper 2024/504, 2024.
- [EG25] Liam Eagen and Ariel Gabizon. MERCURY: A multilinear polynomial commitment scheme with constant proof size and no prover FFTs. IACR Cryptology ePrint Archive, Paper 2025/385, 2025.
- [Gao03] Shuhong Gao. A new algorithm for decoding Reed–Solomon codes. In Vijay K. Bhargava, H. Vincent Poor, Vahid Tarokh, and Seokho Yoon, editors, *Communications, Information and Network Security*, pages 55–68. Kluwer, 2003.
- [GLH⁺25] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. DeepFold: Efficient multilinear polynomial commitment from Reed–Solomon code and its application to zero-knowledge proofs. In *34th USENIX Security Symposium*, 2025. IACR ePrint 2024/1595.
- [GLS⁺23] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In *Advances in Cryptology – CRYPTO 2023*, 2023.
- [GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. Samaritan: Linear-time prover SNARK from new multilinear polynomial commitments. In *Advances in Cryptology – ASIACRYPT 2025*, 2025. IACR ePrint 2025/419.
- [Hab24] Ulrich Haböck. Basefold in the list decoding regime. IACR Cryptology ePrint Archive, Paper 2024/1571, 2024.
- [HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. IACR Cryptology ePrint Archive, Paper 2024/278, 2024.
- [KT24] Tohru Kohrita and Patrick Towa. Zeromorph: Zero-knowledge multilinear-evaluation proofs from homomorphic univariate commitments. *Journal of Cryptology*, 2024. IACR ePrint 2023/917.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. Novel polynomial basis and its application to Reed–Solomon erasure codes. In *55th IEEE Symposium on Foundations of Computer Science (FOCS 2014)*, 2014.
- [LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4), 1992.
- [LN97] Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2 edition, 1997.
- [Mer89] Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology – CRYPTO ’89*, volume 435 of *LNCS*, pages 218–238, 1989.
- [Moh25] Malcom Mohamed. Protocols for univariate sumcheck. arXiv:2505.00554, 2025.

- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Advances in Cryptology – CRYPTO 2020*, 2020.
- [WB86] Lloyd R. Welch and Elwyn R. Berlekamp. Error correction for algebraic block codes. US Patent 4,633,470, 1986.
- [XZZ⁺19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology – CRYPTO 2019*, volume 11694 of *LNCS*, pages 733–764. Springer, 2019.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In *Advances in Cryptology – CRYPTO 2024*, 2024. IACR ePrint 2023/1705.
- [Zei26] Hadas Zeilberger. Khatam: Proximity gaps for multilinear evaluation for all linear codes. In *Advances in Cryptology – CRYPTO 2026*, pages 592–611, 2026.

A Notation

Symbol	Meaning	Where
<i>Integers, bits and points</i>		
$\mathbb{N}, \llbracket a, c \rrbracket$	non-negative integers; integers i with $a \leq i \leq c$ (empty if $a > c$)	§2.1
$n, N = 2^n$	number of variables; size of a table	§2.1
$b_k, (b_1, \dots, b_n)$	bit k of an index b (least significant first); bit vector	§2.1
B_n	Boolean hypercube $\{0, 1\}^n$, identified with $\llbracket 0, N - 1 \rrbracket$	§2.1
$\text{wt}(b), \bar{b}, c \preceq b$	Hamming weight; complement; submask	§2.1
$b = c + 2^t h, b = b_1 + 2b'$	splitting an index	§2.1
$(x, y), (x, b), r \leq_j$	concatenated points; prefix of a point	§2.1
<i>Fields and polynomials</i>		
$\mathbb{F}_q$	base field, finite, odd characteristic	§2.2
$\mathbb{F}, \mathbb{K}$	finite field containing $\mathbb{F}_q$ (challenge field); extension field	§2.2, §6.1
$\mathbb{F}[X]_{<d}, [X^a]P$	polynomials of degree $< d$; coefficient of X^a	§2.2
$P_e, P_o; R_i$	even and odd parts; blocks of a polynomial	Lemma 2.3
<i>Multilinear polynomials and tables</i>		
A	commutative ring	§2.3
$\mathcal{L}_n(A), \mathcal{L}_n$	multilinear polynomials over A , over $\mathbb{F}_q$	§2.3
m_a, e_b	canonical monomial; Lagrange polynomial	Definition 2.4
eq_1, eq	equality polynomials; $e_b(x) = \text{eq}(b, x)$	Definition 2.4 and Lemma 2.5
$f, f(b), \tilde{f}$	table; its entries; its multilinear extension	Proposition 2.6
α_a	coefficient form of $\tilde{f}$	Definition 2.8
$\text{res}_r(f)$	restriction of a table at $r \in A$ or $r \in A^j$	Definition 2.10
ϑ	ring homomorphism, applied entrywise	Lemma 2.12
<i>Smooth domains and codes</i>		
$L, M = 2^\mu$	smooth domain and its order	Definition 2.14

Symbol	Meaning	Where
L^{2^j}	image of L under $\xi \mapsto \xi^{2^j}$ (not a Cartesian power)	Definition 2.14 and Lemma 2.15
ξ, η, ζ	points of a domain; points of the squared domain; arbitrary field point	§2.4, §4.5
fibre $\{\xi, -\xi\}$	the two square roots of $\eta = \xi^2$ in L	Definition 2.16
$\text{ev}_L, \text{RS}_{\mathbb{F}}[L, d], \rho$	evaluation map; Reed–Solomon code; rate d/M	Definition 2.17
P_c	polynomial of degree $< d$ whose evaluation is the codeword c	Lemma 2.19
Frob	Frobenius map $x \mapsto x^q$	Lemma 2.21
Δ	relative Hamming distance	Definition 2.17 and Lemma 2.18
w_e, w_o	even and odd parts of a word	(1), Lemma 2.22
δ	proximity parameter, $0 < \delta \leq (1 - \rho)/2$	Theorem 2.23
$u^0, u^1; \hat{c}^0, \hat{c}^1$	words on a line; codewords of correlated agreement	Theorem 2.23 and Lemma 5.7
<i>Proof systems</i>		
$\mathcal{R}, \mathbf{x}, \mathbf{w}, L_{\mathcal{R}}$	relation, instance, witness, language	§3.1
$\mathbf{P}, \mathbf{V}, \mathbf{P}^*$	prover, verifier, arbitrary prover	Definition 3.1
$\nu, \mathbf{m}_i, \mathbf{c}_i, \mathbf{Ch}_i$	number of rounds; message; challenge; challenge set	Definition 3.1
τ, τ_i, Ω	partial transcript; after i rounds; challenge space	§3.1
E_i, k_i	events and counts of the sequential-challenge bound	Lemma 3.2
$\Omega', \varpi, \mathcal{S}(\varpi), \mathcal{A}, \mathbf{1}_{\mathcal{A}}$	finite probability space, its element, a set of points, an event, its indicator	Lemma 3.3
$L^{\times \kappa}$	set of κ -tuples of elements of L (Cartesian power)	Lemma 3.3, §6.2
$\mathcal{D}, \text{Ext}, \varepsilon_i, \varepsilon_{\text{rbr}}$	doomed set; extractor; round errors; maximum error	Definitions 3.6 and 3.7
F, s_j, s_j^*, r_j, v_j	summed polynomial; round polynomial sent; honest round polynomial; challenge; running claim	§3.3
$\text{Enc}, (w; z, v)$	encoding; instance of an evaluation protocol	Definition 3.10
$\text{dist}, \mathcal{R}_{\text{eval}}^\delta$	distance to encodings; evaluation relation	§3.4
$\text{H}, \lambda_{\text{H}}, Q$	hash function (random oracle); output length; number of oracle queries of the adversary	§3.5
$\mathbf{d}, T, \text{MT}(\mathbf{d})$	vector of leaves; its length; Merkle root	§3.5, Lemma 3.13
$\Sigma, y, \perp, \text{Dom}(y)$	alphabet; implicit instance (partial string); undefined entry; defined positions	§3.5
$\text{tr}, \text{tr}_\emptyset, \mathbf{a}, \gamma_h$	transcript; empty transcript; committed vector; salts	§3.5
$\mathcal{R}_{\leq \delta}, \mathcal{R}_{\text{acc}}, (\top, ())$	relaxed relation; acceptance relation; its instance (explicit $\top$, empty implicit instance $()$)	§3.5
n_y, n'_y	numbers of input and output implicit instances (1 and 0 here)	§3.5
I, h	set of opened positions; position index in a salted Merkle tree	§3.5
$\varepsilon_{\text{sim}}, \varepsilon^{\text{sr}}$	simulation error; state-restoration knowledge error ([CDHZ25])	§3.5, Theorem 3.16
$\varepsilon_{\text{extract}}, \varepsilon_{\text{offline}}, \varepsilon_{\text{online}}$	multi-extractability errors of Merkle commitments ([CDHZ25, Thm. 8.3])	proof of Theorem 3.16
$\mathbf{m}_{\text{FS}}, \mathbf{m}_{\text{VC}}, \mathbf{m}_{\text{MultiExtract}}, \mathbf{m}, \mathbf{m}'$	total query masses ([CDHZ25, Def. 4.3])	proof of Theorem 3.16
$\Pi_i, \mathbf{vr}_i, \beta_i$	IOR message; challenge bit string; its length	§3.5
$\text{KState}_\delta, \varepsilon_i^{\text{rr}}, \varepsilon_{\text{rr}}$	knowledge state function; relaxed round-by-round errors; their maximum	Definition 3.14 and Theorem 3.16

Symbol	Meaning	Where
cm, td, CR, $\widehat{\text{CR}}$	Merkle root; trapdoor; committed and relaxed committed relations	Definition 3.15
BCS[II], ε_{ext} , ε_{com}	compiled reduction; extraction and commutativity errors	Theorem 3.16
$l_{\max}$, q_V , $q_{\widehat{\text{CR}}}$, q_s , T_1	parameters of the bound of Theorem 3.16	Theorem 3.16
<i>Kernel basis and folding</i>		
K_y , K_b , $K_b^{(n)}$	kernel polynomial; Boolean kernel polynomial; with n variables	Definition 4.1
$S_1 \otimes \cdots \otimes S_n$, $S_{(k)}$	Kronecker product; factor on bit k	Definition 4.6
C , $\mathbf{K}_n$, $\text{coef}(U)$	2×2 change of basis; coefficient matrix of (K_b) ; coefficient vector	§4.3
λ , $\lambda(b)$	kernel coordinates	Definition 4.10
φ	kernel coordinates of a low-degree polynomial with t variables	Theorem 4.13
$\gamma_k(\zeta)$, $\psi_k(\zeta)$	scalar and curve of the evaluation identity	Proposition 4.15
U_e , U_o	even and odd parts of U	Lemma 2.3 and Proposition 5.1
pfold_r , U_j	kernel fold of a polynomial; iterated folds	Definition 5.2 and Corollary 5.4
fold_r	kernel fold of a word	Definition 5.5 and Lemma 5.6
cfold_θ , θ	classical FRI fold and its challenge	Definition 5.10
<i>The commitment scheme</i>		
R , ℓ , κ	$\rho = 2^{-R}$; number of folding rounds; number of queries	§6.1
L_j , M_j , N_j , C_j	L^{2^j} ; $M/2^j$; $N/2^j$; $\text{RS}_{\mathbb{F}}[L_j, N_j]$	§6.1
U_f	kernel polynomial of a table	Definition 6.1 and Lemma 6.2
z , v	evaluation point; claimed value	§6.2
f_j , χ_j	honest restricted table; restricted equality table	§6.2, Lemma 6.6
w_0 ; w_j	commitment (instance oracle); prover oracles for $1 \leq j < \ell$, and $w_\ell = \text{ev}_{L_\ell}(G)$	§6.2
g , G	final table; its kernel polynomial	§6.2
ω , ι	generator of L ; generator of the extension field	§8
$\xi_0^{(i)}$, ξ_j	query points; $\xi_0^{2^j}$	§6.2
Π_{eval} , t_{H}	evaluation protocol; cost of one hash	§6.2, §6.5
$\mathcal{J}$, j^+ , d_j	committed levels; next committed level (or ℓ); $d_j = j^+ - j$	§6.3
$S_j(\xi)$, $\text{fold}_r^{(j)}$	coset of ξ at level j (2^{d_j} points); d_j successive folds	§6.3
$\Pi_{\text{eval}}^{\mathcal{J}}$, aug	protocol with committed levels $\mathcal{J}$; augmentation to Π_{eval}	§6.3
$\Pi_{\text{eval}}^{\mathcal{J}, \Sigma}$, $\Sigma_{\mathcal{J}}$, π	its IOR form; alphabet; order of the fibres of y	§7.8
<i>Soundness</i>		
Δ_j^{fib}	fibre distance at level j	Definition 7.1
$\text{dec}_j(w)$, f^*	decoded codeword; decoded table	Definition 7.4
$\text{Good}_j(w, r)$	r is good for w at level j	Definition 7.8
$\mathcal{P}_j$, π	passing sets; passing fraction	§7.4, Lemma 7.11
c_j	codeword chain	Lemma 7.12
$\varepsilon_{\text{fold}}$	folding error $M(1 - 2^{-\ell})/ \mathbb{F} $	Theorem 7.13

Symbol	Meaning	Where
$E_j, f_j^*, \chi_j^*, \sigma_j^*$	bad events; honest tables and claims for f^*	proof of Theorem 7.13
f^H	table decoded within Hamming distance δ	Proposition 7.15
$\hat{w}_j, \mathcal{B}_j$	$\text{fold}_{r_j}(w_{j-1})$; points where it differs from w_j	§7.6
$\lambda[c]$	kernel coordinates of P_c	§7.6
$\mathcal{W}_j(c), \text{dens}_j(c), \sigma_j(c)$	witness set; its density; claimed value for a codeword c	§7.6
s_j^c	honest round polynomial for a codeword c	Lemma 7.18
δ^*	largest proximity parameter $(1 - \rho)/2$	§7.7
ξ_η, fib_j	chosen square root of η ; fibre string of a word over $\Sigma = \mathbb{F}^2$	§7.7
$\mathcal{R}_{\text{eval}}, \Pi_{\text{eval}}^\Sigma$	evaluation relation with implicit instance; Π_{eval} as an IOR	§7.7
$\beta, \phi, \text{num}, \text{pos}$	challenge length; sampling map; enumerations of $\mathbb{F}$ and L	§7.7, Lemma 7.22
E	δ -independent extractor	Definition 7.23
$\bar{u}, \mathcal{U}(u), \mathcal{U}_i$	filling of a partial string; its erased positions; erasures of y and of the oracles	§7.7
$\mathcal{W}_j^\perp, \text{dens}_j^\perp, \mathcal{D}_\delta^\perp$	witness sets, densities and doomed set with erasures	§7.7

B Lean formalisation

The companion Lean 4 project `KBFold` (Lean 4.23.0, Mathlib at the tag `v4.23.0`; about 7200 lines in 17 modules) formalises the mathematics of Sections 2 to 7. It contains no `sorry`. Its only axiom is `bciks.unique`, the correlated-agreement theorem (Theorem 2.23); `#print axioms` shows that completeness, the fold dictionary, the basis theorem and the probability lemmas do not use it, and that soundness, binding and (relaxed) round-by-round knowledge soundness use it and the three standard axioms of Lean (`propext`, `Classical.choice`, `Quot.sound`) only.

Modelling.

- Bit vectors are functions $\text{Fin } n \rightarrow \text{Bool}$; bit k of Lean is bit $k + 1$ of the paper, and tables are functions on bit vectors. The multilinear extension is its evaluation formula (Proposition 2.6).
- One field $\mathbb{F}$ is used, and the smooth domain is a subgroup $L \leq \mathbb{F}^\times$ of order 2^μ ; the paper’s setting $L \leq \mathbb{F}_q^\times \subseteq \mathbb{F}^\times$ is a special case.
- Probabilities are rational numbers computed by counting over finite uniform spaces; rounds and challenges are indexed from 0.
- A deterministic prover of Π_{eval} is a family of messages indexed by the challenges, with an explicit causality hypothesis (message j depends on $r_1, \dots, r_{j-1}$ only) and the hypothesis that round polynomials have degree at most 2, which the paper ensures by the transmission of three values.
- Decoding is defined non-constructively, by uniqueness; the running times of extractors (Remark 7.5) and Fact 2.20 are not formalised.

Correspondence.

Paper	Lean module	Main declarations
Lemmas 2.5, 2.11 and 2.12, Proposition 2.6, Corollary 2.7, and Definition 2.10	Multilinear	eqv_pt, mle_pt, mle_eqTable, mle_res, map_mle
Lemma 2.9	Mobius	mle_eq_sum_mono, table_coeff_form
Definitions 2.14, 2.16 and 2.17 and Lemmas 2.15, 2.18, 2.19 and 2.22	Domain	IsSmoothDomain, card_powDom.and_ker, fibre, RS, RS_agree.lt, RS_unique_decoding, evenW, oddW
Theorem 2.23	BCIKS	axiom bciks_unique
Lemmas 3.2 to 3.4	Probability	sequential_challenges, independent_queries, randomised_prover_le
Definitions 3.5 to 3.7 and Lemmas 3.8, 3.9 and 3.12	RoundByRound	RBRSound, RBRKnowledge, rbr_knowledge_overall, sumcheck_sound_iop, rbr_knowledge_binding
Definitions 4.1 and 4.10 and Theorem 4.8	Kernel	kernel, ofCoords, ofCoordsLin.bijective, exists_unique_coords
Sections 4.2, 4.4 and 4.5	LowDegree, Mobius	coeff_kernel, kron_mul, Kmat_inv, coeff_ofCoords, lowDegree_iff, RS_eq_charCond, eval_ofCoords
Proposition 5.1, Definition 5.2, Theorem 5.3, and Corollary 5.4	Kernel, WordFold	pfold, fold_dictionary, pfoldList_ofCoords, pfoldSeq_ofCoords
Definitions 5.5 and 5.10, Lemmas 5.6 and 5.7, Propositions 5.8 and 5.11, and Corollary 5.9	WordFold	wfold, wfold_line, wfold_ev, codeword_fold, cfold_eq_pfold
Lemmas 6.2 and 6.6 and Theorem 6.7	Fibre, Protocol, Completeness	Enc_injective, honest_table, honS_eval, completeness
Definitions 7.1, 7.4 and 7.8 and Lemmas 7.2, 7.3, 7.6, 7.7, 7.9 and 7.10	Fibre	fibDist, fib_unique, dec, decTable, far_fold, survive, Good, ncard_not_good_le, one_step
Lemmas 7.11 and 7.12, Theorem 7.13, Corollary 7.14, and Proposition 7.15	Soundness	Pass_card, chain, chain_final, soundness_far, soundness_close, eval_binding, no_folding
Definition 7.16, Lemmas 7.17 and 7.18, Theorem 7.19, and Proposition 7.21	RBR, RBRGeneric	Doomed, rbr_step, sCw_eval, rbr_round_one, rbr_round, rbr_final, rbr_knowledge, rbr_l0
Lemmas 7.22, 7.24 and 7.27, Definition 7.23, and Theorem 7.25	NonInteractive	prob_sample_le, KState, rbr_stepE, rbr_erasures_one, rrbr_round, rrbr_final, cr_unique_core
Lemma 6.5, Theorem 7.31(1)	Skip	augW, aug, AcceptsJ, accepts_aug, accProbJ_le, aug_causal, soundness_far_J, soundness_close_J, eval_binding_J

Not formalised.

Theorem 3.16 and Corollary 7.26, which rest on the quantum analysis of Chiesa, Di, Hu and Zheng [CDHZ25]; the Merkle-tree statements (Lemma 3.13 and the collision step of Lemma 7.27); Facts 2.2 and 2.20, Lemma 2.21 and Remark 4.9; running times and operation counts; Sections 8

and 9; Lemma 6.4, and items 2–4 of Theorem 7.31. The Lean verifier of $\Pi_{\text{eval}}^{\mathcal{J}}$ states each check with the value $\text{fold}_{r_{j^+}}$ of the augmented word below j^+ , which Lemma 6.4 shows the verifier computes from the opened coset. In Lemma 7.24 and Theorem 7.25 the Lean statements give the per-round bounds for a model of transcripts by stages (empty, after j rounds, full); they do not build a general interactive-oracle-reduction structure.

Findings of the formalisation. The formalisation found no false statement. It showed that Lemma 3.8 (and hence Lemma 3.12) needs non-negative round errors, now stated in Definitions 3.6 and 3.7; that case 2 of Theorem 7.13 does not need its hypothesis $\Delta_0^{\text{fib}}(w_0, \mathcal{C}_0) \leq \delta$; and several hypotheses that are implied by the parameters (odd characteristic, $\ell \leq n + R$, $N_j \leq M_j$).